

第 9 章：Tuples, Files, and Everything Else (元组、文件及其它)

原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：本章是全书第二部分"核心对象类型"（Objects and Operations）的收官章。它补全了核心类型的最后两块拼图——tuple（元组）与 file（文件），然后站在高处总结所有对象类型的通用性质：比较（comparison）、相等（equality）、拷贝（copy）、真值（truth）。学完本章，你就拥有了理解"Python 里一切皆对象"这句话所需的全部基础。

9.1 开篇引言

英文原文

This chapter rounds out our in-depth tour of the core object types in Python by exploring the tuple, a collection of other objects that cannot be changed, and the file, an interface to external files on your computer. As you'll see, the tuple is a relatively simple object that largely performs operations you've already learned about for strings and lists. The file object is a commo

nly used and full-featured tool for processing files on a host of devices. Because files are so pervasive in programming, the basic overview of files here is supplemented by larger examples in later chapters. This chapter also concludes this part of the book by summarizing properties common to all the core object types we've met—the notions of equality, comparisons, object copies, and so on. We'll also briefly explore other object types in Python's toolbox, including the None placeholder and the namedtuple hybrid; as you'll see, although we've covered all the primary built-in types, the object story in Python is broader than implied thus far. Finally, we'll close this part of the book by taking a look at a set of common object type pitfalls and exploring some exercises that will allow you to experiment with and cement the ideas you've learned. One logistics note up front: as for strings in Chapter 7, our exploration of files here will be limited to fundamentals that most Python programmers—and especially Python newcomers—need to know. In particular, Unicode text files were previewed in Chapter 4, but we're going to postpone full coverage of them until Chapter 37, as optional or deferred reading. For this chapter's purpose, we'll assume that the contents of any text files will be encoded and decoded per your platform's default Unicode encoding (and you won't yet need to know what that means). The basics you'll learn here, though, will apply both to the simpler files in this

中文翻译

本章通过探索 tuple (元组) ——一组不能改变的对象——和 file (文件) ——访问计算机上外部文件的接口，来为我们 Python 核心对象类型的深入巡礼画上句号。你将看到，元组是一个相对简单的对象，其上的大部分操作你在学习字符串和列表时已经见过了。文件对象则是一个常用且功能完备的工具，用于处理各种设备上的文件。由于文件在编程中无处不在，这里对文件的基础介绍将由后续章节中更大的示例来补充。本章还通过总结所有已见核心对象类型的通用性质——相等 (equality)、比较 (comparisons)、对象拷贝 (object copies) 等——来为本部分 (第二部分) 收尾。我们还将简要探索 Python 工具箱中的其它对象类型，包括 None 占位符和 namedtuple (命名元组) 这个混合体；正如你将看到的，虽然我们已经覆盖了所有主要的内置类型，但 Python 中的对象故事远比到目前为止所暗示的更加广阔。最后，我们将通过检视一组常见的对象类型陷阱 (pitfalls)，并做一些练习来结束本部分，让你能够动手实验并巩固所学到的知识。先说明一点后勤上的安排：与第 7 章中关于字符串的处理方式一样，这里我们对文件的探索将限于大多数 Python 程序员——尤其是 Python 新手——需要了解的基础知识。特别是 Unicode 文本文件在第 4 章已经预览过，但我们打算把对它的完整讲解推迟到第 37 章，作为可选或延后的阅读内容。就本章而言，我们假设任何文本文件的内容都将按照你所在平台的默认 Unicode 编码进行编码和解码 (你暂时还不需要知道那意味着什么)。不过，你在这里学到的基础知识，既适用于本章的简单文件

，也适用于第 37 章中的扩展内容。

深度理解

·核心概念：本章承担"收网"角色——先补齐最后两种核心类型（元组、文件），再抽出贯穿所有类型的通用规律（比较、拷贝、真值、类型体系）。学完本章，你应该形成一张完整的心智地图：Python 里到底有哪些类型、它们各属于什么类别、哪些可变哪些不可变。

·底层视角：注意作者把 Unicode 文本文件推迟到第 37 章——这不是随意安排，而是刻意控制认知负荷。本书的教学节奏是"先掌握基础机制（str 文本、默认编码），再深入扩展细节"。你在本章写文本文件时，背后其实已经发生了 Unicode 编码/解码，只是默认行为让你无感。

·设计原因：把"对象类型巡礼"和"语句语法"分开，是因为语句（for、if、while 等）本质上只是操作对象的工具。先彻底搞懂对象，再学操作对象的语句，学起来才顺。

·实际问题：文件是程序与外部世界（磁盘、网络、用户）打交道的唯一通道；元组是不可变数据的安全载体。这两个知识点直接决定你能否写出能"落地"的程序。

·初学者误区：很多人以为 tuple 就是"只读的列表"，实际上它们的语义定位不同（元组是"数据关联"，列表是"会变化的容器"）；还有人以为文件就是字符串，忽略了文件有缓冲、位置指针（seek）等状态。

9.2 Tuples (元组)

英文原文

The last collection type in our survey is the Python tuple. Tuples construct simple groups of objects. They work much like lists, except that tuples can't be changed in place (they're immutable) and are usually written as a series of items in parentheses, not square brackets. Although they don't support as many methods, tuples share most of their properties with lists. Here's a quick look at the basics. Tuples are: Ordered collections of arbitrary objects — Like strings and lists, tuples are positionally ordered collections of objects (i.e., they maintain a left-to-right order among their contents). Like lists and dictionaries, they can embed any kind of object. Accessed by offset — Like strings and lists, items in a tuple are accessed by offset (not by key); they support all the offset-based access operations, such as indexing and slicing. Of the category "immutable sequence" — Like strings and lists, tuples are sequences; they support many of the same operations. However, like strings, tuples are immutable; they don't support any of the in-place change operations applied to lists. Fixed-length, heterogeneous, and arbitrarily nestable — Because tuples are immutable, you cannot change the size of a tuple without making a copy. On the other hand, tuples may contain any type

e of object, including other collection objects (e.g., lists, dictionaries, and other tuples), and so support arbitrary nesting. Arrays of object references — Like lists, tuples are best thought of as object reference arrays: tuples store access points to other objects (references), and indexing a tuple is relatively quick. Table 9-1 highlights common tuple operations. In it, T means a tuple. As shown, a tuple is written as a series of objects (technically, expressions that generate objects), separated by commas and normally enclosed in parentheses. An empty tuple is just a parentheses pair with nothing inside.

Table 9-1. Common tuple literals and operations
(常用元组字面量与操作)

操作	解释
()	空元组 (An empty tuple)
T = (0,)	单元素元组, 不是表达式 (A one-item tuple, not an expression)
T = (0,'Py', 1.2, 3)	四元素元组 (A four-item tuple)
T = 0,'Py', 1.2, 3	另一个四元素元组 (与上一行相同)
T = ('Pat', ('dev','mgr'))	嵌套元组 (Nested tuples)
T = tuple('hack')	由可迭代对象中的元素构成元组 (Tuple of items in an iterable)
T[i], T[i][j], T[i:j], len(T)	索引、索引的索引、切片、长度
T1 + T2、 T 3	拼接、重复 (Concatenate, repeat)
T1 > T2、 T1 == T2	比较: 大小、相等 (Comparisons: magnitude, equality)
'code' in T、 for x in T: print(x)、 [x 2 for x in T]	成员测试、迭代 (Membership, iteration)
T = (x, 0, y, x)	可迭代解包 (Iterable unpacking)

T.index('Py')、T.count('Py')	方法：搜索、计数 (Methods: search, count)
namedtuple('Emp', ['name','jobs'])	命名元组扩展类型 (Named-tuple extension type)

中文翻译

我们调查中的最后一个集合类型是 Python 的 tuple (元组)。元组构造简单的对象分组。它们的工作方式与列表非常相似，区别在于元组不能就地改变（它们是不可变的 immutable），而且通常写成圆括号中的一系列条目，而不是方括号。虽然元组不支持那么多方法，但它们与列表共享大部分性质。下面快速看一下基础。元组是：任意对象的有序集合——与字符串和列表一样，元组是位置上有顺序的对象集合（即在其内容之间保持从左到右的顺序）。与列表和字典一样，它们可以嵌入任意种类的对象。按偏移量访问——与字符串和列表一样，元组中的条目按偏移量（而不是按键）访问；它们支持所有基于偏移量的访问操作，例如索引和切片。属于"不可变序列"类别——与字符串和列表一样，元组是序列 (sequence)；它们支持许多相同的操作。然而，与字符串一样，元组是不可变的；它们不支持任何应用于列表的就地修改操作。定长、异构、可任意嵌套——因为元组是不可变的，你无法在不复制的情况下改变元组的大小。另一方面，元组可以包含任何类型的对象，包括其它集合对象（如列表、字典和其它元组），因此支持任意嵌套。对象引用数组——与列表一样，元组最好被理解为对象引用数组：元组存储指向其它对象的访问点（引用），并且索引元组相对较快。表 9-1 列出了常见的元组操作。其中 T 表示元组。如图所示，元组被写成一系列对象（严格来说，是生成对象的表达式），用逗号分隔，通常用圆括号括起来。

空元组就是一对什么也不包含的圆括号。

深度理解

·核心概念：元组 = "定长的、不可变的、有序的对象引用集合"。五个特征相互支撑：因为不可变，所以定长；因为是引用数组，所以能装任意对象、能任意嵌套。

·底层实现：CPython 中元组是 PyTupleObject，本质是一个连续的内存数组，数组里存的是指向其它对象的指针（PyObject）。索引 T[i] 就是一次指针数组取下标，所以"索引元组相对较快"；且因为没有"预留容量"的问题，元组的空间开销略小于列表（列表还需要多维护一个容量字段）。

·设计原因：元组为什么存在？因为它不可变——不可变就是安全：一个元组被传入函数、被放进字典做键、被多个变量共享，都不会被意外篡改（详见 9.6 节）。

·实际问题：元组常用于"一行记录"（record）、函数多返回值、字典键、以及 % 字符串格式化的替换值。

·初学者误区：① (40) 是整数不是元组，要写 (40,)；② "元组不可变"只保证顶层不可变，元组里的列表照样能改；③ 误以为元组没有方法，其实它至少有 index 和 count

。

9.3 Tuples in Action (元组实战)

英文原文

As usual, let's start an interactive session to explore tuples at work. Notice in Table 9-1 that tuples do not have most of the methods that lists have (e.g., an append call won't work here). They do, however, support the usual sequence operations that we explored for both strings and lists, compare recursively as usual, and support the same iterable-unpacking syntax in their literals that we used for lists in the preceding chapter:

```
$ python3 # 启动你的 REPL >> (1, 2) + (3, 4) # 拼接 (1, 2, 3, 4) >> (1, 2) * 4 # 重复 (1, 2, 1, 2, 1, 2, 1, 2) >> T = (1, 2, 3, 4) >> T[0], T[1:3] # 索引、切片 (1, (2, 3)) >> T == (1, 2, 3, 4), T > (1, 2, 3, 3), T > (1, 2, 3) (True, True, True) >> L = ['code', 'hack'] >> (L, 1, 2, (3, 4)) # 可迭代解包 ('code', 'hack', 1, 2, 3, 4)
```

中文翻译

照例，让我们启动一个交互式会话，来观察元组的实际工作。注意在表 9-1 中，元组没有列表拥有的大部分方法（例如，append 调用在这里会失败）。然而，它们确实支持我们为字符串和列表探索过的通常的序列操作，像往常一样递归地比较，并在它们的字面量中支持我们在前一章为列表使用的同样的可迭代解包语法：python # 交互式会话演示(1, 2) + (3, 4) # 拼接两个元组 -> (1, 2, 3, 4) (1, 2) * 4 # 重复元组四次 -> (1, 2, 1, 2, 1, 2, 1, 2) T = (1, 2, 3, 4) T[0], T[

1:3] # 索引拿到整数 1, 切片拿到子元组 -> (1, (2, 3)) T = (1, 2, 3, 4), T > (1, 2, 3, 3), T > (1, 2, 3) # -> (True, True, True) L = ['code', 'hack'] (L, 1, 2, (3, 4)) # 解包列表和元组的元素 -> ('code', 'hack', 1, 2, 3, 4)

代码分析

·解释器如何处理：当解释器编译 $(1, 2) + (3, 4)$ 时，首先生成两个元组常量对象，然后执行拼接字节码；CPython 中元组拼接会分配一块新的内存，把两个元组引用数组拷贝拼接进去，所以结果 $(1, 2, 3, 4)$ 是一个全新对象，内存原地址的两个源元组不变。 $(1, 2) * 4$ 同理，是构造新元组。这就是“不可变类型每次操作都产生新对象”的铁律。

·递归比较： $T > (1, 2, 3, 3)$ 结果为 True——解释器从左到右逐项比较，T 的前三项 1、2、3 都相等，到第四项 $3 > 3$ 为 False，结果本应为 False？实测为 True，因为 $T = (1, 2, 3, 4)$ 的第四项是 4， $(1, 2, 3, 4) > (1, 2, 3, 3)$ ，第四项 $4 > 3$ 才判定 T 更大，输出 True。而 $T > (1, 2, 3)$ ：前序项全等，但 T 更长（还有第四项），Python 规定长序列结论为“大”。

·解包语法： $(L, 1, 2, (3, 4))$ 用了 PEP 448 的“任意位置解包”（iterable unpacking）。解释器会把 L 迭代展开为三个元素、把 $(3, 4)$ 展开为两个元素，最终构造出一个含 6 个元素的元组。注意问题：结果是一个元组，因为整体用了圆括号字面量语法——即便来源是列表。这说明了“圆括号决定元组，方括号决定列表”。

深度理解

- 核心概念：元组的核心操作（索引、切片、拼接、比较、迭代）与字符串/列表完全同构，因为三者同属序列（sequence）类别，共享同一套序列协议。
- 底层实现：元组拼接/重复在 C 层执行 tupleconcat / tuplerepeat，返回新对象；而元组比较会调用序列比较协议（PyObjectRichCompare），递归进嵌套结构。元组索引走 PyTupleGETITEM，是极快的宏（不做完整性检查），这也是“索引元组相对较快”的来源。
- 设计原因：不可变类型为何“每次生成新对象”？因为修改自身会破坏“共享安全”承诺——既然不可变，唯一的“改变”方式就是造个新的。
- 实际问题：凡是需要“取走即断链”的场景（如把一行数据的字段打包返回），元组都很合适；返回 (status, result) 之类的成对结果是 Python 惯例。
- 初学者误区：在 REPL 里同时输入多个表达式并用逗号隔开，会得到元组 (结果1, 结果2, ...)——本例中 T[0], T[1:3] 就输出了 (1, (2, 3))，初学者常把它误认为是“语法糖”而非“隐式元组构造”。

9.4 Tuple Syntax Peculiarities: Commas and Parentheses（元组语法特性：逗号与括号）

英文原文

The second and fourth entries in Table 9-1 merit a bit more explanation. Because parentheses can also enclose expressions (see Chapter 5), you need to do something special to tell Python when a single object in parentheses is a tuple object and not a simple expression. If you really want a single-item tuple, simply add a trailing comma after the single item, before the closing parenthesis:

```
>>> x = (40) # 一个整数! >>> x 40
```

```
>>> y = (40,) # 一个包含整数的元组>>> y (40,)
```

As a special case, Python also allows you to omit the opening and closing parentheses for a tuple in contexts where it isn't syntactically ambiguous to do so. For instance, the fourth line of Table 9-1 simply lists four items separated by commas. In the context of an assignment statement, Python recognizes this as a tuple, even though it doesn't have parentheses. That's why all the comma-separated items we've typed at the REPL print with parentheses—it's a tuple:

```
>>> 1, 2, 3, 4 # 在 REPL 和其它地方, 没有括号的元组(1, 2, 3, 4)
```

This syntactic trick is also commonly leveraged by the sequence assignment shorthand we used briefly in Chapter 7 and will study in earnest in Chapter 11—names on the left are paired with values on the right and

d assigned by position, but both sides are really tuples without parentheses:

```
>>> a, b, c = 1, 2, 3 # 序列赋值: 两边都是元组>>> a, b, c (1, 2, 3)
```

Now, some people will tell you to always use parentheses in your tuples, and some will tell you to never use parentheses in tuples (and still others have lives and won't tell you what to do with your tuples!). The most common places where the parentheses are required for tuple literals are those where: Parentheses matter—within a function call, or nested in a larger expression Commas matter—within a function call, or embedded in the literal of a larger object like a list or dictionary In most other contexts, the enclosing parentheses are optional. For beginners, the best advice is that it's probably easier to use the parentheses than it is to remember when they are optional or required. Many programmers also find that parentheses tend to aid script readability by making the tuples more explicit and obvious. And for language lawyers in the audience, bear in mind that the comma is really a sort of lowest precedence operator, though only in contexts where it's not otherwise significant. In such contexts, it's the comma that builds tuples, not the parentheses. This makes the latter optional, but can also lead to odd, unexpected syntax errors if parentheses are omitted (e.g., in lambda covered in Part IV). Adding parentheses to your tuples as a habit avoids the odditi

中文翻译

表 9-1 中的第二个和第四个条目值得再多解释一些。因为圆括号也可以括住表达式（见第 5 章），你需要做一些特别的事情来告诉 Python：圆括号中的单个对象是一个元组对象，而不是一个简单表达式。如果你真的想要一个单元素元组，只需在单个条目之后、右括号之前加一个尾随逗号：
`python x = (40) # 这只是整数 40 y = (40,) # 这是包含整数 40 的元组`作为特例，Python 还允许你在语法上不会产生歧义的上下文中省略元组的左右括号。例如，表 9-1 第四行只是列出了四个用逗号分隔的条目。在赋值语句的上下文中，Python 会把它识别为一个元组，即使它没有括号。这就是为什么我们在 REPL 中输入的所有逗号分隔条目都会用圆括号打印出来——它是一个元组：`python 1, 2, 3, 4 # REPL 中结果是 (1, 2, 3, 4)` 这个语法技巧也常被序列赋值（sequence assignment）简写所利用——我们曾在第 7 章简单用过，并将在第 11 章认真研究——左边的名字与右边的值按位置配对并赋值，但两边其实都是没有括号的元组：`python a, b, c = 1, 2, 3 # 序列赋值：两边都是元组` `# a b c 的结果是 (1, 2, 3)` 现在，有人会告诉你元组里永远要用括号，也有人会告诉你元组里永远不要用括号（还有人生活丰富，根本懒得管你元组怎么用！）。对元组字面量来说，括号必须出现的常见位置是这些：括号要紧——在函数调用内部，或嵌套在更大的表达式中逗号要紧——在函数调用内部，或嵌入在更大的对象（如列表或字典）的字面量中在大多数其它上下文中，包围的括号是可有可无的。对初学者来说，最好的建议是：用括号比记住它何时可选、何时必需

要来得容易。许多程序员也发现，括号往往能提高脚本的可读性，让元组更显式、更一目了然。另外，给语言律师 (language lawyer) 读者提个醒：请记住，逗号实际上是一种最低优先级运算符 (lowest precedence operator)，尽管只在它不被其它地方使用时才如此。在这样的上下文中，构建元组的是逗号，而不是括号。这使得后者可选，但如果是省略括号，也可能导致奇怪、意外的语法错误（例如在第 IV 部分要讲的 lambda 中）。把给元组加括号养成习惯，就能避开这些怪癖。

代码分析

·解释器如何处理 (40) 和 (40,): 解析阶段 (parser) 遇到 (40) 时，圆括号被识别为"表达式分组"而无效果→结果就是整数 40；遇到 (40,) 时，字节码是 LOADCONST 40; BUILDTUPLE 1，构造出一个含 1 个元素的元组。尾随逗号是元组的"身份证"。

·解释器如何处理 1, 2, 3, 4: parse 阶段把逗号序列识别为元组构造，同样编译为 BUILDTUPLE 4。赋值 a, b, c = 1, 2, 3 左侧被编译为 UNPACKSEQUENCE 3，右侧先构造含 3 个元素的元组，再按位置解包——这正是第 7 章序列赋值 (swap 技巧 a, b = b, a) 的底层机制。

·设计思想：Python 选择"逗号建元组、括号可选"是为了减少键盘符号——多值赋值、函数返回多个值都不必敲括号。代价是语法有歧义风险，所以实践中建议总是写圆括号。

深度理解

·核心概念：元组字面量的本质规则是——逗号是构造者，括号只是分组器（可选的外壳）。理解这一点， (x) 与 $(x,)$ 的区别、 $a, b = 1, 2$ 的原理都迎刃而解。

·底层实现：语法分析器 (CParser) 在 "comma 上下文" 里把逗号优先级降到最低（比赋值还低），从而允许 $x = 1, 2$ 生效。这是语法层面刻意设计的 "最低优先级运算符"。

·设计原因：呼应 Python 之禅 "可读、少按键"；但括号省略在 $\text{lambda } x: (x, x)$ 这种场景必须强制，否则 $\text{lambda } x: x, x$ 会先求值 lambda 再拼逗号，完全不是本意。

·实际问题：多变量同时返回/交换值的惯用法都建立在 "隐式元组" 之上；函数调用里 $f((1,2))$ 与 $f(1,2)$ 含义完全不同（前者传一个元组，后者传两个参数）——这是最经典的传参陷阱。

·初学者误区：把 (40) 当元组；认为 $(1,2,3)$ 的括号不可省（其实 $x = 1,2,3$ 同样成立）；在 $\text{return } 1, 2$ 与 $\text{return } (1, 2)$ 之间纠结——它们完全相同。

9.5 Conversions, Methods, and Immutability (转换、方法与不可变性)

英文原文

Apart from literal-syntax differences, tuple operation

s (the middle rows in Table 9-1) are identical to string and list operations. The only differences worth noting are that the `+`, `,` and slicing operations return new tuples when applied to tuples, and that tuples don't provide the same methods you saw for strings, lists, and dictionaries.

If you want to sort a tuple, for example, you'll usually have to either first convert it to a list to gain access to a sorting method call and make it a mutable object, or use the newer sorted built-in that accepts any sequence object (and other iterables—a term introduced in Chapter 4 that we'll be more formal about in the next part of this book):

```
>> T = ('cc','aa','dd','bb') >> tmp = list(T) # 把元组的元素做成一个列表
>> tmp.sort() # 对列表排序
>> tmp ['aa','bb','cc','dd']
>> T = tuple(tmp) # 把列表的元素做成一个元组
>> T ('aa','bb','cc','dd')
>> sorted(T, reverse=True) # 或者直接用 sorted 内置函数, 省去几步
['dd','cc','bb','aa']
```

Here, the list and tuple built-in functions are used to convert the object to a list and then back to a tuple. Really, both calls make new objects from any sort of iterables, but the net effect is like a conversion.

In some sense, list comprehensions can also be used to convert tuples. The following, for example, makes a list from a tuple, adding 20 to each item along the way:

```
>> T = (1, 2, 3, 4, 5) >> L = [x + 20 for x in T] # 相当于 list(T) + 表达式逻辑 >> L [21, 22, 23, 24, 25]
```

List comprehensions are really sequence operations—they always build new lists, but they may be used to iterate over any sequence objects, including tuples, strings, and other lists. As you'll see later in the book, they even work on some things that are not physically stored sequences—any iterable objects will do, including files, which are automatically read line by line. Given this, they may be better called iteration tools.

Notice that you'd have to convert the prior example's list result back to a tuple if your code must care. There is no tuple comprehension in Python (as explored later in this book, parenthesized comprehensions make generators), though you simulate one by using tuple to force a generator to give up its values:

```
>> tuple(x + 20 for x in T) # 元组"推导式" = 生成器 + 构建器 (21, 22, 23, 24, 25)
```

Although tuples don't have the same methods as lists and strings, they do have two of their own—index and count work as they do for lists, but they are defined for tuple objects:

```
>> T = (1, 2, 3, 2, 4, 2) # 元组方法 >> T.index(2) # 2  
第一次出现的偏移量: index of 2! 1 >> T.index(2, 2) #  
偏移量 2 之后 2 的出现偏移 3 >> T.count(2) # 有多少个  
2? 3
```

Also, note that the rule about tuple immutability applies only to the top level of the tuple itself, not to its contents. A list inside a tuple, for instance, can be changed as usual:

```
>> T = (1, [2, 3], 4) >> T[1] = 'mod' # 失败: 不能改变元组本身
TypeError: 'tuple' object does not support item assignment
>> T[1][0] = 'mod' # 可行: 能改变其中的可变对象
>> T (1, ['mod', 3], 4)
```

For most programs, this one-level-deep immutability is sufficient for common tuple roles. Which, coincidentally, brings us to the next section.

中文翻译

除了字面量语法的差异，元组操作（表 9-1 中间几行）与字符串和列表操作完全一致。唯一值得注意的区别是：+、和切片操作应用于元组时返回新的元组，并且元组不提供你在字符串、列表和字典上见过的那些方法。例如，如果你想对元组进行排序，通常必须先把它转换成列表，以获得排序方法调用并使之成为可变对象，或者使用更新的 sorted 内置函数——它接受任何序列对象（以及其它 iterable（可迭代对象）——这个词在第 4 章引入，在本书下一部分我们会更正式地讲解）：python T = ('cc','aa','dd','bb') tmp = list(T) # 由元组元素建列表 tmp.sort() # 列表降序排序后 -> ['aa','bb','cc','dd'] T = tuple(tmp) # 由列表元素回建元组 -> ('aa','bb','cc','dd') sorted(T, reverse=True) # 或直接用 sorted 内置函数 -> ['dd','cc','bb','aa'] 这里，list 和 tuple 内置函数被用来把对象转换成列表，再转换回元组。实际上，这两个调用都会从任意可迭代对象生成新对

象，但净效果就像一次转换。从某种意义上说，列表推导式（list comprehension）也可以用来转换元组。例如下面的代码从元组生成一个列表，途中每个条目都加上 20：python T = (1, 2, 3, 4, 5) L = [x + 20 for x in T] # 相当于 list(T) 后再做加法逻辑 -> [21,...] 列表推导式其实都是序列操作——它们总是构建新列表，但可以被用来遍历任何序列对象，包括元组、字符串和其它列表。正如你将在本书后面看到的，它们甚至可以在一些并非物理存储序列的东西上工作——任何可迭代对象都可以，包括文件（文件会被自动逐行读取）。因此，或许把它们叫做迭代工具（iteration tools）更贴切。注意：如果你的代码必须在意类型，就得把前面示例的列表结果转换回元组。Python 中没有元组推导式（正如本书后面将要探讨的，加括号的推导式产生的是生成器 generator），不过你可以用 tuple 强制生成器交出它的值来模拟一个：python tuple(x + 20 for x in T) # 元组"推导式" = 生成器 + 构建器 -> (21,...,25) 尽管元组没有列表和字符串那套方法，它们确实有自己的两个方法——index 和 count，用法与列表相同，但它们是针对元组对象定义的：python T = (1, 2, 3, 2, 4, 2) T.index(2) # 数字 2 第一次出现的偏移量 -> 1 T.index(2, 2) # 从偏移量 2 之后，2 首次出现的偏移量 -> 3 T.count(2) # 2 一共出现几次 -> 3 另外，请注意：关于元组不可变性的规则只适用于元组本身的顶层（top level），而不适用于它的内容。例如，元组里的一个列表可以照常被修改：python T = (1, [2, 3], 4) T[1] = 'mod' # 会失败：TypeError，不能修改元组本身 T[1][0] = 'mod' # 可以：能修改元组内部的可变对象 # T 现在是 (1, ['mod', 3], 4) 对大多数程序来说，这种"一层深的不可变性"对于常见的元组角色已经足够了。而这

恰好把话题引向了下一节。

代码分析

·解释器如何处理转换：list(T) 编译为对内置类型的调用，CPython 执行 PyListFromObject，它调用元组的 iter 协议，逐个把元素指针拷入新列表数组；tuple(tmp) 走 PyTupleNew + 拷贝。它们本质是"迭代 + 拷贝引用"，不是深拷贝——元素本身没有复制。

·解释器如何处理列表推导式：[x + 20 for x in T] 在内存里等价于内联的 for 循环 + append；Python 3 中推导式拥有独立作用域，x 不会泄漏到外部命名空间。每次迭代执行 BINARYADD，结果 append 进列表。

·生成器模拟元组推导式：tuple(x + 20 for x in T) —— 括号内其实是生成器表达式 (generator expression)，此时它不构造任何元素，而是惰性求值；tuple() 强制生成器协程逐次 yield 出值并收集成元组。它比列表推导式省内存（不建中间列表），但生成器只允许迭代一次。

·不可变性的真相：T[1] = 'mod' 触发了 PyTupleSetItem 的条件检查——元组对象设置了不可变的标志，直接抛 TypeError；而 T[1][0] = 'mod' 是"先取出 T[1] 这个列表对象的引用，再对它调用列表的 setitem"，与元组本身无关，自然成功。不可变只锁顶层，不锁内容。

深度理解

·核心概念：元组的方法面 (method set) 只有 index/count，其余序列能力全部来自"协议"而非方法列表——这

是 Python"操作由协议驱动"设计哲学的体现。

- 底层实现：排序时元组不可变，只能转列表或走 `sorted`；`sorted` 是内置函数，内部构建排序用临时结构后返回新列表，元组本身从不被触碰。

- 设计原因：为什么没有元组推导式？因为推导式天然"就地累积构建"，与不可变类型冲突；括号形式另作它用（生成器）。这是语法资源的一种取舍。

- 实际问题：把一组异构字段打包成元组返回、用 `index/count` 在固定数据上做查找统计，是元组最常见的应用；而"元组内嵌列表可变"被广泛用于"可变的函数默认参数替身"等技巧。

- 初学者误区：以为元组完全不可改（其实内部可变对象可改）；以为 `tuple('hack')` 与 `('hack',)` 一样（前者是 `('h','a','c','k')`——逐个可迭代解包，后者是一整个字符串的元组）。

9.6 Why Lists and Tuples? (为什么需要列表和元组?)

英文原文

This seems to always be the first question that comes up when teaching beginners about tuples: why do we need tuples if we have lists? Some of the reason is philosophical: a tuple is meant to be a simple association of objects, while a list is intended to be a data struc

ture that changes over time. In fact, this meaning of "tuple" derives from mathematics, as well its frequent use for a row in a relational database table. The best answer, however, seems to be that the immutability of tuples provides some integrity—you can be sure a tuple won't be changed through another reference elsewhere in a program, but there's no such guarantee for lists. Tuples and other immutables, therefore, serve a similar role to "constant" declarations in other languages, though the notion of constantness is associated with objects in Python, not variables. Tuples can also be used in places that lists cannot—for example, as dictionary keys (see the sparse matrix example in Chapter 8). Some built-in operations may also require or imply tuples instead of lists (e.g., the substitution values in the % string formatting expression of Chapter 7), though some operations have often been generalized to be more flexible. As a rule of thumb, lists are the tool of choice for ordered collections that might need to change; tuples can handle the other cases of fixed associations.

中文翻译

给初学者讲元组时，这似乎总是被第一个提出的问题：既然我们有列表，为什么还需要元组？部分原因是哲学性的：元组意在成为对象的简单关联（association），而列表旨在成为随时间变化的数据结构。事实上，"tuple（元组）"这个词的含义源自数学，也源自它在关系数据库表中常被用作

一行的场景。然而，最好的答案似乎是：元组的不可变性提供了一些完整性（integrity）保障——你可以确信一个元组不会通过程序其它地方的另一个引用被改变，但对列表没有这样的保证。因此，元组和其它的不可变对象承担着类似其它语言中"常量"声明的作用，只不过在 Python 中，"恒定性"的概念是与对象（objects）关联的，而不是与变量关联。元组还可以用在列表不能用的地方——例如，作为字典键（参见第 8 章的稀疏矩阵示例）。一些内置操作也可能要求或暗示使用元组而非列表（例如，第 7 章 % 字符串格式化表达式中的替换值），尽管有些操作已经常常被泛化得更加灵活。作为经验法则：列表是那些可能需要改变的、有序集合的首选工具；元组则能处理固定的关联这种其余情况。

深度理解

·核心概念："list vs tuple"的取舍本质是可变性换安全保障 vs 灵活性换动态能力。元组卖点是"绝不会被悄悄改掉"的完整性，列表卖点是"随时可增删改"的灵活性。

·底层视角：在 Python 中，常量性（constantness）绑定在对象而非变量上——变量名只是标签，可以随时重新指向新对象；被永久保护的是那一个不可变对象本身。这与 C 语言 const（修饰变量）形成鲜明对比。

·设计原因：元组可作字典键、可作集合元素，本质都是"需要对象可哈希（hashable）"——不可变对象一旦算出哈希值就永远有效，而可变列表的哈希会随内容改变而失效。这是 Python 把"可哈希"与"不可变"绑定的根本原因。

·实际问题：坐标 (x, y)、RGB 颜色、配置项“键值对”、“函数返回的固定记录”都用元组；而需要收集、筛选、排序的动态数据用列表。 $m = [[0]4]4$ 这类矩阵也可用 $\{(r,c): \text{value}\}$ 稀疏字典，键就用元组。

·初学者误区：误以为“元组比列表快所以多用作性能优化”——其实多数场景性能差异可忽略；真正决定选型的是数据是否需要改变和是否要做字典键。

9.7 Records Revisited: Named Tuples (再看记录：命名元组)

英文原文

In fact, the choice of data types is even richer than the prior section may have implied—Python programmers can choose from an assortment of both built-in core types, and extension types built on top of them.

For example, in the prior chapter's sidebar "Why You Will Care: List Versus Dictionary Versus Set", we saw how to represent record-like information with both a list and a dictionary and noted that dictionaries offer the advantage of more mnemonic keys that label data. As long as we don't require mutability, tuples can serve similar roles, with positions for record fields like lists:

```
>> pat = ('Pat', 40.5, ['dev','mgr']) # 元组记录 >> pat
('Pat', 40.5, ['dev','mgr']) >> pat[0], pat[2] # 按位置访
```

问 ('Pat', ['dev','mgr'])

As for lists, though, field numbers in tuples generally carry less information than the names of keys in a dictionary. To review, here's the same record recoded as a dictionary with named fields:

```
>> pat = dict(name='Pat', age=40.5, jobs=['dev','mgr']) # 字典记录
>> pat['name'], pat['jobs'] # 按键访问 ('Pat', ['dev','mgr'])
```

In fact, we can convert parts of the dictionary to tuples if needed:

```
>> tuple(pat.values()) # 值转元组 ('Pat', 40.5, ['dev','mgr'])
>> list(pat.items()) # 键值对转元组列表 [('name', 'Pat'), ('age', 40.5), ('jobs', ['dev','mgr'])]
```

But really, this is a false dichotomy: with extra code, we can implement objects that offer both positional and named access to record fields.

For example, the `namedtuple` utility, noncore but always available in the standard library's `collections` module, implements an extension type that adds logic to tuples that allows components to be accessed by both position and attribute name, and can be converted to dictionary-like form for access by key if desired.

Attribute names come from classes and are not exactly dictionary keys, but they are similarly mnemonic:

```
>> from collections import namedtuple # 导入扩展类型
>> Rec = namedtuple('Rec', ['name','age','jobs']) #
```

生成一个类 >> pat = Rec('Pat', age=40.5, jobs=['dev','mgr']) # 一条命名元组记录 >> pat Rec(name='Pat', age=40.5, jobs=['dev','mgr']) >> pat[0], pat[2] # 按位置访问 ('Pat', ['dev','mgr']) >> pat.name, pat.jobs # 按属性访问 ('Pat', ['dev','mgr'])

Converting to a dictionary also supports key-based behavior when needed:

```
>> D = pat.asdict() # 转字典形式 >> D['name'], D['jobs'] # 也能按键访问 (('Pat', ['dev','mgr']) >> D {'name': 'Pat', 'age': 40.5, 'jobs': ['dev','mgr']})
```

As you can see, named tuples are a tuple/class/dictionary hybrid. They also represent a classic trade-off. In exchange for their extra utility, they require extra code to use (the two startup lines in the preceding examples that import the type and make the class) and incur some performance costs to work this magic.

Still, they are an example of the kind of custom data types that we can build on top of built-in types like tuples when extra utility is desired. They are also extensions, not core types—they live in the standard library and fall into the same category as Chapter 5's `Fraction` and `Decimal`—so we'll delegate to the Python library manual for more details.

Watch for a final rehash of this record representation thread when we explore how user-defined classes compare in Chapter 27. As you'll find there, classes label fields with names too, but can also provide program

logic to process the record's data in the same code package.

中文翻译

事实上，数据类型的选择比上一节可能暗示的还要丰富——Python 程序员既可以选择各种内置核心类型，也可以选择构建在它们之上的扩展类型。例如，在前一章的边栏“你在意什么：列表、字典还是集合”中，我们看到如何用列表和字典两者来表示类似记录的信息，并指出字典的优势在于：它有更具助记性的键来标记数据。只要不需要可变性，元组也可以充当类似的角色，像列表一样用位置来表示记录字段：`python pat = ('Pat', 40.5, ['dev','mgr'])` # 元组记录，字段 0/1/2 `pat[0], pat[2]` # 按位置访问 -> `('Pat', ['dev','mgr'])` 不过，和列表一样，元组里的字段编号携带的信息通常不如字典键名多。来回顾一下：同一个记录用带命名字段的字典重写是这样的：`python pat = dict(name='Pat', age=40.5, jobs=['dev','mgr'])` # 字典记录 `pat['name'], pat['jobs']` # 按键访问事实上，如果需要，我们可以把字典的部分内容转成元组：`python tuple(pat.values())` # 值转元组 -> `('Pat', 40.5, ['dev','mgr'])` `list(pat.items())` # 键值对转元组列表 -> `[('name','Pat'), ('age',40.5), ('jobs',['dev','mgr'])]` 但实际上，这是一个错误的二分法：借助额外的代码，我们可以实现既提供位置访问又提供命名访问的记录字段的类。例如，`namedtuple` 工具——它并非核心类型，但在标准库的 `collections` 模块中始终可用——实现了一种为元组添加逻辑的扩展类型，允许组件既按位置又按属性名访问，并且如果需要，还可以转换成类似字典的形式以按键访问。属性名来自类（`class`），与字典键不完全是同

一种东西，但同样是助记性的：python from collections import namedtuple # 导入扩展类型 Rec = namedtuple('Rec', ['name','age','jobs']) # 生成一个类 Rec pat = Rec('Pat', age=40.5, jobs=['dev','mgr']) # 一条命名元组记录 # pat -> Rec(name='Pat', age=40.5, jobs=['dev','mgr']) pat[0], pat[2] # 按位置访问 -> ('Pat', ['dev','mgr']) pat.name, pat.jobs # 按属性名访问 -> ('Pat', ['dev','mgr']) 转换为字典也支持在需要时的基于键的行为：python D = pat.asdict() # 类似字典的形式 D['name'], D['jobs'] # 也能按键访问 # D -> {'name':'Pat','age': 40.5,'jobs': ['dev','mgr']} 如你所见，命名元组是一个元组/类/字典的混合体 (hybrid)。它们也代表一种经典的权衡 (trade-off)：为了换来额外的实用性，它们需要额外的代码来使用（前面示例中导入类型、制作类的那两行启动代码），并为施展这种魔法付出一些性能代价。即便如此，它们是那种“当我们想要额外实用性时，可以构建在内置类型（如元组）之上的自定义数据类型”的范例。它们也是扩展 (extensions)，而非核心类型——它们住在标准库里，与第 5 章的 Fraction 和 Decimal 属于同一类别——至于更详细的资料，我们委托给 Python 库手册。当我们研究第 27 章中用户自定义的类 (classes) 如何比较时，注意这个记录表示话题将迎来最后的重述。你会在那里发现：类也用名字来标记字段，但还可以在同一个代码包中提供逻辑 (logic) 来处理记录的数据。

代码分析

·解释器如何处理 namedtuple: namedtuple('Rec', ['name','age','jobs']) 是一个工厂函数——它调用 type() 在

运行时动态生成一个新类（这是元编程：type 是类的类）。生成的 Rec 类继承 tuple，同时用 slots 建立只读属性；pat[0] 走元组的下标协议，pat.name 走属性访问协议（内部其实是按字段名映射到元组偏移）。因此它是"元组 + 轻量类"的真正混合。

·底层机制：asdict() 调用 OrderedDict 风格地把每个字段名与对应位置值组合成键值对；字段名硬编码在生成的类里。因为 Rec 是 tuple 的子类，它依旧不可变、依旧可哈希、依旧能当字典键。

·设计思想：namedtuple 解决了"字典太啰嗦、位置上没语义、手写类太笨重"三难问题，是"标准库足够好"哲学的样板——一行工厂调用换来兼具三者优点的类型，代价仅是启动代码和略慢的属性访问。

深度理解

·核心概念：命名元组 = 元组的不可变 + 字典的命名可读 + 类的属性访问。它证明了 Python 的类型不是铁板一块，可以在核心类型之上"走出"新类型。

·底层实现：属性访问 pat.name 底层是 getattr/getattribute 通过继承关系找到描述符 (descriptor)，比元组直接下标多一次查找，所以有轻微性能开销——这就是书中所说的性能代价。

·设计原因：标准库把 namedtuple 放在 collections 而非语言核心，体现了"核心保持精简，扩展通过导入获得"的分层设计。

·实际问题：解析 CSV/DAT 记录、返回多字段查询结果、配置项访问都常用 `namedtuple`；Python 3.7 以后还可使用 `dataclass`（更重、更灵活），但其定位不同——`namedtuple` 体积小、不可变、可哈希、解包方便。

·初学者误区：忘记要 `import`（它不在内建名字里）；误用可变字段初值导致共享 bug（`Rec('x', [], [])` 每次都要显式传新列表）；混淆 `namedtuple` 与正常类——它不能加方法定义，后来的 `dataclass` 才有更开放的空间。

9.8 Files (文件)

英文原文

You may already be familiar with the notion of files, which are named storage compartments on your PC, phone, or other computer that are managed by your operating system. The last major built-in object type that we'll examine on our object-types tour provides a way to access those files inside Python programs. In short, the built-in `open` function creates a Python file object, which serves as a link to a file residing on your device. After calling `open`, you can transfer strings of data to and from the associated external file by calling the returned file object's methods. Compared to the types you've seen so far, file objects are outliers. They are considered a core type because they are created by a built-in function, but they're not numbers, sequences, or mappings, and they don't respond to

expression operators; they export only methods for common file-processing tasks. Most file methods are concerned with performing input from and output to the external file associated with a file object, but other file methods allow us to seek to a new position in the file, flush output buffers, and so on. Table 9-2 summarizes common file operations.

Table 9-2. Common file operations (常用文件操作)

操作	解释
<code>output = open(r'C:\data','w')</code>	创建输出文件 (Windows 路径, 'w' = 写)
<code>input = open('/home/me/data','r')</code>	创建输入文件 (Unix 路径, 'r' = 读)
<code>input = open('data')</code>	创建输入文件 (当前目录, 'r' 是默认)
<code>aString = input.read()</code>	把整个文件读进单个字符串
<code>aString = input.read(N)</code>	把开头的 N 个字符 (或字节) 读进字符串
<code>aString = input.readline()</code>	把下一行 (包括 \n 换行符) 读进字符串
<code>aList = input.readlines()</code>	把整个文件读进一行一字符串的列表 (含 \n)
<code>output.write(aString)</code>	把字符串 (字符或字节) 写进文件
<code>output.writelines(aList)</code>	把列表中的所有行字符串原样写进文件
<code>output.close()</code>	手动关闭 (对象被回收时会自动完成)
<code>output.flush()</code>	不清除连接, 把输出缓冲刷到磁盘
<code>anyFile.seek(N)</code>	把文件位置改到偏移量 N, 供下一次操作使用
<code>for line in open('data'): use line</code>	文件迭代器逐行读取
<code>open('f.txt', encoding='utf-8')</code>	Unicode 文本文件 (用 str 字符串)
<code>open('f.bin','rb')</code>	字节文件 (用 bytes 字符串)

中文翻译

你可能已经熟悉文件的概念——文件是你的 PC、手机或其它电脑上由操作系统管理的、有名称的存储分区。在我们对象类型之旅中要考察的最后一个主要内置对象类型，提供了一种在 Python 程序内部访问这些文件的方式。简言之，内置的 `open` 函数会创建一个 Python 文件对象（file object），它充当通向设备上某个文件的链接。调用 `open` 之后，你可以通过调用返回的文件对象的方法，把数据字符串传入或传出关联的外部文件。与你到目前为止见过的类型相比，文件对象是局外人（outliers）。它们之所以被认为是核心类型，是因为它们由内置函数创建；但它们不是数字、序列或映射，也不响应表达式运算符；它们只导出用于常见文件处理任务的方法（methods）。大多数文件方法关心的是对与文件对象关联的外部文件执行输入和输出，但其它文件方法允许我们在文件中 `seek`（跳到）一个新位置、`flush`（刷新）输出缓冲区等等。表 9-2 总结了常见的文件操作。

深度理解

- 核心概念：文件对象与数字/序列/映射完全不同——它是状态化（有位置指针、有缓冲）的 I/O 门面，只通过方法交互，不通过运算符。这提醒我们：“内置类型”并不要求“长得像序列”，核心类型的判据是“由语言内建入口创建”。
- 底层实现：文件对象的本质是操作系统文件描述符（file descriptor，整数）的包装；Python 在它和系统调用（`read/write/lseek/fsync`）之间加了一层 `io` 模块的缓冲逻

辑。读写出名是"字节流", 文本模式则多一次编码/解码。

·设计原因: 文件只用方法、不用运算符, 是因为 I/O 操作 (读写定位刷新) 天然是"动作"而非"表达式运算"; 对象模型并不追求把一切抽象成运算符。

·实际问题: 任何持久化、日志、配置、数据交换都离不开文件; open 是 Python 程序与外部世界交流的第一道门。

·初学者误区: 以为文件对象"像字符串/列表", 试图用 len(file) 或 file[0] 访问内容; 文件必须先 read 进来才能当数据用。另外"关闭文件"在 CPython 里多数可省略 (自动回收), 但养成 close 习惯才稳妥。

9.9 Opening Files (打开文件)

英文原文

To open a file, a program calls the built-in open function, with the external filename first, followed by a processing mode. Both arguments are strings. The call returns a file object, which in turn has methods for data transfer: `afile = open(filename, mode)` `afile.method()` The first argument to open, the external filename, may include a platform-specific and absolute (complete) or relative (partial) directory-path prefix that identifies a file's location in the host device's filesystem. The filesystem is just a hierarchy of folders that stores files and nested folders. A filename `a/b/file.txt`, for

example, includes the path prefix `a/b` that leads to a file on Unix (e.g., macOS, Linux, or Android), and `a\b\file.txt` does the same on Windows. If filename has no directory-path prefix at all, the file is assumed to exist in, and hence is relative to, the current working directory (CWD). The CWD is the folder from which a script is run (e.g., where you are in a console when you launch a Python command). In a REPL, the CWD is wherever you are working at the time; to see what the CWD is, run a `pwd` in most system shells, or Python's `os.getcwd()` after `import os` in a REPL. As you'll see in Chapter 37's expanded file coverage, the filename may also contain non-ASCII Unicode characters that Python automatically translates to and from the underlying host's encoding. These characters may be provided in the filename literally, or in a pre-encoded byte string that you'll learn about later in this book. The second argument to `open`, processing mode, is typically the string `'r'` to open for text input (the default), `'w'` to create and open for text output, or `'a'` to open for appending text to the end (e.g., for adding to logfiles). The processing mode argument can specify additional options:- Adding a `b` to the mode string (e.g., `'wb'`) allows for processing binary file content. End-of-line translations and Unicode encodings used for text are turned off.- Adding a `+` to the mode (e.g., `'r+'`) opens the file for both input and output. You can read and write to the same file object, often in conjunction with `s`

seek operations to reposition in the file. Both of the first two arguments to open must be Python strings. An optional third argument takes an integer to control output buffering—passing a zero means that output is unbuffered (it's transferred to the external file immediately on a write method call), and additional arguments may be provided for special types of files (e.g., the string name of a Unicode encoding for text files). You can also use name=value keywords to pass open arguments (e.g., file=name, mode=mode), though this is somewhat above our pay grade in this chapter. We'll cover file fundamentals and explore some basic examples here, but we won't go into all file-processing mode options; run help(open) in a REPL or consult the Python library manual for additional details.

中文翻译

要打开一个文件，程序调用内置的 open 函数，外部文件名在前，处理模式（processing mode）在后。两个参数都是字符串。该调用返回一个文件对象，它再拥有用于数据传输的方法：`python afile = open(filename, mode) # 打开文件，得到文件对象afile.method() # 再调用文件对象的方法` open 的第一个参数——外部文件名——可以包含一个平台特定且绝对（完整）或相对（部分）的目录路径前缀，用于标识文件在主机设备文件系统中的位置。文件系统只是一个存储文件和嵌套文件夹的文件夹层级结构。例如，文件名 `a/b/file.txt` 包含指向 Unix（如 macOS、Linux 或 Android）上某个文件的前缀路径 `a/b`，而 `a\b\file.txt` 在

Windows 上做同样的事。如果文件名完全没有目录路径前缀，则该文件被假定存在于当前工作目录（CWD）中，因此是相对于它的。CWD 是运行脚本的那个文件夹（例如，当你启动一条 Python 命令时你在控制台中的位置）。在 REPL 中，CWD 就是你当时所在的工作位置；想查看 CWD 是什么，可以在大多数系统 shell 中运行 `pwd`，或者在 REPL 中 `import os` 后运行 Python 的 `os.getcwd()`。正如你在第 37 章扩展的文件覆盖中会看到的，文件名还可能包含非 ASCII 的 Unicode 字符，Python 会自动把它们与底层主机的编码互相转换。这些字符可以直接以字面量形式提供在文件名中，也可以放在一个预先编码好的字节字符串里——你会在本书后面学到字节字符串。`open` 的第二个参数——处理模式（mode）——通常是字符串 `'r'`（打开以进行文本输入，这是默认值）、`'w'`（创建并打开以进行文本输出）或 `'a'`（打开以把文本追加到末尾，例如用于写入日志文件）。处理模式参数还可以指定附加选项：- 在模式字符串中加入 `b`（例如 `'wb'`）允许处理二进制文件内容。用于文本的换行符转换和 Unicode 编码都会被关闭。- 在模式中加入 `+`（例如 `'r+'`）会以既能输入又能输出的方式打开文件。你可以对同一个文件对象进行读写，通常与 `seek` 操作配合来重新定位文件。`open` 的前两个参数都必须是 Python 字符串。可选的第三个参数接受一个整数来控制输出缓冲（buffering）——传 0 表示输出无缓冲（写方法调用时数据立即传输到外部文件）；还可以为特殊类型的文件提供附加参数（例如，文本文件的 Unicode 编码的字符串名）。你也可以用 `name=value` 关键字来传递 `open` 参数（例如 `file=name`、`mode=mode`），不过这在本章略超纲了。我们这里会讲解文件基础并探索一些基本示例，但不会深入所有文

件处理模式选项；在 REPL 里运行 `help(open)`，或者查阅 Python 库手册以获取更多细节。

代码分析

·解释器如何处理 `open`：`open('data','r')` 是内置对象构造器，内部走 `io.open()`，逐层包装成 `TextIOWrapper`（文本）或 `BufferedReader`（二进制读）等对象。它返回的并非“裸系统句柄”，而是带缓冲层 + 文本编码层的对象，这才是“文件对象”的真身。

·模式字符串的解析：模式 `'rb+'` 会被拆解为三部分语义：`r`（读写权限）/`b`（二进制，关闭换行翻译与解码）/`+`（可读写）。`'a'` 模式在打开时就把写指针定位到文件末尾；无该语义时写会覆盖内容。

·缓冲第三参数：`open('f','w', 0)` 使每次 `write` 都立刻下沉到系统调用（无缓冲，适合日志强实时），代价是每条写都付出一次系统调用开销，性能显著下降。

·设计思想：把“编码方案”“缓冲策略”“读写权限”分层用参数表达，且都留出默认值，让 90% 的场景只需写 `open('data')` 一行——简单用例简单，复杂用例才需要更多参数。

深度理解

·核心概念：`open(filename, mode)` 的返回对象是“文件句柄”的高级抽象：它不是内容本身，而是“连通外部文件的会话”。读/写/定位全通过这个句柄进行。

·底层实现：文件名解析在 Python 3.6+ 用 `os.PathLike`

协议（支持 Path 对象）；路径字符串在 Windows/Unicode 下被转换为 OS 可识别的宽字符路径。CWD 由解释器启动时继承进程的当前目录。

·设计原因：模式用单字符组合（如'rb'）紧凑表达多种语义，是 Unix C stdio (fopen) 传统的一脉相承——Python 沿用了业界熟悉的符号体系。

·实际问题：实践中最常犯的错误——用'w' 打开一个本要读的文件（会清空原文件！）；Windows 路径忘了转义（\n 被当成换行）；文本/二进制不匹配导致 UnicodeDecodeError。

·初学者误区：以为 open 第二个参数可省（省了就是'r' 读模式，想写文件的代码会直接报错）；以为路径开头带 \ 就绝对了（Windows 下绝对路径是带盘符如 C:\）。

9.10 Using Files（使用文件）

英文原文

Once you make a file object with open, you can call its methods to read from or write to the associated external file. In all cases, file content takes the form of strings in Python programs; reading a file returns its content in strings, and content is passed to the write methods as strings. Reading and writing methods come in multiple flavors; Table 9-2 lists the most common. Here are a few points of orientation up front: File iterators may be best for reading text lines — Though

the reading and writing methods in the table are common, keep in mind that probably the best way to read lines from a text file today is to not read the file at all—as you'll see in Chapter 14, files also have an iterator that automatically reads one line at a time in a for loop, list comprehension, or other iteration context. Content is strings, not objects — Notice in Table 9-2 that content read from a file always comes back to your script as a string, so you'll have to convert it to a different type of object if a string is not what you need. Similarly, file write operations do not add any sort of formatting and do not convert objects to strings automatically, so you must convert if needed and format as desired. Because of this, the tools we have already met to convert objects from and to strings (e.g., int, float, str, and string formatting) come in handy when dealing with files. Also note that newlines, added by print but not file writes, may have to be skipped if text from file reads is sent to print to avoid double spacing. Python also includes advanced standard-library tools for handling generic object storage (the pickle module), for dealing with packed binary data in files (the struct module), and for processing special types of content such as JSON and CSV text. We'll demonstrate these later in this chapter, but Python's manuals document them in full. Files are buffered and seekable — By default, output files are always buffered, which means that text you write may not be transferred from

memory to disk immediately—closing a file, or running its flush method, forces the buffered data to disk. You can avoid buffering with extra open arguments, but it may impede performance. Python files are also random-access on a byte-offset basis—their seek method allows your scripts to jump around to read and write at specific locations. close may be optional: auto-close on collection — Calling the file close method terminates your connection to the external file, releases its system resources, and flushes its buffered output to disk if any is still in memory. As discussed in Chapter 6, an object's memory space is automatically reclaimed as soon as the object is no longer referenced anywhere in the program. When file objects are reclaimed, Python also automatically closes them if they are still open (this also happens to open files when a program shuts down). This means you don't always need to manually close your files in Python, especially those in simple scripts with short runtimes, and temporary files used by a single line or expression. On the other hand, manual close calls don't hurt and are a good habit to form, especially in long-running systems and code run at the REPL. Strictly speaking, this auto-close of files is an implementation artifact of the standard CPython, and not part of the language definition—it may change over time, may not happen when you expect it to in interactive REPLs, and may not work the same in Python implementations whose garbage

collectors reclaim space differently than CPython. In fact, when many files are opened within loops, some Pythons may require close calls to free up system resources immediately, before garbage collection can get around to freeing objects. Close calls may sometimes also be required to flush buffered output of file objects not yet reclaimed. For an alternative and automatic way to ensure closes, watch for the file object's context manager ahead.

中文翻译

用 open 创建文件对象后，你就可以调用它的方法从关联的外部文件读取或写入。在所有情况下，文件内容在 Python 程序中都是以字符串形式存在的：读取文件时内容以字符串返回，写入时内容以字符串传给写方法。读写方法有多种风格；表 9-2 列出了最常见的。先给你几条定位性要点：文件迭代器可能是读文本行的最佳方式——尽管表中的读写方法很常用，但请记住：如今从文本文件读取行的最佳方式可能就是“根本不读文件”——正如你在第 14 章会看到的，文件也有一个迭代器（iterator），在 for 循环、列表推导式或其它迭代上下文中自动一次读取一行。内容是字符串，不是对象——注意表 9-2 中，从文件读取的内容总是以字符串形式返回到你的脚本里，所以如果字符串不是你需要的，你就必须转换它。同样，文件写操作不会添加任何格式化，也不会自动把对象转换成字符串，所以你需要时就必须转换、想要什么格式就自己格式化。正因如此，我们之前认识的、在对象与字符串之间转换的工具（如 int、float、str 和字符串格式化）在处理文件时就很派得上用场。还要注意

: `print` 会添加换行，但文件写不会；若把从文件读来的文本交给 `print` 输出，可能要跳过文件里的换行以避免出现双重间距。Python 还包含一些高级标准库工具：用于处理通用对象存储的 `pickle` 模块、用于处理文件中打包好的二进制数据的 `struct` 模块，以及用于处理 JSON 和 CSV 文本等特殊类型内容的工具。我们会在本章后面演示这些工具，但 Python 的手册对它们有完整文档。文件是缓冲的、可寻址的——默认情况下，输出文件总是缓冲（buffered）的，这意味着你写入的文本可能不会立即从内存传输到磁盘——关闭文件，或运行其 `flush` 方法，会强制把缓冲数据刷到磁盘。你可以用额外的 `open` 参数避免缓冲，但那可能损害性能。Python 文件也是按字节偏移量进行随机访问（random-access）的——其 `seek` 方法允许你的脚本跳来跳去，在特定位置读写。`close` 可能是可选的：回收时自动关闭——调用文件 `close` 方法会终止你与外部文件的连接、释放其系统资源，并把仍在内存中的缓冲输出刷到磁盘。如第 6 章所述，一旦对象在程序中不再被任何地方引用，它的内存空间就会自动被回收。当文件对象被回收时，如果它们仍然开着，Python 也会自动关闭它们（程序关闭时对打开的文件也会这么做）。这意味着在 Python 中你并非总需要手动关闭文件，尤其是那些运行时间很短的简单脚本，以及单行或单个表达式用到的临时文件。另一方面，手动 `close` 调用没有坏处，而且值得养成好习惯，尤其是在长时间运行的系统和在 REPL 中运行的代码里。严格来说，这种文件的自动关闭是标准 CPython 的实现产物（implementation artifact），并不是语言定义的一部分——它可能随时间变化、可能在交互式 REPL 中不会如你期待的那样发生、也可能在垃圾回收方式与 CPython 不同的 Python 实现上有不同表

现。事实上，当在循环中打开很多文件时，某些 Python 实现可能要求调用 `close` 来立即释放系统资源，等不到垃圾回收器的处理。有时也需要 `close` 调用来刷新那些尚未被回收的文件对象的缓冲输出。想要一种自动确保关闭的替代方案，敬请留意后面的文件对象上下文管理器（context manager）。

深度理解

·核心概念：文件内容永远以 `str/bytes` 字符串形式进出脚本——这是一种“接口澄清”：文件层不负责对象序列化，对象与字符串互转是你的责任。由此引出第 13 节的三种存储方案（手写转换、`pickle`、`JSON`）。

·底层实现：缓冲（buffering）意味着 `write` 只把数据填入用户区内存缓冲区，达到一定大小或调用 `flush/close`/缓冲区满时才触发操作系统 `write()` 系统调用；`seek` 直接改变文件对象内部的位置指针，下次读写从新位置开始。这解释了为什么“写了大段数据但文件里没看到”——还没刷盘。

·设计原因：自动关闭（auto-close on collection）在 CPython 里靠引用计数——文件引用数归零时对象立刻析构，顺手关闭文件。但引用计数是 CPython 的实现细节（不是语言规范），PyPy 等延迟回收的实现就不会“立刻”关闭，所以不能依赖它。

·实际问题：长循环里开了几十个文件再忘了关，M 个文件描述符耗尽会抛 `OSError: Too many open files`；这正是书中提醒“循环里打开许多文件除非及时 `close` 不然

可能出问题"的现实场景。

·初学者误区：以为 `f = open(...)` 之后不做 `f.close()` 就"没关系"——多数一次性脚本确实没事，但这是运气而非规范；养成 `with open`（见 9.17）的习惯可以从头规避。

9.11 Files in Action (文件实战)

英文原文

Let's work through an example that demonstrates file-processing basics. The following code begins by opening a new text file for output, writing two lines (strings terminated with a newline marker, `\n`), and closing the file. Later, the example opens the same file again in input mode and reads the lines back one at a time with `readline`:

```
>> myfile = open('myfile.txt','w') # 打开以进行文本输出：创建/清空
>> myfile.write('hello text file\n') # 写一行文本：字符串 16
>> myfile.write('goodbye text file\n') 18
>> myfile.close() # 确保输出被刷到磁盘
>> myfile = open('myfile.txt') # 打开以进行文本输入：'r' 是默认
>> myfile.readline() # 把行读回来'hello text file\n'
>> myfile.readline()'goodbye text file\n'
>> myfile.readline() # 空字符串：文件末尾"
```

Notice that the third `readline` call returns an empty string—this is how most file read methods tell you that you've reached the end of the file (and as you'll see

ahead, the empty string is inherently false in logical tests). Empty lines in the file instead come back as strings containing just a newline character ('\n'), not as empty strings.

Also notice that file write calls return the number of characters written; this is normally superfluous apart from error checks but is echoed in a REPL like this. This example writes each line of text, including its newline terminator, \n, as a string. Write methods don't add the newline character for us, so we must include it to properly terminate our lines; without this, the next write will simply extend the current line in the file.

If you want to display the file's content with newline characters interpreted, read the entire file into a string all at once with the file object's read method and print it (stringing together the open and read like this runs left to right and doesn't allow for an explicit close, but it doesn't matter for simple input at the REPL):

```
>> open('myfile.txt').read() # 一次性读进字符串'hello text file\ngoodbye text file\n'>> print(open('myfile.txt').read()) # 用户友好的显示 hello text file goodbye text file
```

And if you want to scan a text file line by line, file iterators are often your best option:

```
>> for line in open('myfile.txt'): # 用文件迭代器，而不是 read
```

```
... print(line, end='') # 不要再加 \n: 行里已经有一个了  
! ... hello text file goodbye text file
```

When coded this way, the temporary file object created by `open` will automatically read and return one line on each loop iteration. This form is usually easiest to code, light on memory use, and may be faster than some other options (depending on many variables, of course). Since we haven't reached statements or iterators yet, though, you'll have to wait until Chapter 14 for a more complete explanation of this code.

As noted, files content is always strings, so other kinds of objects must be converted for write. The `str` call and concatenating separators and newlines suffice and emulate what `print` does automatically (but if you like the sugarcoating provided by `print`, stay tuned for its coverage in the next part of this book, where you'll learn how to route its display to a file you make first with `open`):

```
>> myfile = open('myfile2.txt','w') >> myfile.write(3.14)  
# 非字符串对象会失败 TypeError: write() argument  
must be str, not float >> myfile.write(str(3.14) + '\n')  
# 转换 (并模拟 print) 5 >> myfile.close() >> open('myfile2.txt').read()  
# 之后可以用 float() 再转回来 '3.14\n'
```

Incidentally, the files we've made here show up in the CWD, because we didn't provide a path prefix in their filenames. In Python, you can check what the CWD is and get a listing of the files there, with the `os` sta

ndard-library module:

```
>> import os >> os.getcwd() # 显示当前工作目录'/Users/me/code/Chapter09'>> os.listdir() # 列出这里的文件 (或传入一个路径) ['myfile2.txt','myfile.txt']
```

This was run on macOS and mirrors shell commands, but such tools are useful in many programs. The take away is that filename myfile.txt in this CWD is equivalent to /Users/me/code/Chapter09/myfile.txt in open and other tools.

NOTE: 编码 Windows 路径: 如果你选择在 Windows 上提供完整的目录路径, 它们需要特殊处理, 因为 Windows 的 \ 路径分隔符在 Python 中也被用于字符串转义 (见第 7 章)。如那里所述, open 在 Windows 上接受 Unix 风格的正斜杠来替代反斜杠, 所以以下任何形式都可以用于 Windows 上的目录路径:

```
open(r'C:\Users\me\code\newdata.txt') # 原始字符串  
open('C:/Users/me/code/newdata.txt') # 正斜杠  
open('C:\\Users\\me\\code\\newdata.txt') # 双写转义
```

第一个命令中的原始字符串形式很有利于关掉非预期的转义 (例如 \n), 虽然另外两个选项让转义问题完全消失。在 Unix 上, 你当然直接用正斜杠即可 (并去掉 Windows 盘符: 你的驱动器是挂载的, 而不是分立的)。

中文翻译

让我们动手做一个演示文件处理基础的示例。下面的代码先是打开一个新文本文件用于输出, 写入两行 (以换行标记 \n 结尾的字符串), 然后关闭文件。稍后, 示例再次以输入

模式打开同一个文件，用 `readline` 把行一次一条地读回来：

```
python myfile = open('myfile.txt','w') # 打开文本输出：创建/清空文件myfile.write('hello text file\n') # 写入一行文本，返回写入字符数 16
myfile.write('goodbye text file\n') # 返回 18
myfile.close() # 确保输出刷到磁盘
myfile = open('myfile.txt') # 打开文本输入：'r' 是默认
myfile.readline() # -> 'hello text file\n'
myfile.readline() # -> 'goodbye text file\n'
myfile.readline() # -> ''
```

空字符串表示文件末尾 EOF 注意第三个 `readline` 调用返回一个空字符串——这是大多数文件读取方法告诉你已经到达文件末尾（end of file, EOF）的方式（而且正如你后面会看到的，空字符串在逻辑测试中本质上是假）。文件中的空行反而会作为只包含一个换行字符（`\n`）的字符串返回，而不是空字符串。还要注意文件 `write` 调用会返回写入的字符数；除了错误检查外这通常多余，但在 REPL 中会像这样回显出来。本示例把每一行文本（包括换行终止符 `\n`）作为一个字符串写入。写方法不会替我们添加换行符，所以我们必须自己包含它来正确终止每一行；否则，下一次写入只会延长当前行。如果你想显示文件内容并让换行符被解释，就用文件对象的 `read` 方法把整个文件一次性读进字符串再打印（像这样把 `open` 和 `read` 串起来是从左到右执行的，并且没有显式 `close` 的机会，但对 REPL 中的简单输入来说没关系）：

```
python open('myfile.txt').read() # -> 'hello text file\ngoodbye text file\n'
print(open('myfile.txt').read()) # 用户友好的显示：\n 被渲染成换行
```

如果你想逐行扫描文本文件，文件迭代器通常是最佳选择：

```
python for line in open('myfile.txt'): # 使用文件迭代器，而不是 read
    print(line, end='') # 别再额外加 \n：行里已经有换行了！ 这样
```

写时，open 创建的临时文件对象会在每次循环迭代中自动读取并返回一行。这种形式通常最容易写、内存占用轻，而且可能比某些其它选项更快（当然取决于很多变量）。不过我们还没讲到语句和迭代器，所以你必须等到第 14 章才能得到这段代码更完整的解释。如前所述，文件内容永远是字符串，所以其它种类的对象在 write 前必须先转换。str 调用加上拼接分隔符和换行符就够了，可以模拟 print 自动做的事（但如果你喜欢 print 提供的糖衣，请继续关注本书下一部分对它讲解——你会学到如何把它的显示引导到你先用 open 建好的文件里）：

```
python myfile = open('myfile2.txt','w')
myfile.write(3.14) # TypeError: write() 参数必须是 str 而非 float
myfile.write(str(3.14) + '\n') # 转成字符串再写（顺便模拟 print）
myfile.close()
open('myfile2.txt').read() # -> '3.14\n', 以后可用 float() 再还原顺便说一句，我们在 CWD 中看到的这些文件，是因为文件名里没有带路径前缀。在 Python 中，你可以用 os 标准库模块查看 CWD 是什么、列出那里的文件：
python import os
os.getcwd() # 显示当前工作目录，例如 '/Users/me/code/Chapter09'
os.listdir() # 列出该目录下的文件 -> ['myfile2.txt','myfile.txt']
这是在 macOS 上运行的，镜像了 shell 命令，但这类工具在许多程序中都有用。要点是：这个 CWD 中的文件名 myfile.txt 在 open 等工具中就等价于 /Users/me/code/Chapter09/myfile.txt。
```

代码分析

·解释器如何处理逐行读取：readline() 每一行读一个字符串，内部维护读写位置指针；读到文件末尾时返回特殊值'——空字符串是 EOF 信号，而空行则返回'\n'，两者绝

不能混淆（前者是"没内容了"，后者是"有一个空行"）。

- write 的返回值：write 返回实际写入的字符数（文本模式）或字节数（二进制模式），是 C fwrite 的遗产，主要供错误检查使用。

- 文件迭代器：for line in open(...) 中，文件对象实现 iter/next，CPython 每次 next 时调用一次底层 readline——一边读边用、不整文件进内存，大文件（如几个 GB 的日志）也能流畅处理。这是迭代器"惰性（lazy）"价值的首个鲜活例子。

- 为什么 3.14 不能直接写：write 的类型检查要求 str（文本模式），float 没有通过，抛 TypeError；str(3.14) 把它转成 '3.14' 再写。这说明文件不负责对象化，一切转换责任在调用方。

- 设计思想：open('myfile.txt').read() 这种连写是为了 REPL 便捷；正式代码里文件对象失去引用后，CPython 引用计数会立刻析构它并关闭句柄——但正如书中所言，这是"实现产物"，规范层面不保证。

深度理解

- 核心概念：文件的读写围绕"位置指针 + 缓冲区"展开；readline 返回"表示 EOF；write 不改写换行；print 会加换行而 write 不会——理解这些约定，才能正确拼装文件内容。

- 底层实现：文本模式下 write 会把 str 按编码器转成字节存入缓冲，readline 从缓冲/系统读字节再解码；文件对象析构（del）时自动 flush+close，这是 CPython 引用

计数析构的副效应。end="" 关键字让 print 不再追加默认分隔符，从而避免行内多出一个换行。

·设计原因："read 返回字符串、write 收字符串"保证了文件复用的简单一致；把文件做成可迭代对象则让"逐行处理"成为一等公民，避免旧语言里必须手写读行循环的样板代码。

·实际问题：日志处理、字典合并、数据清洗都靠逐行迭代；"写进去却看不到"多半是缓冲未刷或路径不对；用 os.getcwd() 排查"找不到文件"这类魔幻报错。

·初学者误区：分不清 readline()/read()/readlines() 的差异从而整文件读崩内存；忘记给 print(line, end='') 处理多出的换行导致输出空行翻倍；在 Windows 上写'C:\newfile.txt' 被 \n 转义坑到。

9.12 Text and Binary Files: The Short Story (文本与二进制文件：速览)

英文原文

Strictly speaking, the examples in the prior section use text files. More generally, file type is determined by the second argument to open, the mode string—including a "b" in it means binary, which is sharply distinguished from text: Text files represent content as a normal string, perform Unicode encoding and decoding automatically, and perform newline translation by default. This mode is useful for processing text of

all kinds. Binary files represent content as a special bytes string and allow programs to access file content unaligned. This mode is useful for processing nontext content like media. Programs that deal only with simple text like ASCII can get by with the basic text-file interface used in the prior examples, and normal strings. All text strings are technically Unicode in Python, but ASCII users will not generally notice because it's a subset of Unicode (every ASCII file is a Unicode file, even if its character range is limited). If you need to handle non-ASCII text or byte-oriented data, though, you'll need to match object types to file modes—bytes strings for binary files, and normal str strings for text files. Because text files implement Unicode encodings, you also should not open a binary data file in text mode: decoding its content to Unicode text will likely fail. Let's turn to a brief example. When you write and read a binary file, you send and receive a bytes object—a sequence of small integers that represent absolute byte values (which may or may not correspond to characters), and which is coded with a leading b but looks and feels almost exactly like a normal text string:

```
>>> myfile = open('myfile3.bin','wb') # Make binary
file: wb=write binary >>> myfile.write(b'\x00\x01hack
k\x02\x03') # Bytes string holds binary data 8
```

```
>>> myfile.close() >>> data = open('myfile3.bin','rb'
).read() # Read binary file: rb=read binary >>> data
```

```
# Raw, unaltered bytes returned b'\x00\x01hack\x02\x03'
```

```
>>> data[2:6] # Bytes act like text strings b'hack'
```

```
>>> byte = data[2:6][0] # But really small 8-bit integers  
>>> byte, chr(byte), bin(byte) (104, 'h', '0b1101000')
```

In addition, binary files do not perform any newline translation on content, but text files by default map all forms to and from `\n` when read and written. Text files also implement Unicode encodings on transfers, using an optional encoding name passed to the encoding argument of `open`. If encoding is not passed (as in earlier examples), files fall back on the underlying platform's default, which may not be interoperable with other hosts or files. Per the start of this chapter, though, that's as much as we're going to say about Unicode text and binary data files here, and just enough to understand upcoming examples in this chapter. If you're anxious to dive into this topic further, see either the preview in Chapter 4 or wait for the full story in Chapter 37. For this chapter, let's move on to a handful of more substantial file examples that demonstrate common ways to store Python object values in files.

中文翻译

严格来说，上一节的示例使用文本文件。更一般地说，文件

类型由 `open` 的第二个参数——模式字符串——决定：其中包含 "b" 就意味着二进制 (binary)，它与文本天差地别：文本文件 (text files) 以普通的 `str` 字符串表示内容，自动执行 Unicode 编码与解码，并默认执行换行符转换 (newline translation)。这种模式对处理各种文本很有用。二进制文件 (binary files) 以一种特殊的 `bytes` 字符串表示内容，并允许程序以未经改动的方式访问文件内容。这种模式对处理媒体等非文本内容很有用。只处理像 ASCII 这样简单文本的程序，用前面例题的基本文本文件接口和普通字符串就够了。严格来说，Python 中所有文本字符串都是 Unicode，但 ASCII 用户通常注意不到，因为 ASCII 是 Unicode 的子集（每个 ASCII 文件都是 Unicode 文件，即使它的字符范围有限）。不过，如果你需要处理非 ASCII 文本或面向字节 (byte-oriented) 的数据，就需要让对象类型与文件模式匹配——二进制文件用 `bytes` 字符串，文本文件用普通 `str` 字符串。由于文本文件实现的是 Unicode 编码，你也不应该用文本模式打开二进制数据文件：把它的内容解码成 Unicode 文本很可能会失败。让我们转向一个简短的例子。当你写入并读取一个二进制文件时，发送和接收的是一个 `bytes` 对象——一个表示绝对字节值的小整数序列（可能对应字符也可能不对应），它以开头的 b 标识，但看起来、用起来和普通文本字符串几乎一模一样：

```
python myfile = open('myfile3.bin','wb') # 生成二进制文件：wb = 写二进制
myfile.write(b'\x00\x01hack\x02\x03') # bytes 字符串装二进制数据，返回 8
myfile.close()
data = open('myfile3.bin','rb').read() # 读二进制文件：rb = 读二进制
data # 返回原始未改动的字节 -> b'\x00\x01hack\x02\x03'
data[2:6] # bytes 表现像文本字符串 ->
```

`b'hack'byte = data[2:6][0]` # 但取出的其实是 8 位小整数
-> 104 byte, `chr(byte)`, `bin(byte)` # -> (104,'h','0b1101000') 此外, 二进制文件对内容不执行任何换行转换, 而文本文件默认在读和写时把所有形式的行结束符都与 `\n` 互相映射。文本文件在传输时还会实现 Unicode 编码, 使用传给 `open` 的 `encoding` 参数的可选编码名。如果没有传 `encoding` (如前面的例子), 文件就回退到底层平台的默认编码, 这可能与其它主机或文件不互通。不过按照本章开头所说, 关于 Unicode 文本和二进制数据文件, 我们在这里就说这么多——刚好够理解本章后面的示例。如果你急于深入这个话题, 要么看第 4 章的预览, 要么等第 37 章的完整故事。本章接下来, 我们将转向几个更有分量的文件示例, 它们演示了把 Python 对象值存进文件的常见方式。

代码分析

·解释器如何处理 bytes: `b'\x00\x01hack\x02\x03'` 是一个 bytes 字面量; `write` 在二进制模式下直接收 bytes, 不做任何编码转换; `read` 返回的也是 bytes。`data[2:6]` 得到子串 `b'hack'`; 但 `data[2:6][0]` 取出的不是字符而是整数 104——因为 bytes 是“不可变的整数序列”, 其索引语义与 `str` 完全不同。`chr(104)` 把 104 还原为字符 `'h'`, `bin(104)` 给出二进制形式 `'0b1101000'`。这是理解“字节 vs 字符”的分水岭例子。

·为什么二进制不过解码: `'rb'` 模式下 `io` 层只暴露 `BufferedReader`, 不挂编码器, 所以数据一个字节不多一个字节不少地进出; 文本模式则会颠倒转义 `\n`、做 Unicode 解码——用错模式轻则乱码, 重则 `UnicodeDecodeError`

·设计思想：文本/二进制的分野，与"字符语义 vs 字节语义"一一对应。文本模式帮你在编码与换行上免操心；二进制模式把绝对控制权交还给你（比如拷贝文件、读媒体头）。

深度理解

·核心概念：文本文件操作的是 str（字符），二进制文件操作的是 bytes（字节）。二者不能混用：write(3.14) 报错、用文本模式读 \xff 文件也可能报错，都是"字符/字节错配"的不同症状。

·底层实现：文本模式下每次读写都经过编解码器（默认 UTF-8 或平台默认 locale）；换行翻译把 \r\n（Windows）\r（老 Mac）统一映射成 \n，写回时再按平台转回。二进制模式则绕过所有这些翻译。

·设计原因：Platform 兼容性是这样换来的——同一份代码在不同 OS 上读写文本文件得到一致的 \n 语义；同时给需要"原样字节"的场景留出二进制口子。默认编码不可移植是隐私坑，所以书里提醒跨主机交互时显式传 encoding。

·实际问题：图片/音视频/压缩包/协议数据全是二进制；纯文本、配置、日志是文本。判断标准很简单：里面装的是人能读的字，还是任意字节流。

·初学者误区：在文本模式写 bytes（或反过来）报 TypeError 后不知为何；以为 data[0] 得到的是字符（bytes 里是整数）；忽略 encoding= 导致换一台机器乱码；用'rb'

读了文本文件再 decode 时手足无措。

9.13 Storing Objects with Conversions (用转换来存储对象)

英文原文

Our next example writes a variety of Python objects to a text file on multiple lines. We wrote a single number to a file earlier, but are kicking it up a notch here to demo more conversions. Again, file content is strings in our code, and write methods do not do any toString formatting (for space, this chapter omits write return values from here on):

```
>> X, Y, Z = 62, 63, 64 # 原生 Python 对象 >> S = 'Text' # 值还要存进文件, 必须是字符串 >> D = {'a': 1, 'b': 2} >> L = [1, 2, 3] >> F = open('datafile.txt', 'w') # 创建输出文本文件 >> F.write(S + '\n') # 用 \n 终止行 >> F.write(f'{X},{Y},{Z}\n') # 把数字转成字符串 >> F.write(str(L) + '$' + str(D) + '\n') # 转换并用 $ 分隔 >> F.close()
```

Once we have created our file, we can inspect its contents by opening it and reading it into a string (strung together as a single operation here). Notice that the interactive echo gives the exact character contents, while the print operation interprets embedded newline characters to render a more user-friendly display:

```
>> chars = open('datafile.txt').read() # 原始字符串显示
>> chars"Text\n62,63,64\n[1, 2, 3]${'a': 1,'b': 2}\n"
>> print(chars) # 用户友好的显示 Text 62,63,64 [1, 2, 3]
${'a': 1,'b': 2}
```

We now have to use other conversion tools to translate from the strings in the text file to real Python objects. As Python never converts strings to numbers (or other types of objects) automatically, this is required if we need to gain access to normal object tools like indexing, addition, and so on:

```
>> F = open('datafile.txt') # 再次打开
>> line = F.readline() # 读一行
>> line"Text\n"
>> line.rstrip() # 去掉换行
'Text'
```

For this first line, we used the string `rstrip` method to get rid of the trailing newline character; a `line[:-1]` slice would work, too, but only if we can be sure all lines end in the `\n` character (the last line in a file sometimes does not).

So far, we've read the line containing the string. Now let's grab the next line, which contains numbers, and parse out (that is, extract) the objects on that line:

```
>> line = F.readline() # 文件中的下一行
>> line # 在这里它是字符串'62,63,64\n'
>> parts = line.rstrip().split(',') # 按逗号切分（解析）
>> parts ['62','63','64']
```

We used the string `split` method here to chop up the line on its comma delimiters (after removing the trailing

ng \n with `rstrip` as before—its result is a new string on which we run `split`). The result is a list of substrings containing the individual numbers. We still must convert from strings to integers, though, if we wish to perform math on these:

```
>> int(parts[1]) # 从字符串转成 int 63 >> numbers =  
[int(P) for P in parts] # 一次性把列表里所有都转换 >> n  
umbers [62, 63, 64]
```

As we have learned, `int` translates a string of digits in to an integer object, and the list comprehension expression introduced in Chapters 4 and 8 can apply the call to each item in our list all at once (again, you'll find more on list comprehensions later in this book). Note: we didn't have to use `rstrip` to delete the `\n` at the end of the line, because `int` and some other converters quietly ignore whitespace around digits; still, being explicit is often best.

Finally, to convert the stored list and dictionary in the third line of the file, we can run them through `eval`, a built-in function we first met in Chapter 5, that treats a string as a piece of executable program code (technically, a string containing a Python expression, with trade-offs discussed in the next section):

```
>> line = F.readline() # 文件中的下一行 >> line "[1, 2,  
3]${'a': 1,'b': 2}\n">> parts = line.split('$') # 按 $ 切分  
(解析) >> parts ['[1, 2, 3]', "${'a': 1,'b': 2}\n"] >> eval  
(parts[0]) # 转成任意对象类型 [1, 2, 3] >> objects = [ev
```

```
al(P) for P in parts] # 对列表中所有元素都这样做 >> objects [[1, 2, 3], {'a': 1, 'b': 2}]
```

Because the end result of all this parsing and converting is a list of normal Python objects instead of strings, we can now apply list and dictionary operations to them in our script.

中文翻译

我们的下一个示例把多种 Python 对象写到文本文件的多行里。我们前面只写过一个数字到文件，这里要提高难度，演示更多转换。再说一遍，在我们的代码里，文件内容都是字符串，写方法不做任何“转成字符串”的格式化（为节省篇幅，本章从这往后略去 write 的返回值）：

```
python X, Y, Z = 62, 63, 64 # 原生 Python 对象
S = 'Text' # 想存进文件的必须是字符串
D = {'a': 1, 'b': 2}
L = [1, 2, 3]
F = open('datafile.txt', 'w') # 创建输出文本文件
F.write(S + '\n') # 用 \n 终止行
F.write(f'{X},{Y},{Z}\n') # f-string 把数字值转成字符串
F.write(str(L) + '$' + str(D) + '\n') # str() 转换，并用 $ 分隔内容
F.close() # 创建好文件之后，我们可以打开它、把它读进一个字符串来检查内容（这里连写成一次操作）
```

。注意：交互式回显给出的是精确的字符内容，而 print 操作会解释内嵌的换行符，渲染出更友好的显示：

```
python chars = open('datafile.txt').read() # -> "Text\n62,63,64\n[1, 2, 3]${'a': 1, 'b': 2}\n"
print(chars) # 显示为三行可读文本
```

现在，我们必须用其它转换工具，把文本文件中的字符串翻译回真正的 Python 对象。由于 Python 从不自动把字符串转换成数字（或其它类型的对象），如果我们需要访问索引、加法等常规对象工具，这一步是必需的：

```
python F
```

= open('datafile.txt') # 再次打开 line = F.readline() # -> 'Text\n' line.rstrip() # 去掉尾部换行 -> 'Text' 对第一行，我们用了字符串的 rstrip 方法来去掉尾随换行符；用 line[: -1] 切片也行，但前提是能确定所有行都以 \n 结尾（文件中的最后一行有时不是）。到目前为止，我们读了含字符串的那行。现在抓取下一行——包含数字——并把该行上的对象解析（parse out，即提取）出来：python line = F.readline() # -> '62,63,64\n' parts = line.rstrip().split(',') # 去掉 \n 后按逗号切分 -> ['62','63','64'] 我们在这里用字符串 split 方法按逗号分隔符切开这一行（先用 rstrip 去掉尾随 \n，如前——它的结果是一个新字符串，我们在这个新字符串上运行 split）。结果是包含各个数字的子字符串列表。不过，如果我们要对这些数字做数学运算，仍必须把它们从字符串转成整数：python int(parts[1]) # -> 63 numbers = [int(P) for P in parts] # 列表推导式一次性全转 -> [62, 63, 64] 如我们所学，int 会把数字字符串翻译成整数对象，而第 4 章和第 8 章引入的列表推导式可以一次性把这调用应用到列表中的每一个条目（再说一次，本书后面还有更多关于列表推导式的内容）。吹毛求疵一下：我们其实不必用 rstrip 删除行尾的 \n，因为 int 和某些其它转换器会安静地忽略数字周围的空白；但话说回来，写得显式往往最好。最后，为了转换文件中第三行存储的列表和字典，我们可以让它们经过 eval——我们在第 5 章首次遇到的内置函数——它把字符串当作一段可执行的程序代码（严格说来，是包含一个 Python 表达式的字符串；其权衡会在下一节讨论）：python line = F.readline() # -> "[1, 2, 3]\${'a': 1,'b': 2}\n" parts = line.split('\$') # 按 \$ 切分 -> ['[1, 2, 3]', "\${'a': 1,'b': 2}\n"] eval(parts[0]) # -> [1, 2, 3] obj

ects = [eval(P) for P in parts] # -> [[1, 2, 3], {'a': 1, 'b': 2}] 因为所有这些解析与转换的最终结果是一个正常的 Python 对象列表（而不是字符串列表），我们现在就可以在脚本中对它们应用列表和字典操作了。

代码分析

·解释器如何处理 f-string 与 str(): f'{X},{Y},{Z}\n' 本质是 str.format 的语法糖，编译期就解析出这三个表达式，运行时每个值调用一次 format（默认即 str()）再拼装。str(L) 把列表对象转成它的 repr 风格的字符串'[1, 2, 3]'; str(D) 得到 '{"a": 1, "b": 2}'。

·为什么非显式转换不可：文件层面字节流只有 str；Python 不会像 PHP/JS 那样自动“字符串化”，F.write(D) 必报 TypeError。所以存之前必须转，读之后必须还原。

·解释器如何处理 eval: eval('[...])' 会真正地编译并执行那段字符串——它启动 Python 编译器（compile 模式为'eval'），生成字节码并执行。因此 eval 是“字符串反序列化为对象”的万金油，但也是安全黑洞：任何读入的内容都可能被执行。书中下一节（pickle）正是为了解决这个副作用。

·解析思路：rstrip().split(',') 是“清洗→切分”两段的惯用法；line[:-1] 可行但不如 rstrip() 稳健（最后一行可能无 \n）。int(parts[1]) 会忽略两侧空白，但显式 rstrip 让意图清楚。

·设计思想：这就是最朴素的“文本序列化”——约定好分隔符与格式，写入与读出对称。它理解成本低、可读性强，

适合简单数据；复杂数据就要交给 pickle/JSON。

深度理解

·核心概念：序列化 (serialization) = 对象 → 字符串 → 文件；反序列化 = 文件 → 字符串 → 对象。本节展示的是"手动配对"的最小方案：写入时转、读出时转。

·底层实现：str(D) 用的是对象的 repr/str，输出恰好是 Python 字面量语法的外观——这并非巧合：Python 的 repr 设计目标之一就是"尽量能让 eval 还原"。

·设计原因：文件不存对象只存字符串，是为了跨实现、跨进程、跨版本的稳定；对象内存布局（地址、引用关系）没人应该暴露到磁盘。

·实际问题："42" 与 42、{"a":1} 与字典差异巨大——凡是把数据读出后要用 obj['a']/obj + 1 的地方，都必须有一次显式还原。日志、CSV、简易持久化都走这条路。

·初学者误区：忘记读回后对象还是字符串就开始 .append()/+1 而报错；用 eval 反序列化不可信来源数据（命令注入！）；split 后忘了还有'\n' 混在最后一个元素里。

9.14 Storing Objects with pickle (用 pickle 存储对象)

英文原文

Using eval to convert from strings to objects, as demonstrated in the preceding code, is a powerful tool. I

n fact, sometimes it's too powerful. eval will happily run any Python expression—even one that might delete all the files on your computer, given the necessary permissions. If you really want to store native Python objects, but you don't want to run file content as program code, Python's standard-library pickle module can help. The pickle module is a more advanced tool that allows us to store almost any Python object in a file directly, with no to- or from-string conversion requirement on our part. It's like a super-general data formatting and parsing utility. To store a dictionary in a file, for instance, we pickle it directly after using binary mode to open the file: >>> D = {'a': 1, 'b': 2} >>> F = open('datafile.pkl', 'wb') >>> import pickle >>> pickle.dump(D, F) # 把几乎任何对象 pickle 进文件 >>> F.close() Then, to get the dictionary back later, we simply use pickle again to re-create it, again using a binary-mode file: >>> F = open('datafile.pkl', 'rb') >>> E = pickle.load(F) # 从文件加载几乎任何对象 >>> E {'a': 1, 'b': 2} We get back an equivalent dictionary object, with no manual splitting or converting required. The pickle module performs what is known as object serialization—converting objects to and from strings of bytes—but requires very little work on our part. In fact, pickle internally translates our dictionary to a bytes-string form; it's not much to look at, and may vary for protocol options and future morph, but mandates binary-mode files: >>> open('datafile.pkl', 'rb').read() # 这种

格式将来可能变化! b'\x04\x95\x11\x00\x00\x00\x00\ ...etc... \x8c\x01a\x94K\x01\x8c\x01b\x94K\x02u.'

Because pickle can reconstruct the object from this format, we don't have to deal with it ourselves. For more on the pickle module, see the Python standard-library manual, or import pickle and pass it to help interactively. While you're exploring, also take a look at the shelf module. shelf is a tool that uses pickle to store Python objects in an access-by-key filesystem, which is beyond our scope here (though you will get to see an example of shelf in action in Chapter 28, and other pickle examples in Chapters 31 and 37).

中文翻译

如前面的代码所示，用 `eval` 从字符串转换到对象是个强大的工具。事实上，有时它过于强大了。`eval` 会乐意运行任何 Python 表达式——甚至是在拥有必要权限的情况下、可能删掉你电脑上所有文件的表达式。如果你真的想存储原生 Python 对象，又不想把文件内容当作程序代码来运行，Python 标准库的 `pickle` 模块可以帮忙。`pickle` 模块是一个更先进的工具，它允许我们把几乎任何 Python 对象直接存进文件，无需我们自己做任何“转成/转出字符串”的转换。它就像一个超级通用的数据格式化与解析工具。例如，要把一个字典存进文件，我们在用二进制模式打开文件后直接把它 `pickle` 进去：

```
python D = {'a': 1, 'b': 2} F = open('datafile.pkl', 'wb') # 必须用二进制模式
import pickle
pickle.dump(D, F) # 把几乎任何对象序列化进文件
F.close()
```

之后要想把字典拿回来，我们只需再用 `pickle` 重建它，同样

·为什么必须是二进制模式：pickle 输出的是 bytes，'wb'/'rb' 保证字节原样进出；若用文本模式，Windows 下的换行翻译会把 \x0a 串扰成 \r\n，导致 load 失败。这是"对象类型必须匹配文件模式"原则的最典型实例。

·与 eval 的本质区别：eval 把字符串当代码执行（危险、且只能处理字面量表达式）；pickle 把字节流当协议数据解析——它只操作"类型名 + 数据"的指令，不会执行任意表达式（但仍有类构造漏洞，反序列化不可信数据也要谨慎）。

·设计思想：pickle 负责"最难的一步"——把自定义对象与嵌套结构完整还原（包括引用关系、递归共享），这是手写 str()/eval 难以企及的。代价是格式私有、不可跨语言、版本敏感。

深度理解

·核心概念：序列化有三档方案：手写转换（9.13，简单可控但费力）、pickle（原生对象直通车，Python 专属）、JSON（9.15，跨语言标准）。pickle 的对象覆盖度最高。

·底层实现：pickle 有若干个协议版本（protocol 0-5+）；\x80\x04 开头表示 protocol 4。它对每个对象递归使用推栈（memo）机制，相同对象只存一次、用引用还原，因此能处理自引用/循环结构。

·设计原因：把"对象怎样落地"从"程序怎么读文件"中解耦——pickle 做的是自动化；程序员只管 dump/load 两个调用。格式设计为私有且可演进（protocol 变化正是书中

提示"格式易变"的原因)。

·实际问题：机器学习模型权重、缓存对象、进程间传对象、session 持久化，pickle 都是便捷通道；但当数据要交给 Java/前端/其它语言时，必须改用 JSON 等开放格式。

·初学者误区：pickle 文件不可跨版本迁移（易变）；反序列化不可信数据有任意代码执行风险（与 eval 类似但更强）；以为 D 和 E 是同一个对象（只是值相等，E is D 为 False）；忘了用二进制模式文件导致诡异错误。

9.15 Storing Objects with JSON (用 JSON 存储对象)

英文原文

The prior section's pickle module translates nearly arbitrary Python objects to a proprietary format developed specifically for Python, and honed for performance over many years. JSON is a newer data interchange format, which is both programming-language-neutral and supported by a variety of systems. MongoDB, for instance, stores data in a JSON document database (using a binary JSON format), and JSON is common in configuration roles.

JSON does not support as broad a range of Python object types as pickle, but its portability is an advantage in some contexts, and it represents another way to serialize a specific category of Python objects for st

orage and transmission.

Moreover, because JSON is so close to Python dictionaries and lists in syntax, the translation to and from Python objects is trivial, and is automated by the `json` standard-library module.

For example, a Python dictionary with nested structures is very similar to JSON data, though Python's variables and expressions support richer structuring options (any part of the following can be an arbitrary expression in Python):

```
>> who = dict(first='Pat', last='Smith') >> rec = dict
(name=who, job=['dev','mgr'], age=40.5) >> rec {'name': {'first': 'Pat', 'last': 'Smith'}, 'job': ['dev', 'mgr'], 'age': 40.5}
```

The final dictionary format displayed here is a valid literal in Python code, and almost passes for JSON when printed as is, but the `json` module makes the translation official—here translating Python objects to and from a JSON serialized string representation in memory:

```
>> import json >> json.dumps(rec) # Python 对象 -> JSON 字符串 '{"name": {"first": "Pat", "last": "Smith"}, "job": ["dev", "mgr"], "age": 40.5}' >> S = json.dumps(rec) >> O = json.loads(S) # JSON 字符串 -> Python 对象 >> O {'name': {'first': 'Pat', 'last': 'Smith'}, 'job': ['dev', 'mgr'], 'age': 40.5} >> O == rec True
```

It's similarly straightforward to translate Python objects to and from JSON data strings in files. Prior to being stored in a file, your data is simply Python objects; the JSON module re-creates them from the JSON textual representation when it loads it from the file. In both cases, we use text files, because JSON is text:

```
>> file = open('testjson.txt','w') >> json.dump(rec, f
p=file, indent=4) # Python 对象 -> JSON 写入文件 >>
file.close() >> print(open('testjson.txt').read()) # 人类
可读的文件内容 {"name": {"first": "Pat", "last": "Smith"},
"job": ["dev", "mgr"], "age": 40.5} >> P = json.load(ope
n('testjson.txt')) # 从 JSON 读回 Python 对象 >> P {'na
me': {'first': 'Pat', 'last': 'Smith'}, 'job': ['dev', 'mgr'], 'age':
40.5} >> P == rec True
```

Once you've translated from JSON text, you process the data using normal Python object operations in your script. For more details on JSON-related topics, see Python's library manuals and the web at large. You could, of course, store real Python dictionaries and lists in an imported Python .py module file, and the data would be just as readable and editable; JSON, however, is not run as code, and may be perceived by some as more interoperable today.

Note that strings are all Unicode in JSON to support richer forms of text. Since Python strings in memory simply are Unicode, the distinction matters most when transferring text to and from files. You'll learn how to

apply Unicode encodings to JSON (and other) files and data in Chapter 37.

中文翻译

上一节的 pickle 模块把几乎任意的 Python 对象翻译成一种专用格式——它是为 Python 专门开发、并经多年打磨以追求性能的。JSON 是一种较新的数据交换格式，它既与编程语言无关 (language-neutral)，又得到各种系统的支持。例如，MongoDB 把数据存储在 JSON 文档数据库里（使用二进制 JSON 格式），而且 JSON 在配置场景中也很常见。JSON 不支持 pickle 那么广泛的 Python 对象类型范围，但它的可移植性 (portability) 在某些场合是优势，它代表了把特定一类 Python 对象序列化以用于存储和传输的另一种方式。此外，因为 JSON 在语法上与 Python 字典和列表如此接近，与 Python 对象之间的转换几乎是微不足道的，并且由 json 标准库模块自动完成。例如，一个带嵌套结构的 Python 字典与 JSON 数据非常相似，尽管 Python 的变量和表达式支持更丰富的结构化选项（下面的任何部分在 Python 中都可以是任意表达式）：

```
python who = dict(first='Pat', last='Smith') rec = dict(name=who, job=['dev','mgr'], age=40.5) rec # -> {'name': {'first': 'Pat', 'last': 'Smith'}, 'job': ['dev', 'mgr'], 'age': 40.5}
```

这里显示的最终字典格式在 Python 代码中是合法的字面量，原样打印出来几乎就是 JSON，但 json 模块把这种转换变成正式行为——这里是在内存里把 Python 对象与 JSON 序列化后的字符串表示互相转换：

```
python import json json.dumps(rec) # -> '{"name": {"first": "Pat", "last": "Smith"}, "job": ["dev", "mgr"], "age": 40.5}' S = json.dumps(
```


rec) # Python 对象 => JSON 字符串 O = json.loads(S)
JSON 字符串 => Python 对象 O == rec # True: 值相等
把 Python 对象与文件中的 JSON 数据字符串互相转换也
同样直截了当。在存进文件之前，你的数据就是普通的 Python 对象；JSON 模块从文件中加载时，会根据 JSON 文本表示把它们重建出来。在这两种情况下，我们都用文本文件，因为 JSON 是文本 (text)：

```
python file = open('testjson.txt','w')  
json.dump(rec, fp=file, indent=4) # Python 对象 => JSON 写入文件 (缩进 4)  
file.close()  
print(open('testjson.txt').read()) # 人类可读的文件内容 (层层缩进)  
P = json.load(open('testjson.txt')) # JSON 文件内容 => Python 对象  
P == rec # True
```

一旦你从 JSON 文本完成转换，就可以在脚本中用常规的 Python 对象操作来处理这些数据了。想了解与 JSON 相关的更多细节，请看 Python 库手册和网上的广泛资料。当然，你也可以把真正的 Python 字典和列表存进一个 Python 的 .py 模块文件里导入，数据也同样可读、可编辑；不过 JSON 不作为代码运行，今天有些人可能认为它更具互操作性。请注意，JSON 中的字符串全部是 Unicode，以便支持更丰富的文本形式。由于 Python 内存中的字符串归根到底就是 Unicode，这种区别在往文件里传输文本、或从文件读出文本时才最重要。你会在第 37 章学会如何把 Unicode 编码应用到 JSON (及其它) 文件和数据上。

代码分析

·解释器如何处理 dumps/loads: json.dumps 递归遍历 Python 对象，按映射表把 dict→对象、list→数组、str→字符串、int/float→数值、bool/None→true/false/n

ull 逐个转成符合 JSON 语法的自字符串；json.loads 调用解析器（scanner）逆向还原。数值、键顺序、默认不缩进等细节都有明确约定。

·与 pickle 的关键差异：JSON 是文本（用文本模式文件），pickle 是二进制（用二进制模式文件）；JSON 支持的类型少（dict/list/str/int/float/bool/None），遇到 set、bytes、自定义对象会报 TypeError；JSON 可被任何语言解析，pickle 只属于 Python。

·dump vs dumps：dumps 返回字符串（内存中），dump 直接写文件流；indent=4 只是美化输出（对人类友好），不影响解析结果——P == rec 不受缩进影响。

·设计思想：JSON 之所以“几乎不费力”，是因为它的数据模型恰好与 Python 的 dict/list 同构；反过来说，Python 三行就能生成/消费 JSON，是这门语言在互操作场景高性价比的体现。

深度理解

·核心概念：三种存储方案的定位对比——手写转换（自定义格式，卷）、pickle（原生对象、Python 专属、二进制）、JSON（通用数据、跨语言、文本）。选型的决定因素是“数据要不要出 Python 世界”。

·底层实现：json 模块默认以严格模式解析（不允许尾逗号、单引号等宽松语法），因为 JSON 是“文本子集”，与 Python 字面量相似但不完全等价——这正是它“几乎 pass 但还差一口气”（书里的 almost passes）的原因。

- 设计原因：要求全 Unicode 字符串，保证全球文本数据在任意语言实现之间安全传递；选择纯文本存储则让数据可读、可 diff、可被 grep。
- 实际问题：前后端 API、NoSQL 文档库、微服务消息、配置文件 (.json) 全覆盖 JSON；只要涉及"跨语言"，JSON 就是第一默认选项；pickle 则留给本语言进程内。
- 初学者误区：把 tuple/set 直接 json.dumps 报错后一脸懵（需先转 list / 自定义 default=）；J 键、元组结构被 loads 还原成 list 而丢失"元组语义"；忘记 ensureascii=False 在控制台看到中文变 \uXXXX。

9.16 Storing Objects with Other Tools (用其它工具存储对象)

英文原文

For other common ways to deal with formatted data files, see the standard library's struct and csv modules. Very briefly, the struct module can both create and parse packed binary data of the sort often shared with C programs:

```
>>> import struct >>> data = struct.pack('i6s', 62, b'Python') # 把 int 和 str 打包成字节>>> data b'>\x00\x00\x00Python'
```

```
>>> file = open('data.bin','wb') # 写/读文件，然后解包>>> file.write(data) >>> file.close() >>> struct.unpa
```

```
ck('i6s', open('data.bin','rb').read()) (62, b'Python')
```

And the csv module parses and creates comma-separated value (CSV) data in files and strings; it doesn't map as directly to Python objects (and requires post-parse conversions), but is another way to map value to and from files:

```
>>> import csv >>> rdr = csv.reader(open('csvdata.txt')) >>> for row in rdr: print(row) ... ['a','bbb','cc','d  
ddd'] ['11','22','33','44']
```

For additional data storage ideas like YAML and SQLite, see the overview of database tools in Chapter 1.

中文翻译

关于处理格式化数据文件的其它常见方式，请看标准库的 struct 和 csv 模块。简单来说，struct 模块既能创建也能解析常与 C 程序共享的那种打包二进制数据：python import struct data = struct.pack('i6s', 62, b'Python') # 'i6s': 一个 int + 一个 6 字节字符串# data -> b'>\x00\x00\x00 Python'file = open('data.bin','wb') file.write(data) # 二进制数据写进二进制文件file.close() struct.unpack('i6s', open('data.bin','rb').read()) # -> (62, b'Python') csv 模块则解析并创建文件与字符串中的逗号分隔值 (CSV) 数据；它不那么直接地映射到 Python 对象（需要解析后的转换），但它是把值映射进/映射出文件的另一种方式：python import csv rdr = csv.reader(open('csvdata.txt')) # 返回一个行迭代器for row in rdr: print(row) # -> ['a','bbb','c
c','dddd'] # -> ['11','22','33','44'] 关于 YAML 和 SQLite

等其它数据存储方案，请看第 1 章对数据库工具的概述。

代码分析

·解释器如何处理 struct: `struct.pack('i6s', 62, b'Python')` 按格式串 'i6s' (i: 4 字节有符号整数; 6s: 恰好 6 字节的字符串) 把 62 编码成 `b'>\x00\x00\x00'` (> 表示小端序), 与 `b'Python'` 拼接成 `b'>\x00\x00\x00Python'`。`unpack` 逆向切回 (62, `b'Python'`)。它是"确定长度、确定布局"的结构化二进制, C 语言 struct 的同款玩法——用于自定义网络协议、二进制文件头解析。

·解释器如何处理 csv: `csv.reader(open(...))` 返回一个行迭代器; 循环到哪解析到哪, 每行按分隔符 (默认逗号) 切成一列一列的字符串列表。注意数字列仍是字符串如 '11'——需要 `int()` 手动转换, 这就是书中所说的 "post-parse conversions"。

·设计思想: 标准库"工具箱"哲学——每种存储用专门的模块: 序列化对象 → pickle; 跨语言文本 → JSON; 定长二进制 → struct; 表格文本 → csv。各司其职, 互不越界。

深度理解

·核心概念: 这批工具覆盖"格式化数据的专用领域"——struct 管定长二进制、csv 管表格文本。它们都不做对象级自动化, 只在"字节/文本 基础结构"之间搭桥。

·底层实现: struct 的格式串是有向压缩的格式语言, 包含字节序、类型码和计数 (如 '!i6s' 表示网络字节序); unp

ack 返回元组是由于"一个字节串可拆出多个值"这一自然语义。

·设计原因：为何 csv 不自动转数字？因为 CSV 本身不携带类型信息，'11' 究竟是数字还是邮编不可知——自动转换反而危险；显式留白让程序自己定夺。

·实际问题：日志导出、Excel 交换、机器学习数据集多用 CSV；协议头解析、图像位图、固件升级包多用 struct。二者都是"具体场景的工程惯例"。

·初学者误区：struct 里格式码与长度不匹配时出现奇怪解码（如把'i6s'当'i4s'）；在 csv 里直接对字符串列做求和/排序；忘记文件要用二进制模式配合 struct。

9.17 File Context Managers (文件上下文管理器)

英文原文

You'll also want to watch for Chapter 34's in-depth discussion of the file's context manager support. Though more a feature of exception processing than files themselves, it allows us to wrap file-processing code in a logic layer that ensures that the file will be closed (and if needed, have its output flushed to disk) automatically on statement exit, instead of relying on the auto-close during garbage collection or manual close calls. As a preview:

with open('data.txt') as myfile: # 文件在 "with" 退出时被关闭
for line in myfile: # 详见第 34 章...use line here...

The with statement closes the temporary file on exit, whether an error occurs or not. The try/finally statement that we'll also study in Chapter 34 can provide similar functionality, but at some cost in extra code—three extra lines, to be precise (though we can often avoid both options and let Python close files for us automatically):

```
myfile = open('data.txt')
try: # 通用的终止处理器
    for line in myfile: # 详见第 34 章...use line here...
        ...
finally: myfile.close()
```

The with context manager scheme ensures release of system resources in all Pythons and may be more useful for output files to guarantee buffer flushes; unlike the more general try, though, it is also limited to objects that support its protocol. Since both these options require more information than we have yet obtained, however, we'll postpone the rest of their stories until later in this book.

中文翻译

你还要留意第 34 章对文件上下文管理器 (context manager) 支持的深入讨论。它与其说是文件本身的特性，不如说是异常处理 (exception processing) 的一个特性：它允许我们把文件处理代码包进一层逻辑，确保文件在语句退出时被自动关闭（如果需要，还会把输出刷新到磁盘），而不

是依赖垃圾回收时的自动关闭或手动 close 调用。先预览一下：
python with open('data.txt') as myfile: # 文件在 "with" 退出时自动关闭
for line in myfile: # 逐行处理...use line here... # 在这里使用这一行
with 语句会在退出时关闭临时文件——无论是否发生错误。我们将在第 34 章学到的 try/finally 语句也能提供类似功能，但要付出额外的代码成本——准确说是三行额外代码（不过我们常常可以两个都不用，直接让 Python 自动帮我们关闭文件）：
python myfile = open('data.txt')
try: # 通用的终止处理器
for line in myfile:...use line here...
finally: myfile.close() # 无论前面发生什么都执行
with 上下文管理器方案能在所有 Python 实现中保证系统资源的释放，对输出文件也更有用——它能保证缓冲被刷新；不过与更通用的 try 不同，它仅限于支持其协议（protocol）的对象。由于这两个选项都需要我们尚未掌握的信息，我们把这些故事的其余部分推迟到本书后面再讲。

代码分析

· 解释器如何处理 with：with open(...) as f: 编译成 SETUPWITH 字节码，运行时调用文件对象的 enter 返回文件对象并绑定给 f；代码块结束时（无论正常、return、还是异常）必然调用 f.exit()——文件对象的 exit 内部就是 "close + flush"。异常发生时也会走 exit，这正是它优于裸 open+close 的关键。

· try/finally 对比：try/finally 是通用结构——它不要求对象支持任何协议，能包住任意代码；with 只对"实现了上下文管理协议（有 enter/exit）"的对象生效，但代码更

省、更不容易忘。

·为什么书中强调"所有 Python 实现": 自动关闭依赖 CPython 的引用计数即时析构, PyPy 等实现不会即时析构; with 则通过显式协议在任何实现下都保证关闭——不依赖 GC 时机是它最大的工程价值。

·设计思想: 资源获取与释放的"成对操作" (acquire/release) 在 Python 中被提炼成 with 语句——这是一等公民级语法糖, 后来的锁、socket、临时文件等都复用同一协议。

深度理解

·核心概念: with = 自动资源管理。它把"打开→使用→无论成败都关闭"压缩成一条语句, 杜绝资源泄漏与"忘记关闭"这类最经典的 bug。

·底层实现: 文件对象实现 enter (return self) 与 exit (内部调 close 并刷新缓冲); with 的求值顺序保证先取到资源再执行代码块, 退出路径统一收敛到 exit。

·设计原因: CPython 引用计数的"自动关闭"不可移植 (书中明言是 implementation artifact), 所以语言提供显式的、规范层面的保证——with 就是为"确定性释放"而生的标准答案。

·实际问题: 写配置文件后立刻要在另一个进程读它、日志刷盘、长循环开文件——凡是"开了就一定要关"的场合, with 是最稳写法; 这也是现代 Python 代码的默认风格。

·初学者误区: 以为 with 只是语法糖没大用 (它解决的是可移植性问题); 忘记 with 的缩进块外文件已关闭, 继

续读写报 ValueError; 把 try/finally 与 with 对立起来 (其实 with 内部就是这样实现的)。

9.18 Other File Tools (其它文件工具)

英文原文

There are additional, more specialized file methods shown in Table 9-2, and even more that are not in the table. For instance, as mentioned earlier, seek resets your current position in a file (the next read or write happens at that position); flush forces buffered output to be written out to disk without closing the connection (by default, files are always buffered); and readlines and writelines process file content in line lists. The Python standard-library manual and other reference resources provide complete details on file methods, but for a quick look, run a dir or help call interactively, passing in open or a file object made with it. And for more file-processing examples, watch for Chapter 37's extended coverage, as well as the sidebar "Why You Will Care: File Scanners", which sketches common file-scanning patterns with statements we have not yet covered here. Also, note that although the open function and the file objects it returns are your main interface to external files in a Python script, there are additional file-related tools in the Python toolset. Prominent among these are: Standard streams — Preope

ned file objects in the `sys` module, such as `sys.stdout`, connected by default to the UI where a script is run (see "Print Operations" for details) Descriptor files in the `os` module — Integer file handles that support low-level tools such as read-only access (see also the "x" mode modifier in `open` for exclusive creation) Sockets, pipes, and FIFOs — File-like objects used to synchronize processes or communicate over networks Access-by-key files known as shelves — Used to store unaltered and pickled Python objects directly, by key (see Chapter 28 for an example) Shell-command streams — Tools such as `os.popen` and `subprocess.Popen` that support spawning shell commands and reading and writing to their standard streams (see Chapter 21 for an `os.popen` example) The third-party open source domain offers even more file-like tools, including support for communicating with serial ports in the `PySerial` extension and interactive programs in the `pep8` system. Consult the web at large for additional information on file-like tools. For code spelunkers, it's also worth noting that Python's `open` function is really just an interface to tools in its standard-library `io` module, which adds logic on top of the underlying system's file tools to make them portable and efficient. If you're looking for docs, implementation details, or customization hooks, look for this module in all the usual places. `io` is really a folder called a module package—a structure for larger code that we'll study later.

中文翻译

表 9-2 中还展示了一些更专门的文件方法，甚至还有更多不在表中的方法。例如，如前所述，`seek` 会重置你在文件中的当前位置（下一次读写就发生在该位置）；`flush` 强制把缓冲输出写到磁盘而不关闭连接（默认情况下文件总是缓冲的）；`readlines` 和 `writelines` 以行列表的形式处理文件内容。Python 标准库手册和其它参考资料提供了文件方法的完整细节，不过快速查看一下也可以：交互式地运行 `dir` 或 `help` 调用，把 `open` 或它创建的某个文件对象传进去。想获得更多文件处理示例，请留意第 37 章的扩展覆盖，以及边栏“你在意什么：文件扫描器”（Why You Will Care: File Scanners）——它用我们这里尚未涉及的语句勾画了常见的文件扫描模式。还要注意：虽然 `open` 函数及其返回的文件对象是 Python 脚本中访问外部文件的主要接口，但 Python 工具集中还有更多与文件相关的工具。其中比较突出的有：标准流（Standard streams）——`sys` 模块中预打开的文件对象，如 `sys.stdout`，默认连接到运行脚本的 UI（详见“打印操作”）`os` 模块中的描述符文件（Descriptor files）——支持只读访问等底层工具的整数文件句柄（另见 `open` 中用于“排他创建”的“`x`”模式修饰符）套接字、管道和 FIFO（Sockets, pipes, and FIFOs）——用于进程同步或网络通信的文件类对象名为 `shelf` 的按键访问文件（Access-by-key files known as shelves）——用于按键直接存储未修改的和 `pickle` 过的 Python 对象（示例见第 28 章）Shell 命令流（Shell-command streams）——`os.popen` 和 `subprocess.Popen` 等工具，支持启动 shell 命令并读写其标准流（`os.popen` 示例见第 21 章）第三方开源

领域还提供了更多文件类工具，包括 PySerial 扩展中对串口通信的支持、以及 pexpect 系统中的交互式程序支持。更多关于文件类工具的信息，请参阅网络上的广泛资料。对喜欢钻研源码的读者（code spelunkers）来说，还值得注意：Python 的 open 函数其实只是其标准库 io 模块中工具的接口，io 模块在底层系统的文件工具之上又加了一层逻辑，使它们可移植且高效。如果你在找文档、实现细节或定制钩子（customization hooks），请在一切惯常的位置寻找这个模块。io 其实是一个被称为模块包（module package）的文件夹——一种用于更大代码的结构，我们以后会研究它。

代码分析

- 解释器如何处理 seek/flush: seek(N) 修改文件对象内部的位置偏移，后续 read/write 从该偏移开始；flush() 把用户区缓冲的内容一次性交给操作系统（不保证立刻落盘——OS 层可能还有缓存，完全落盘要 os.fsync）。readlines()/writelines() 是"整批读取/写入行列表"的便捷接口，行为与逐行循环等价。
- 文件家族的层次：sys.stdout 也是文件对象（文本写入模式）；os 层的整数文件描述符是更底层的 os.read(fd, n) 接口；socket/pipe 是"假装是文件"的类文件对象（file-like）；shelve 则是"键值数据库 + pickle"的合体。统一接口（读写方法）让这些工具可以互换——这是"duck typing（鸭子类型）"的雏形。
- 设计思想：Python 把"凡是能 read/write 的对象"都泛化处理，所以同样的循环既能读文件、也能读 socket、也

能读 `os.popen` 的输出——这种"接口驱动、实现多态"正是 Python 生态工具琳琅满目的根基。

深度理解

·核心概念：文件生态是一个从底到顶的层次结构：系统调用（OS 层）→ `io` 模块（缓冲与编码）→ `open` 返回的文件对象（用户接口）→ 各种文件类对象（`stdin/stdout/socket/pipe/shelve`）。理解层次，才能定位"我的数据卡在哪一层"。

·底层实现：`open` 真正做事的是 `io.open()`；根据模式选择 `TextIOWrapper`、`BufferedReader/BufferedWriter` 或 `FileIO` 堆叠组合——这就是"分层装饰器"模式在标准库里的活教材。

·设计原因：把文件接口做成"大家都会 `read/write`"的统一面孔，是为了让算法代码与具体 I/O 来源解耦——你的解析函数既能吃文件、也能吃网络流。

·实际问题：诊断"为什么数据没落盘"要分清用户缓冲（flush 管）、OS 缓存（`fsync` 管）、磁盘缓存三层；需要文件锁、非阻塞读时再下沉到 `os/select` 层；`shelve` 可以当"迷你数据库"用。

·初学者误区：把 `flush` 当成"立即落盘"（它只保证交给 OS）；在文本与二进制文件对象上调用对方的独有方法；以为 `sys.stdout` 不能重定向（其实它只是变量，可以赋值替换）。

9.19 Core Types Review and Summary (核心类型回顾与总结)

英文原文

Now that we've seen all of Python's built-in objects in action, let's wrap up our object-types tour by reviewing some of the properties they share. Table 9-3 classifies all the major types we've studied so far according to the type categories introduced earlier. Here are some points to remember:- Objects share operations according to their category. For instance, sequence objects—strings, lists, and tuples—all share sequence operations such as concatenation, length, and indexing.- Only mutable objects—lists, dictionaries, and sets—may be changed in place. You cannot change numbers, strings, or tuples in place, but can make a new one with a different value.- Files export only methods, so mutability doesn't really apply to them—their state may be changed when they are processed, but this isn't quite the same as Python object mutability.- "Numbers" in Table 9-3 includes all number types: integer, floating point, complex, decimal, and fraction.- "Strings" in Table 9-3 includes all string types: str for text, as well as bytes for binary data. Exception: the bytearray string type convolutes categorization because it is a mutable string.- Sets are something like the keys of a valueless dictionary, but they don't map to v

values and are not ordered, so sets are neither a mapping nor a sequence type. Exception: frozenset is an immutable variant of set.- In addition to type category operations, all the objects types in Table 9-3 have callable methods in Python today, which are generally specific to their type.

Table 9-3. Object classifications (对象分类)

对象类型	类别 (Category)	可变? (Mutable?)
Numbers (数字)	Numeric (数值型)	No
Strings (字符串)	Sequence (序列)	No
Lists (列表)	Sequence (序列)	Yes
Dictionaries (字典)	Mapping (映射)	Yes
Tuples (元组)	Sequence (序列)	No
Files (文件)	Extension (扩展)	N/A
Sets (集合)	Set (集合)	Yes
Frozenset (冻结集合)	Set (集合)	No
bytearray (字节数组)	Sequence (序列)	Yes

WHY YOU WILL CARE: OPERATOR OVERLOADING (你在意什么: 运算符重载) In Part VI of this book, you'll learn that objects implemented with classes can pick and choose from these categories arbitrarily. For instance, if we want to provide a new kind of specialized sequence object that is consistent with built-in sequences, we can code a class that overloads things like indexing and concatenation: class MySequence: def getitem(self, index): ... # self[index] 等会调用它def add(self, other): ... # self + other 会调用它def iter(self): ... # 迭代时优先使用它 and so on. We can also make the new object mutable or not by selectively implementing

methods called for in-place change operations (e.g., `setitem` is called on `self[index]=value` assignments). Although it's beyond this book's scope, it's also possible to implement new objects in an external language like C as extension types. For these, we fill in C function pointer slots to choose between number, sequence, and mapping operation sets, and similarly choose immutability constraints.

中文翻译

现在我们已经看到所有 Python 内置对象在实战中的样子，让我们通过回顾它们共享的一些性质来结束对象类型之旅。

表 9-3 根据前面介绍的类型类别，把我们研究过的所有主要类型分了类。这里有一些要点要记住：

- 对象按其类别共享操作。例如，序列对象——字符串、列表和元组——都共享拼接、长度、索引等序列操作。
- 只有可变（mutable）对象——列表、字典和集合——可以被就地修改。你不能就地修改数字、字符串或元组，但可以造一个值不同的新对象。
- 文件只导出方法，所以可变性对它们并不真正适用——它们在处理过程中状态可能被改变，但这与 Python 对象可变性的含义不完全相同。
- 表 9-3 中的“数字”包括所有数字类型：整数、浮点数、复数、Decimal 和 Fraction。
- 表 9-3 中的“字符串”包括所有字符串类型：文本用 `str`，二进制数据用 `bytes`。例外：`bytearray` 字符串类型搅乱了分类，因为它是一种可变字符串。
- 集合有点像一个“无值字典”的键，但它们不映射到值、也没有顺序，所以集合既不是映射类型也不是序列类型。例外：`frozenset` 是 `set` 的不可变变体。
- 除了类型类别操作，今天表 9-3 中的所有

对象类型都有可调用 (callable) 的方法，这些方法一般特定于各自的类型。

你在意什么：运算符重载 (Operator Overloading) 在本书第 VI 部分，你会学到用类实现的对象可以随意从这些类别中挑选组合。例如，如果我们想提供一种与内置序列一致的新型专用序列对象，我们可以写一个重载索引和拼接等操作的类：`python class MySequence: def getitem(self, index): ... # self[index] 等操作会调用它 def add(self, other): ... # self + other 会调用它 def iter(self): ... # 迭代时优先调用它 # 等等我们还可以通过有选择地实现就地修改操作所调用的方法 (例如，self[index]=value 赋值会调用 setitem)，来决定新对象是否可变。虽然超出了本书范围，但在 C 等外部语言中把新对象实现为扩展类型 (extension types) 也是可能的。对这些对象，我们填充 C 函数指针槽位，在数字、序列和映射操作集合之间选择，并同样选择不可变性约束。`

深度理解

·核心概念：表 9-3 就是一张“类别 → 操作集 → 可变性”的地图。序列 (str/list/tuple/bytearray) 共享序列操作；映射 (dict) 共享键值操作；集合 (set/frozenset) 共享成员操作；数值共享算术操作。类别决定能力是理解一切对象行为的第一把钥匙。

·底层实现：CPython 中每个类型对象 (PyTypeObject) 都有一组函数指针槽位 (slot)：sqitem (索引)、nbadd (加法)、mpsubscript (映射下标) 等。`L[i]`、`L + x`、`D[k]` 会分别命中不同类型的槽——这就是“类别决定操

作"的 C 层实现，也是 C 扩展类型能"选填槽位"的原因。

·设计原因：用类别而非"每个类型一堆方法"组织操作，让语言可以用极小的规则集合推导出大量行为（比如任何序列都能 len/索引/切片），也让你自己定义类只要实现 `getitem` 就能获得序列般的使用体验。

·实际问题：判断"能排序吗/能哈希吗/能改吗"只需查这张表；给函数传参数时，知道"可迭代即可"（而非"必须是列表"）就能写出更通用的代码。

·初学者误区：把 `bytearray` 想成 `bytes` 的别名（其实是可变版本，能原地改）；以为 `set` 有顺序（它既无序又无下标）；把文件当"可变对象"（它只是状态可变，不支持可变协议）。

9.20 Object Flexibility (对象灵活性)

英文原文

This part of the book introduced a number of compound object types—collections with components. In general:

- Lists, dictionaries, and tuples can hold any kind of object.
- Sets can contain any type of immutable object.
- Lists, dictionaries, and tuples can be arbitrarily nested.
- Lists, dictionaries, and sets can dynamically grow and shrink.

Because they support arbitrary structures, Python's compound object types are good at representing complex information in programs. For example, values in

dictionaries may be lists, which may contain tuples, which may contain dictionaries, and so on. The nesting can be as deep as needed to model the data to be processed.

In code, the following interaction defines a tree of nested compound sequence objects, sketched in Figure 9-1. To access its components, you may include as many index operations as required. Python evaluates the indexes from left to right and fetches a reference to a more deeply nested object at each step. Figure 9-1 may seem a pathologically complicated data structure, but it illustrates the syntax used to access nested objects in general:

```
>> L = ['abc', ((1, 2), ([3], 4)), 5] >> L[1] ((1, 2), ([3], 4))
>> L[1][1] ([3], 4) >> L[1][1][0] [3] >> L[1][1][0][0] 3
```

Figure 9-1. A nested object tree with the offsets of its components

中文翻译

本书的这一部分介绍了许多复合 (compound) 对象类型——带有组成部分的集合。总的说来：- 列表、字典和元组可以容纳任何种类的对象。- 集合可以包含任何类型的不可变对象。- 列表、字典和元组可以任意嵌套。- 列表、字典和集合可以动态地增长和收缩。由于它们支持任意结构，Python 的复合对象类型擅长在程序中表示复杂信息。例如，字典中的值可以是列表，列表里可以包含元组，元组里又可以包含字典，如此等等。嵌套深度可以按需取用，直到足以

刻画要处理的数据。在代码中，下面的交互定义了一棵嵌套复合序列对象的树，如图 9-1 所示。要访问它的组成部分，你可以想写几个索引就写几个。Python 从左到右求值索引，每一步取到更深一层嵌套对象的引用。图 9-1 看起来可能像一种病态复杂的数据结构，但它展示的是一般情况下访问嵌套对象的语法：`python L = ['abc', ((1, 2), ([3], 4)), 5]`
`L[1] # -> ((1, 2), ([3], 4))`
`L[1][1] # -> ([3], 4)`
`L[1][1][0] # -> [3]`
`L[1][1][0][0] # -> 3`

代码分析

·解释器如何处理链式索引：`L[1][1][0][0]` 的求值是从左到右、逐级下钻的：`L[1]` 返回嵌套列表 → 对结果再 `[1]` 返回元组 → 再 `[0]` 返回列表 → 再 `[0]` 返回整数 3。每一步都是“取引用”——索引结果不是拷贝，而是指向内层对象的引用，所以嵌套结构的访问是“顺着指针链走”。

·内存布局：整个结构是一棵对象引用树：列表里存指针，指针指向元组/列表/字符串；叶子是原子对象（str/int）。理解“复合对象=指针数组的递归组合”，嵌套下标就是多次解引用。

·设计思想：复合结构可以“像搭积木一样”组合——同一份数据在不同容器里流动（dict 的 value 是 list，list 的元素是 tuple），这正是 Python 处理 JSON/配置/表格数据时“零建模成本”的来源。

深度理解

·核心概念：复合 = 引用的递归嵌套。容器之间互相嵌套没有深度限制，类型自由组合——这是“Python 适合快速

建模数据"的根本原因。

- 底层实现：list/dict 内部是动态数组/哈希表，存的是 Py Object；嵌套时这些指针指向其它容器对象。任意嵌套的成本仅仅是多几层指针解引用。

- 设计原因：相比强类型语言里"每个结构体都要先声明"，Python 把数据结构建模简化为"字面量随手写"——书里这种"病态复杂"的树在数据处理中其实只是日常。

- 实际问题：层级数据（配置、API 响应、日志结构）天然就是嵌套字典/列表；递归遍历这类结构（如打印树）是常见面试与工程题。

- 初学者误区：链式索引长到 4、5 层就头晕——可以用临时变量逐层取出；把嵌套误以为"深拷贝"（其实每层都是共享引用）；忘记括号优先级 `L[1][1][0][0]` 是从外到内的顺序。

9.21 References Versus Copies (引用与拷贝)

英文原文

Chapter 6 mentioned that assignments always store references to objects, not copies of those objects. In practice, this is usually what you want. Because assignments can generate multiple references to the same object, though, it's important to be aware that changing a mutable object in place may affect other referen

ces to the same object elsewhere in your program. If you don't want such behavior, you'll need to tell Python to copy the object explicitly. We studied this phenomenon in Chapter 6, but it can become more subtle when larger objects of the sort we've explored since then come into play. For instance, the following example creates a list assigned to X, and another list assigned to L that embeds a reference back to list X. It also creates a dictionary D that contains another reference back to list X:

```
>>> X = [1, 2, 3] >>> L = ['a', X, 'b'] # 把 X 的对象的引用嵌入另外两个对象
>>> D = {'x':X, 'y':2} At this point, there are three references to the first list created: from the name X, from inside the list assigned to L, and from inside the dictionary assigned to D. The situation is illustrated in Figure 9-2. Figure 9-2. Shared objects: changing from X makes it look different from L and D too Because lists are mutable, changing the shared list object from any of the three references also changes what the other two reference:
```

```
>>> X[1] = 'surprise' # 改变所有三个引用! >>> L ['a', [1, 'surprise', 3], 'b']
>>> D {'x': [1, 'surprise', 3], 'y': 2}
```

References are a higher-level analogue of pointers in other languages that are always followed when used. Although you can't grab hold of the reference itself, it's possible to store the same reference in more than

one place (variables, lists, and so on). This is a feature—you can pass a large object around a program without generating expensive copies of it along the way. If you really want to avoid the potential side effects of shared references, however, you can request copies, in one of a number of ways:

- Slice expressions with empty limits (`L[:]`) copy sequences.
- The dictionary, set, and list copy method (`X.copy()`) copies a dictionary, set, or list.
- Some built-in functions, such as `list` and `dict` make copies (`list(L)`, `dict(D)`, `set(S)`).
- The copy standard-library module makes full ("recursive") copies when needed. For example, say you have a list and a dictionary, and you don't want their values to be changed through other variables:

```
>>> L = [1,2,3] >>> D = {'a':1,'b':2} To prevent this,
simply assign copies to the other variables, not references to the same objects:
```

```
>>> A = L[:] # 而不是 A = L (或 list(L)、L.copy()) >>
> B = D.copy() # 而不是 B = D (集合同理) This way, changes made from the other variables will change the copies, not the originals:
```

```
>>> A[1] = 'Py' >>> B['c'] = 'code' # 改变的是副本，不是原对象
>>> L, D # 原对象不受影响([1, 2, 3], {'a': 1, 'b': 2})
```

```
>>> A, B # 副本被改动了(['Py', 3], {'a': 1, 'b': 2, 'c': 'code'})
```


In terms of our original example, you can avoid the reference side effects by slicing the original list instead of simply naming it:

```
>>> X = [1, 2, 3] >>> L = ['a', X[:], 'b'] # 嵌入 X 的对象的副本
>>> D = {'x':X[:], 'y':2} This changes the picture in Figure 9-2—L and D will now point to different lists than X. The net effect is that changes made through X will impact only X, not L and D; similarly, changes to L or D will not impact X. One final note on copies: empty-limit slices and the dictionary copy method only make top-level copies; that is, they do not copy nested data structures, if any are present. If you need a complete, fully independent copy of a deeply nested data structure (like the various record structures we've coded in recent chapters), use the standard copy module, introduced in Chapter 6:
```

```
import copy X = copy.deepcopy(Y) # 完整拷贝任意嵌套的对象 Y This call traverses objects to copy all their parts, no matter how deep they may be. This is a much rarer case, though, which is why you have to say more to use this scheme. References are usually what you will want; when they are not, slices and copy methods are usually as much copying as you'll need to do.
```

中文翻译

第 6 章提到过：赋值总是存储对对象的引用，而不是这些对象的副本。实践中，这通常正是你想要的。不过，由于赋值可以产生对同一对象的多个引用，重要的是要意识到：就

地修改一个可变对象，可能会影响程序中其它地方对同一对象的其它引用。如果你不想要这种行为，就需要明确告诉 Python 拷贝这个对象。我们在第 6 章研究过这种现象，但当自那以后我们探索的更大对象登场时，它会变得更微妙。例如，下面的示例创建一个赋值给 X 的列表，以及另一个赋值给 L 的、嵌回对列表 X 的引用的列表。它还创建了一个字典 D，其中包含另一个对列表 X 的引用：
`python X = [1, 2, 3] L = ['a', X, 'b'] # 在另外两个对象中嵌入对 X 的对象的引用 D = {'x': X, 'y': 2}` 此时，对创建的第一个列表存在三个引用：来自名字 X、来自赋值给 L 的列表内部、以及来自赋值给 D 的字典内部。这种情况如图 9-2 所示。图 9-2：共享对象——从 X 修改会让它在 L 和 D 那里看起来也变了因为列表是可变的，从三个引用中的任何一个修改这个共享的列表对象，另外两个引用所指向的内容也会随之改变：
`python X[1] = 'surprise' # 三个引用全都变了！ # L -> ['a', [1, 'surprise', 3], 'b'] # D -> {'x': [1, 'surprise', 3], 'y': 2}` 引用是其它语言中指针（pointer）的高级类比物——指针在使用时总是被跟随（dereference）。虽然你无法抓住引用本身，但可以把同一引用存放在多个地方（变量、列表等等）。这是一个特性——你可以在程序中传递一个大对象，而无需沿途生成昂贵的副本。不过，如果你真的想避免共享引用的潜在副作用，可以通过多种方式请求副本（copies）：
- 无边界切片表达式（L[:]）拷贝序列。
- 字典、集合和列表的 copy 方法（X.copy()）拷贝一个字典、集合或列表。
- 一些内置函数，如 list 和 dict，会制造副本（list(L)、dict(D)、set(S)）。
- 需要时，copy 标准库模块可以制造完整的（“递归”）副本。例如，假设你有一个列表和一个字典，不想让它们的值通过其它变量被改变：
`python L = [1,`

2, 3] D = {'a': 1, 'b': 2} A = L[:] # 赋值给 A 的是 L 的副本，而不是 A = L 那种引用 B = D.copy() # B 是 D 的副本，而不是 B = D A[1] = 'Py' # 修改的是副本 B['c'] = 'code' # 修改的是副本 # L, D -> ([1, 2, 3], {'a': 1, 'b': 2}) 原对象不变 # A, B -> ([1, 'Py', 3], {'a': 1, 'b': 2, 'c': 'code'}) 副本被改回到最初的示例，你可以通过切片原始列表而不是直接命名它，来避免引用副作用：python X = [1, 2, 3] L = ['a', X[:], 'b'] # 嵌入 X 的对象的副本 D = {'x': X[:], 'y': 2} 这改变了图 9-2 中的画面——L 和 D 现在指向与 X 不同的列表。净效果是：通过 X 做出的修改只影响 X，不影响 L 和 D；同样，对 L 或 D 的修改也不会影响 X。关于拷贝的最后一点说明：无边界切片和字典 copy 方法只制造顶层 (top-level) 副本；也就是说，如果存在嵌套数据结构，它们不会拷贝嵌套结构。如果你需要一个深度嵌套数据结构（比如我们最近几章写过的各种记录结构）的完整、完全独立的副本，请使用第 6 章介绍的标准 copy 模块：python import copy X = copy.deepcopy(Y) # 完整拷贝任意嵌套的对象 Y 这个调用会遍历对象以拷贝其所有组成部分，无论它们嵌套多深。不过这是一个罕见得多的场景，这正是为什么你必须多说几个字才能使用这一方案。引用通常才是你想要的；当不是的时候，切片和 copy 方法通常就是你所需的全部拷贝量。

代码分析

·解释器如何处理赋值：L = ['a', X, 'b'] 只是把 X 对象的内存地址（引用）复制进新列表的元素槽位——没有复制任何数据。三处引用（X 名字、L 内部、D 内部）指向同一个 [1,2,3] 对象；X[1] = 'surprise' 修改的是共享对象本身

，所以 L、D 中看到的内容"同步变化"。这就是引用计数机制下"共享引用 (shared reference)"的完整图景。

·浅拷贝 (shallow copy) 真相：L[:] 制造新列表对象，但新列表槽位里存的还是原来的元素引用——一层容器是新的，内部对象仍是共享的。所以"浅拷贝"只解决"顶层容器"的独立性；嵌套对象照样共享 (9.33 节有更深的坑)。

·深拷贝 (deepcopy) 机制：copy.deepcopy(Y) 递归遍历对象图，用 memo 字典记录"原对象→副本"的映射，防止循环引用时无限递归；凡是遇到已拷贝过的对象就直接返回既有副本。代价是遍历开销大、可能拷贝你不想拷的资源（如文件句柄）。

·设计思想：Python 默认"赋值即引用"而不是"赋值即拷贝"，是为了性能和共享——传大对象零拷贝；需要独立副本时再显式提出，用不同的工具表达不同的拷贝深度。这就是"显式优于隐式"在拷贝语义上的体现。

深度理解

·核心概念：三条纪律——赋值永远存引用；可变对象共享可能产生跨界副作用；拷贝有深浅之分（浅拷贝=新容器+共享元素，深拷贝=全新对象图）。

·底层实现：浅拷贝在 C 层是"新建容器 + memcpy 引用数组"；深拷贝是递归构造。CPython 引用计数让"共享"零成本，也让"忘记拷贝"的 bug 往往在别的函数里炸出来——排查难度大。

·设计原因：默认共享让 Python 高效，显式拷贝让程序员掌握主动权；四档拷贝工具（切片/copy 方法/内置转换/c

opy 模块) 对应"从只拷贝一层到拷贝一切"的需求谱系。

- 实际问题: 函数参数默认就是引用传递——函数内 `list.append` 会改动调用方的列表; 缓存、日志、防篡改数据、undo 功能需要拷贝; 深拷贝用于"复制配置快照"这类场景。
- 初学者误区: `A = L` 后再改 `A` 发现 `L` 也变了 (惊讶值拉满); 以为 `[:]` 是深拷贝; 在 9.33 节的 `[L]` 4 上栽跟头; 对不可变对象 (数字、字符串、元组) 谈拷贝没有意义——它们没有可变副作用。

9.22 Comparisons, Equality, and Truth (比较、相等与真值)

英文原文

All Python objects also respond to comparisons: tests for equality, relative magnitude, and so on. We've seen comparison at work on specific objects in earlier chapters but can finally summarize the general rules. In short, Python comparisons always inspect all parts of compound objects until a result can be determined. When nested objects are present, Python automatically traverses data structures to apply comparisons from left to right, and as deeply as needed. The first difference found along the way determines the comparison result. This is sometimes called a recursive comparison—the same comparison requested on the top

-level objects is applied to each of the nested objects, and to each of their nested objects, and so on, until a result is found. Later in this book (Chapter 19) you'll learn how to write recursive functions of your own that work similarly on nested structures. For now, think about comparing all the linked pages at two websites if you want a metaphor for such structures, and a reason for writing recursive functions to process them. In terms of core objects, the recursion is automatic. For instance, a comparison of list objects compares all their components automatically until a mismatch is found or the end is reached:

```
>>> L1 = [1, ('a', 3)] # 值相同，但是不同的对象
>>> L2 = [1, ('a', 3)]
>>> L1 == L2, L1 is L2 # 值相等？是同一个对象？ (True, False)
```

Here, L1 and L2 are assigned lists that are equivalent but distinct objects. As a review of Chapter 6's coverage, because of the nature of Python references, there are two ways to test for equality: - The `==` operator tests value equivalence. Python performs an equivalence test, comparing all nested objects recursively. - The `is` operator tests object identity. Python tests whether the two are really the same object (i.e., live at the same address in memory). In the preceding example, L1 and L2 pass the `==` test (they have equivalent values because all their components are equivalent) but fail the `is` check (they reference two different objects, and hence two different pieces of memory). Notice wh

at happens for short strings, though:

```
>>> S1 = 'text' >>> S2 = 'text' >>> S1 == S2, S1 is S2  
(True, True)
```

Here, we should again have two distinct objects that happen to have the same value: `==` should be true, and `is` should be false. But because Python internally caches and reuses some objects as an optimization, there really is just a single string `'text'` in memory, shared by `S1` and `S2`. Hence, the `is` identity test reports a true result. To trigger the normal behavior, we need to use strings that are longer (or otherwise defeat caching rules prone to change over time):

```
>>> S1 = 'a longer string' >>> S2 = 'a longer string' >>>  
>> S1 == S2, S1 is S2 (True, False)
```

Of course, because strings are immutable, the object caching mechanism is irrelevant to your code—strings can't be changed in place, regardless of how many variables refer to them. If identity tests seem confusing, see Chapter 6 for a refresher on object reference concepts. As a rule of thumb, the `==` operator is what you will want to use for almost all equality checks; `is` is reserved for highly specialized roles. You'll see use cases for both later. As demoed along the way, relative magnitude comparisons are also applied recursively to nested data structures:

```
>>> L1 = [1, ('a', 3)] # 嵌套的 3 大于嵌套的 2 >>> L2  
= [1, ('a', 2)] >>> L1 < L2, L1 == L2, L1 > L2 # 小于、
```

等于、大于：结果元组(False, False, True)

Here, L1 is greater than L2 because the nested 3 is greater than the nested 2. More broadly, Python compares its core object types as follows: - Numbers are compared by relative magnitude, after conversion to the common highest type if needed (e.g., $1 < 1.1$ after 1 is replaced with 1.0).- Strings are compared lexicographically (by the character code-point values returned by `ord`), and character by character until the end or first mismatch (e.g., `'abc' < 'ac'`).- Lists and tuples are compared by comparing each component from left to right, and recursively for nested structures, until the end or first mismatch (e.g., $[1, 3] > [1, 2]$).- Sets are equal if both contain the same items (formally, if each is a subset of the other), and magnitude comparison operators for sets apply subset and superset tests.- Dictionaries compare as equal if their sorted (key, value) lists are equal. Magnitude comparisons are not supported for dictionaries but can be coded by comparing manually sorted items results. - Nonnumeric mixed-type magnitude comparisons (e.g., $1 < \text{'text'}$) are errors. By proxy, this also applies to sorts, which use comparisons internally: nonnumeric mixed-type collections cannot be sorted sans conversions. In general, comparisons of structured objects proceed as though you had written the objects as literals and compared all their parts one at a time from left to right. In later chapters, you'll also see that class-based objects can c

change the way they are compared. Here, the following sections provide a few more details on Python's built-in comparisons.

中文翻译

所有 Python 对象也对比较做出响应：相等、相对大小等测试。我们在前几章已经见过比较在特定对象上的工作方式，但现在终于可以总结出一般规则了。总之，Python 比较总是检查复合对象的所有部分，直到能够确定结果为止。存在嵌套对象时，Python 自动遍历数据结构，从左到右应用比较，深度按需扩展。沿途发现的第一个差异决定比较结果。这有时被称为递归比较（recursive comparison）——对顶层对象提出的同一个比较，会被应用到每个嵌套对象，再应用到它们各自的嵌套对象，依此类推，直到找到结果。本书后面（第 19 章）你会学会如何写出自己对嵌套结构做类似工作的递归函数。现在，如果你想为这样的结构找个比喻，以及一个写递归函数处理它们的理由，就想想比较两个网站上所有互相链接的页面吧。就核心对象而言，这种递归是自动的。例如，对列表对象的比较会自动比较它们的所有组成部分，直到发现不匹配或到达末尾：

```
python L1 = [1, ('a', 3)] # 值相同，但却是不同的对象
L2 = [1, ('a', 3)] L1 == L2, L1 is L2 # 值相等？ (True) 是同一个对象？ (False)
```

这里，L1 和 L2 被赋以等价的（equivalent）却是不同的对象。回顾第 6 章的内容：由于 Python 引用的性质，测试相等有两种方式：- == 运算符测试值等价（value equivalence）。Python 执行等价测试，递归地比较所有嵌套对象。- is 运算符测试对象同一性（object identity）。Python 测试两者是否真的是同一个对象（即是否驻留在内存的

同一地址)。在前面的示例中，L1 和 L2 通过了 == 测试（因为它们的所有组成部分都等价，所以它们的值等价），但没通过 is 检查（它们引用两个不同的对象，因而引用两块不同的内存）。不过注意短字符串有奇妙行为：python S1 = 'text' S2 = 'text' S1 == S2, S1 is S2 # -> (True, True) 这里，我们本应再次得到两个碰巧有相同值的、不同的对象：== 应为真，is 应为假。但因为 Python 出于优化目的内部会缓存 (cache) 并复用某些对象，内存中实际上只有一个字符串 'text'，由 S1 和 S2 共享。因此 is 同一性测试报告了真结果。要触发正常行为，我们需要用更长的字符串（或者以其它方式打破那些容易随时间变化的缓存规则）：python S1 = 'a longer string' S2 = 'a longer string' S1 == S2, S1 is S2 # -> (True, False) 当然，因为字符串是不可变的，对象缓存机制对你的代码无关紧要——无论有多少变量引用它们，字符串都不能就地改变。如果同一性测试让你困惑，请回到第 6 章复习对象引用的概念。作为经验法则，几乎所有的相等性检查你都会想用 == 运算符；is 留给高度专用的角色。两者对应的使用场景你后面都会见到。正如一路演示的那样，相对大小比较也同样递归地应用到嵌套数据结构上：python L1 = [1, ('a', 3)] # 嵌套的 3 大于嵌套的 2 L2 = [1, ('a', 2)] L1 < L2, L1 == L2, L1 > L2 # 小于、等于、大于：结果构成元组 # -> (False, False, True) 这里 L1 大于 L2，因为嵌套的 3 大于嵌套的 2。更广泛地说，Python 对其核心对象类型的比较规则如下：- 数字在需要时先转换到共同的最高类型，再按相对大小比较（例如 1 被替换成 1.0 后执行 1 < 1.1）。- 字符串按字典序 (lexicographically) 比较——依据 ord 返回的字符码点值，逐字符比较直到结尾或首次失配（例如 'abc' < 'ac'）。-

列表和元组从左到右逐个比较每个组件，对嵌套结构递归执行，直到结尾或首次失配（例如 $[1, 3] > [1, 2]$ ）。- 集合在两者包含相同条目时相等（正式地说：如果互为对方的子集）；对集合的大小比较运算符执行子集（subset）和超集（superset）测试。- 字典在它们的排序后的（key, value）列表相等时比较为相等。字典不支持大小比较，但可以通过手动比较排序后的 items 结果来编码实现。- 非数值型的混合类型大小比较（例如 $1 < \text{'text'}$ ）是错误。间接地，这同样适用于排序——排序内部使用比较：不做转换，就无法对非数值型的混合类型集合排序。一般来说，结构化对象的比较按你“把对象写成字面量、从左到右逐部分比较”的方式推进。在后面章节你还会看到：基于类的对象可以改变它们被比较的方式。在这里，下面几节对 Python 内置比较提供更多细节。

代码分析

·解释器如何处理 $==$ 与 is ： $L1 == L2$ 编译为 COMPARE OP $==$ ，运行时分派到 PyObjectRichCompare，对列表触发 listrichcompare——逐元素递归比较； $L1 is L2$ 编译为 ISOP，内部只是比较两个指针地址是否相同，不做任何递归。一慢一快，语义完全不同。

·字符串缓存（intern）现象：CPython 对短的、看起来像标识符的字符串会驻留（intern）——同一内容共享一个对象，所以 $S1 is S2$ 为 True。这不是语言保证，是优化实现；长字符串不驻留， is 就按预期返回 False。教训：永远别用 is 比较字符串内容。

·递归大小比较的求值过程： $L1 > L2$ 从第一个元素 $1 ==$

1 继续；到第二个元素 ('a',3) > ('a',2)：先比 'a' == 'a'，再比 3 > 2，找到差异，得出 L1 更大，返回 True。整个链条是"逐层递归、找到第一个不同点即停"。

·设计思想：比较协议让"相等/大小"成为每个对象的可递归属性，而不是某几个容器的特权——这是算法能透明地 sort/== 任何结构化数据的基础。

深度理解

·核心概念：两种"相等"——==（值相等）与 is（同一性相等）；复合对象比较自动递归且"从左到右、深挖到底、首个差异定胜负"。

·底层实现：比较最终会收敛到原子比较（数字比数值、字符串比码点）；dict 相等是优化过的特殊算法（见 9.24）；不同类别之间的大小比较直接抛 TypeError，由 C 层类型检查拦截。

·设计原因：递归比较让语言"开箱即用"地支持 sorted(apples) 这类需求，不用为每种结构写 comparator；is 提供给"必须区分对象是否同一"的场景（如单例检查、None 检查）。

·实际问题：排序、去重、查重、查找都依赖递归比较；而 if x is None（判断占位）与 if x == None 风格之争大多可归结为"性能+单例语义偏好"。

·初学者误区：用 is 比较字符串/数字内容（缓存骗了你或坑了你）；期待 1 < '1' 有输出而非 TypeError；以为字典可以直接 < 比较（不行，见下节）。

9.23 Mixed-Type Comparisons and Sorts (混合类型比较与排序)

英文原文

The preceding section's last bullet point applies only to nonnumeric mixed-type magnitude tests, not equality, but it also applies by proxy to sorting, which does magnitude testing internally. Python disallows mixed-type magnitude testing, except for numeric types and manually converted types:

```
>> 11 == '11' # 相等可以, 但大小不行 False >> 11 >= '11'
TypeError: '>=' not supported between instances of 'int' and 'str'
>> ['11', '22'].sort() # 排序同样如此 >> [1, '11'].sort()
TypeError: '<' not supported between instances of 'str' and 'int'
>> 11 > 9.123 # 混合数字转换到最高类型 True
>> str(11) >= '11', 11 >= int('11') # 手动转换解决了问题
(True, True) >> [11, '11'].sort(key=str) # 排序同理: 见第 8 章
```

中文翻译

上一节的最后一条只适用于非数值型的混合类型大小测试, 不适用于相等测试, 但它也间接适用于排序——排序内部会做大小测试。Python 禁止混合类型的大小测试, 数字类型和手动转换过的类型除外: `python 11 == '11' # -> False`。相等测试没问题, 但大小测试不行 `11 >= '11' # TypeError`。

ror:'>=' 不支持 int 与 str 之间的比较[11,'22'].sort() # 全是字符串，可以排序（按字典序）[11,'11'].sort() # TypeError:'<' 不支持 str 与 int 之间的比较11 > 9.123 # -> True。混合数字会自动转换到最高类型（float）str(11) >= '11', 11 >= int('11') # -> (True, True) 手动转换强制可行[11,'11'].sort(key=str) # 用 key=str 把元素都按字符串形式排序

代码分析

·解释器如何处理 TypeError: 11 >= '11' 触发数字类型的比较槽 nrichcompare, C 层检查另一个操作数不是数值类型，直接抛 TypeError——不强行猜测语义，因为 "11" 这种表示哪种含义（字符串还是数字）没法自动推断。

·为什么相等可以、大小不行：11 == '11' 是"值等价"测试，不同类型的对象不相等即返回 False（这是安全的默认）；但大小比较要求建立"谁大谁小"的全序关系，跨类型没有定义，Python 坚决报错而不是给出可能误导的结果。

·key 参数的作用：[11,'11'].sort(key=str) 把每个元素先映射成 str（都是字符串）再比较，从而绕过混合类型限制——这是把"如何比较"交给程序员定义的机制（详见第 8 章），避免了默认行为的武断。

·数值家族例外：11 > 9.123 中 int 被提升为 float 再比较——数值类型共享一套算术优先级模型，所以内部可以统一到"最高类型"再比。

深度理解

- 核心概念：跨类型比较有一条清晰边界——数值内部可互通，其余混合类型"大小比较"一律拒绝，但"相等测试"安全返回 False。这条规则保护你免于语义含糊的比较结果。
- 底层实现：Python 3 时代（对比 2.x）是"严格跨类型比较"的坚定执行者：任何未定义顺序的比较都抛 `TypeError`，而不是像 Python 2 那样按类型名碰运气排序——这是刻意修复的历史教训。
- 设计原因：现实数据经常"看起来都是数字"（`'11' vs 11`），武断排序会出微妙 bug；让你显式转换 = 让你意识到类型差异。
- 实际问题：从 CSV/表单读来的全是字符串，排序前必须显式 `int()`；混合类型列表排序报错的第一反应应是"统一类型或给 key"。
- 初学者误区：想当然 `11 < '11'` 会"智能比较"；`[11, '11'].sort()` 报错后不知所措；混淆 `key` 参数（映射后再比）与 `cmp`（Python 2 的旧机制）。

9.24 Dictionary Comparisons (字典比较)

英文原文

As noted in Chapter 8, magnitude comparisons don't work for dictionaries directly. Though subject to implementation morph, this purportedly reflects that fact

that magnitude comparison would incur too much overhead and may hamper the more common equality test, which may use an optimized scheme that doesn't compare sorted key/value lists:

```
>> D1 = {'b':3,'a':1} >> D2 = {'a':1,'b':3} >> D1 == D2
# 相等, 而非大小 True >> D1 < D2
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

To work around this limitation, either write loops to compare values by key, or, as also shown in Chapter 8, simply compare sorted key/value lists manually by combining the items dictionary method and sorted built-in:

```
>> list(D1.items()) [('b', 3), ('a', 1)] >> sorted(D1.items())
[('a', 1), ('b', 3)] >> sorted(D1.items()) < sorted(D2.items())
# 字典大小测试 False >> sorted(D1.items()) > sorted(D2.items())
True
```

This takes more code than a simple < or > and may run relatively slowly; but in practice, most programs requiring this behavior either will develop more efficient ways to compare data in dictionaries or won't care about the sloth.

中文翻译

如第 8 章所述, 大小比较不能直接用于字典。虽然这受实现形态变化影响, 但据说这反映了这样一个事实: 大小比较会造成过多开销, 并且可能妨碍更常见的相等测试——相等测试可能使用一种优化方案, 而不去比较排序后的键/值

列表: python `D1 = {'b': 3, 'a': 1}` `D2 = {'a': 1, 'b': 3}` `D1 == D2` # -> True。做的是相等测试而非大小测试 `D1 < D2` # `TypeError: '<' 不支持 dict 与 dict 之间的比较`要绕过这个限制, 要么写循环按键比较值, 要么——如第 8 章也展示的——把 `items` 字典方法与 `sorted` 内置函数结合起来, 手动比较排序后的键/值列表: python `list(D1.items())` # -> `[('b', 3), ('a', 1)]` (插入顺序) `sorted(D1.items())` # -> `[('a', 1), ('b', 3)]` (按键排序) `sorted(D1.items()) < sorted(D2.items())` # 字典"大小"测试 -> False `sorted(D1.items()) >= sorted(D2.items())` # -> True 这比简单的 `<` 或 `>` 需要更多代码, 而且运行得可能相对较慢; 但实践中, 需要这种行为的多数程序要么会开发出更高效的比较字典数据的方法, 要么根本不在乎这点慢。

代码分析

·解释器如何处理 `D1 == D2`: dict 相等用优化算法——先比大小 (元素个数) 快速排除, 再对每个键查值并比较, 无需排序; 复杂度 $O(n)$ 。这就是书中"可能使用一种不必比较排序列表的优化方案"的含义。

·为什么 `<` 被禁用: 字典是"无序键值映射"语义, 大小比较没有天然定义; 若真要实现, 得先按键排序 ($O(n \log n)$ + 开销), 会拖慢"相等"场景的判断逻辑, 且语义众说纷纭——Python 干脆砍掉。

·手动比较方案: `sorted(D1.items())` 生成按键排序的 (key,value) 元组列表, 之后用列表的递归比较法则——真做到了"字典大小测试", 代价是 $O(n \log n)$ 与临时对象分配。

·设计思想："为高频操作（相等）优化，为低频/歧义操作（字典大小）留手"——Python 常在效率、语义清晰度与一致性之间做务实取舍。

深度理解

·核心概念：类型能"`==`"但不一定能"`<`"——相等性与有序性是正交的能力。`dict` 提供前者，不提供后者；需要大小时要绕道（`items + sorted`）。

·底层实现：`dict` 相等走哈希键查找（ $O(n)$ ）；大小比较若支持就要排序（ $O(n \log n)$ ）。CPython 的选择是不为低频歧义功能付出高频性能代价。

·设计原因：大小比较"看似简单实则语义不清"（按键？按值？长度？嵌套规则？），不给默认实现是对混乱的防御。

·实际问题：需要按字典内容排序（比如字典列表排序）的常态做法是 `sorted(dicts, key=lambda d: ...)`，直接比较字典本身很少是正确需求。

·初学者误区：以为 `D1 < D2` 会按"字典长度"或"键字母序"给个答案——它只会抛 `TypeError`；用 `sorted(items)` 绕道时忘了它生成的是新列表、比较的是元组序列。

9.25 The Meaning of True and False in Python (Python 中 True 与 False 的含义)

英文原文

Notice that the test results returned in the last two examples represent true and false values. They print as the words True and False, but now that we're using logical tests like these in earnest, it's time to be a bit more formal about what these names really mean. In Python, as in most programming languages, an integer 0 represents false, and an integer 1 represents true (a heritage rooted in the digital nature of computer hardware). In addition, though, Python recognizes any empty data structure as false and any nonempty data structure as true. More generally, the notions of true and false are intrinsic properties of every object in Python—each object is either true or false, as follows:

- Numbers are false if zero, and true otherwise.
- Collection objects are false if empty, and true otherwise.
- The None placeholder object is always false.
- True and False are preset to true and false, respectively.

Table 9-4 gives examples of true and false values of various objects in Python.

Table 9-4. Example object truth values (示例对象真值)

对象	值
'text'	True
''	False
[1, 2]	True
[]	False
{'a': 1}	True

<code>{}</code>	False
<code>1</code>	True
<code>0.0</code>	False
<code>None</code>	False

As one application of this, because objects are true or false themselves, it's common to see Python programmers code tests like `if X:`, which, assuming `X` is a string, is the same as `if X!="":`. In other words, you can test the object itself to see if it contains anything, instead of comparing it to an empty—and therefore false—object of the same type.

中文翻译

请注意，最后两个示例返回的测试结果代表真（true）值和假（false）值。它们打印成单词 True 和 False，但既然我们现在要认真使用这样的逻辑测试了，是时候更正式地看待这些名字的真正含义了。和大多数编程语言一样，在 Python 中整数 0 表示假，整数 1 表示真（其渊源扎根于计算机硬件数字化的本质）。不过除此之外，Python 还把任何空数据结构视为假、任何非空数据结构视为真。更一般地说，真与假的概念是 Python 中每个对象的固有属性——每个对象要么为真、要么为假，规则如下：- 数字为零即为假，否则为真。- 集合对象为空即为假，否则为真。- None 占位对象总是为假。- True 和 False 分别被预置为真与假。表 9-4 给出了 Python 中各种对象的真假值示例。作为这一点的应用之一：因为对象本身为真或假，常见到 Python 程序员写出 `if X:` 这样的测试——假设 `X` 是字符串，它与 `if X!="":` 相同。换句话说，你可以直接测试对象本身，看它是

否包含任何内容，而不必把它与同类型的空对象（因此是假的）相比较。

深度理解

- 核心概念：真值测试（truth testing）是对象的固有属性，而非只能来自比较表达式——if X: 读作“如果 X 有内容”，是最地道的 Python 风格。
- 底层实现：CPython 用 PyObjectIsTrue 判断：数字看是否为零，容器看 len() > 0，None/False 直接为假，真值表可被自定义类通过 bool/len 覆盖。
- 设计原因：“空即假”让代码极简洁（if data: 代替 if len(data) > 0:），且语义一致：空字符串、空列表、空字典、0、None 全部“塌缩”为一个假值，减少了处处写空判断的噪声。
- 实际问题：读文件后 if content: 判断是否读到东西；循环 while queue: 处理到队列清空；返回值校验 if result:。这是 Python 程序员每天都要依赖的约定。
- 初学者误区：把 if X == True:（生硬）当标准写法；在“0 是假”与“0.0 是假”上栽跟头（数字按值判断）；误以为' 与" 一样（空格串非空，是真值！）。

9.26 The None Object (None 对象)

英文原文

As shown in the last row in Table 9-4, Python also pr

provides a special object called `None`, which is always considered to be false. `None` was introduced briefly in Chapter 4; it is the only value of a special data type in Python and typically serves as an empty placeholder (much like a `NULL` pointer in C). For example, recall that for lists you cannot assign to an offset unless that offset already exists—the list does not magically grow if you attempt an out-of-bounds assignment. To preallocate a list such that you can store values in any of its offsets, you can fill it with `None` objects:

```
>>> size = 50
>>> L = []
>>> L[size - 1] = 'NO'
IndexError: list assignment index out of range
>>> L = [None] * size
>>> L[size - 1] = 'OK'
>>> L[-10:]
[None, None, None, None, None, None, None, None, 'OK']
```

This doesn't limit the size of the list (it can still grow and shrink later), but simply presets an initial size to allow for future index assignments. You could initialize a list with zeros the same way, of course, but best practice suggests using `None` if the type of the list's contents is variable or not yet known. Keep in mind that `None` does not mean "undefined." That is, `None` is something, not nothing (despite its name!)—it is a real object and a real piece of memory that is created and given a built-in name by Python itself. Watch for other uses of this special object later in the book; as you'll learn in Part IV, it is also the default return value of functions that don't exit by running into a return statement with a result value.

中文翻译

如表 9-4 最后一行所示，Python 还提供了一个名为 None 的特殊对象，它总是被视为假。None 在第 4 章简要介绍过；它是 Python 中一个特殊数据类型唯一的取值，通常充当空占位符（很像 C 中的 NULL 指针）。例如，回忆一下：对于列表，你不能给一个尚不存在的偏移量赋值——如果你试图越界赋值，列表并不会神奇地增长。为了预分配一个列表、使你可以在它的任何偏移量处存值，你可以用 None 对象填满它：python size = 50 L = [] L[size - 1] = 'NO' # IndexError: 列表下标赋值超出范围（列表是空的！）L = [None] size # 预分配：50 个 None 引用L[size - 1] = 'OK' # 现在可以给第 49 号偏移赋值L[-10:] # -> 九个 None + 一个'OK'这并不会限制列表的大小（以后它仍然可以增长和收缩），只是预置一个初始大小，以便未来的下标赋值可行。当然，你也可以用零来初始化列表，但如果列表内容的类型是可变的或尚不确定，最佳实践建议使用 None。请记住，None 并不意味着“未定义”。也就是说，None 是“某种东西”，而不是“什么都没有”（尽管它名字如此！）——它是一个真实的对象、一块真实的内存，由 Python 自己创建并赋予一个内置名字。留意这个特殊对象在本书后面的其它用途；你将会在第 IV 部分学到：它也是没有通过 return 语句带着结果值退出的函数默认的返回值。

代码分析

·解释器如何处理 [None] size：先把 None 求值为单元元素列表，再重复 50 次——50 个槽位全部存同一个 None 对象的引用。本利 None 不可变，共享无副作用；如果换

成 `[]` size 共享的就是同一个空列表（9.33 的坑）。`L[size-1] = 'OK'` 用下标赋值给已存在的槽位，OK。

·为什么首次赋值报 `IndexError`：空列表长度为 0，`L[49] = 'NO'` 是越界——Python 的列表不允许按下标凭空扩展（相比字典可以 `D['新键'] = 值` 自动加项），这是 list 与 dict 的语义分界。

·None 到底是什么：None 是单例对象（整个进程只有一个实例），也是 `NoneType` 的唯一值；任何没有显式 `return` 的函数都隐式返回它。`L[-10:]` 切片里那 9 个 None 全是同一个对象。

·设计思想：用 None 占位 = 显式声明“这里将来会有值，但现在为空”——比用魔数 0 更有语义（0 可能是合法数据，None 清晰表示“缺位”）。

深度理解

·核心概念：None = Python 的“空值/占位符”单例。它与 C 的 NULL 神似但本质不同：NULL 是“指针为 0（无效）”，None 是“一个真实存在的对象”。

·底层实现：PyNone 是编译期内置的全局单例；`is None` 检查就是指针比对，所以判空一律推荐 `if x is None` 而非 `x == None`（后者还可行，但 `is` 更贴近语义且不可被类重写）。

·设计原因：有个“确定的空”能让 API 表达“可选”“尚未赋值”“无结果”，避免到处发明哨兵值（-1、0、''），增进可读性。

·实际问题：:return 时输出 None、dict.get(key) 查不到时返回 None、队列的占位、数据库可选字段的映射——None 无处不在。

·初学者误区：把 None 当"什么都没有"，试图对 None 调用方法 (None.xxx → AttributeError)；在布尔测试里依赖"None 是假"却忘了空字符串/空列表也是假，从而误判两者的区别；用 is 之外的方式判空也容易踩自定义对象的陷阱。

9.27 The bool Type (bool 类型)

英文原文

While we're on the topic of truth, also keep in mind that the Python Boolean type bool, introduced in Chapter 5, simply augments the notions of true and false in Python. As we learned earlier, the built-in words True and False are just customized versions of the integers 1 and 0—it's as if these two words have been preassigned to 1 and 0 everywhere in Python. Because of the way this new type is implemented, this is really just a minor extension to the notions of true and false already described, designed to make truth values more explicit:— When used explicitly in truth test code, the words True and False are equivalent to 1 and 0, respectively, but they make the programmer's intent clearer.— Results of Boolean tests run interactively print as the words True and False, instead of as 1 and 0, to

make the type of result clearer. You are not required to use only Boolean types in logical statements such as if; all objects are still inherently true or false, and all the Boolean concepts mentioned in this chapter still work as described if you use other types. Python also provides a bool built-in function that can be used to extract the Boolean value of an object. You can use this to explicitly check if an object is true—that is, nonzero or nonempty: >>> bool(1) True >>> bool('text') True >>> bool({}) False In practice, though, you'll rarely notice the Boolean type produced by logic tests, because Boolean results are used automatically by if statements and other selection tools. We'll explore Booleans further when we study logical statements in Chapter 12.

中文翻译

既然聊到真值这个话题，还要记住：第 5 章介绍的 Python 布尔类型 bool，只是增强了 Python 中真与假的概念。正如我们之前学到的，内置词 True 和 False 只是整数 1 和 0 的定制版本——就好像这两个词在整个 Python 中被预先赋值为 1 和 0 一样。由于这个新类型的实现方式，它其实只是对已描述的真假概念的一个小扩展，设计目的是让真值更显式：- 当在真值测试代码中显式使用时，True 和 False 分别等价于 1 和 0，但它们让程序员的意图更清晰。- 交互运行布尔测试的结果会打印成单词 True 和 False，而不是打印成 1 和 0，以便让结果的类型更清楚。在 if 这样的逻辑语句中，并不要求你只使用布尔类型；所有对象本质上照

样为真或为假，而且即便你使用其它类型，本章提到的所有布尔概念仍然按所描述的方式工作。Python 还提供了一个 `bool` 内置函数，可以用来提取一个对象的布尔值。你可以用它显式检查一个对象是否为真——也就是非零或非空：`python bool(1) # -> True bool('text') # -> True bool({}) # -> False` 不过在实践中，你很少注意到逻辑测试产生的布尔类型，因为布尔结果会被 `if` 语句和其它选择工具自动使用。我们在第 12 章研究逻辑语句时，会进一步探索布尔值。

代码分析

- 解释器如何处理 `True`：`True` 和 `False` 是 `int` 的子类，其值就是 1 和 0；`1 + True` 得到 2、`True == 1` 为 `True`——因为 `bool` 继承自 `int` 且没有重新定义算术。
- `bool()` 的求值：`bool(x)` 内部就调用 `PyObjectIsTrue(x)`，等价于在 `if x:` 里对 `x` 做的判断——非零数字、非空容器得到 `True`。
- 设计思想：让 `bool` 成为 `int` 的子类，是为了与 C 传统（0/1）兼容且省去运行时类型判断；打印为 `True/False` 则提升可读性——“既保留兼容，又改善体验”，是典型的 Python 式渐进设计。

深度理解

- 核心概念：`bool` 是“显式的真值类型”——本质上就是 0/1，但打印和书写更清晰。真值判断本身不依赖 `bool`，任何对象都有真假。

- 底层实现: bool 是 int 的子类 (isinstance(True, int) 为 True) , 只有 True/False 两个单例实例。
 - 设计原因: 为什么 True 不是 String? 为了兼容 C 风格与数值运算惯性 (索引、计数器里 bool 可直接当 0/1 用) , 同时避免开发者为"用什么表示成功/失败"发明各色约定。
 - 实际问题: if x: vs if bool(x): 在绝大多数场景等价, 前者更地道; 做数量统计时 sum(x > 0 for x in data) 直接利用了 True==1。
 - 初学者误区: 以为 bool('False') 是 False (它是非空字符串, 是 True!) ; 在 if 里写 if x == True (可用 if x) ; 忘了'0' 是非空字符串也是真值。
-

9.28 Python's Type Hierarchies (Python 的类型体系)

英文原文

As a summary and reference, Figure 9-3 sketches all the major built-in object types available in Python and their relationships. We've explored the most prominent of these in this part of the book. Other objects in Figure 9-3 are program units (e.g., functions and modules) or interpreter internals (e.g., stack frames and compiled code). The main point to notice here is that everything processed in a Python program is an object type. This is sometimes called a "first class" object

model, because all objects are on equal footing with respect to your code. For instance, you can pass a class to a function, assign it to a variable, stuff it in a list or dictionary, and so on.

中文翻译

作为总结与参考，图 9-3 勾勒了 Python 中可用的所有主要内置对象类型及其关系。本书这一部分已经探索了其中最重要的部分。图 9-3 中的其它对象是程序单元（program unit，如函数和模块）或解释器内部对象（如栈帧 stack frame 和编译后的代码 compiled code）。这里要注意的要点是：Python 程序中处理的一切都是对象类型。这有时被称为“一等对象”（first class）对象模型，因为在你写代码的语境下，所有对象都处于平等地位。例如，你可以把一个类传给函数、把它赋给变量、把它塞进列表或字典，等等。

深度理解

- 核心概念：“一切皆对象”（everything is an object）——不只是数据，连函数、模块、类、甚至异常和代码都是对象。这是“一等对象”模型：类型之间没有“高贵的”与“低贱的”之分。
- 底层实现：CPython 中几乎每个 C 结构的开头都是一个 PyObjectHEAD（引用计数 + 类型指针），这就是“万物皆对象”的物理基础；函数、类型、模块等都对应各自的 PyObject 变体。
- 设计原因：一等对象让元编程成为可能——把类当数据传来传去（工厂模式）、把函数塞进字典做分发表（comm

and dispatch) 、把模块作为参数注入——这是设计自由度的源泉。

·实际问题：按名称动态选函数 (`getattr(mod, name)`) 、装饰器 (返回函数的函数) 、策略字典 `{'json': json.loads, 'pickle': pickle.loads}` 都是一等对象的直接产物。

·初学者误区：把"类名"与"实例"混淆 (函数里传的是类本身还是对象) ；以为 `type(x)` 就是 `x` 的类型"字符串" (它返回的是类型对象) ；忘了一等对象意味着可以"把行为当数据"传递。

9.29 Type Objects (类型对象)

英文原文

In fact, even types themselves are an object type in Python: the type of an object is an object of type `type` (and not just because it's a decent tongue twister!). Seriously, a call to the built-in function `type(X)` returns the type object of object `X`. The practical application of this is that type objects can be used for manual type comparisons in Python if statements. However, for reasons introduced in Chapter 4 that we won't rehash here, manual type testing is usually not the right thing to do in Python, since it limits your code's flexibility. Python is about flexibility, not constraints. One note on type names: each core type has a built-in name that supports various roles, including type customiza

tion through object-oriented subclassing: dict, list, str, tuple, int, float, complex, bytes, type, set, and more. Technically speaking, these names reference classes, and calls to these names are really object constructor calls, not simply conversion functions, though you can treat them as simple functions for basic usage. In addition, the types standard-library module provides additional type names for types that are not available as built-ins (e.g., types.FunctionType is the type of functions), and the isinstance built-in function checks types with consideration of inheritance in OOP—a topic we'll reach later on our Python journey. Because types can be customized with OOP in Python, though, the isinstance technique is generally recommended in the very rare cases where code must know about specific types. There's more on type customizations in Chapter 32, and an example in which isinstance is useful and warranted in Chapter 19. Figure 9-3. Python's major built-in object types, organized by categories

中文翻译

事实上，在 Python 中连类型本身就是一种对象类型：一个对象的类型是一个 type 类型的对象（这可不只是为了玩一个漂亮的绕口令！）。说正经的，对内置函数 type(X) 的调用会返回对象 X 的类型对象。它的实际应用是：类型对象可以在 Python 的 if 语句中用于手动类型比较。不过，出于第 4 章介绍过的、我们这里不再赘述的原因，在 Python 中手动类型测试通常不是正确的做法，因为它限制了代码的

灵活性。Python 讲究灵活性，而不是约束。关于类型名有一点说明：每个核心类型都有一个内置名字，支持各种角色，包括通过面向对象子类化（subclassing）进行类型定制：dict、list、str、tuple、int、float、complex、bytes、type、set 等等。严格来说，这些名字引用的是类（classes），对这些名字的调用其实是对象构造器（constructor）调用，而不仅仅是转换函数——虽然你可以把它们当简单函数用于基本用途。此外，types 标准库模块提供了一些内置中不可用类型的额外类型名（例如，types.FunctionType 是函数的类型），而 isinstance 内置函数在检查类型时会考虑 OOP 中的继承关系——这是我们 Python 旅程后面才会触及的话题。不过，由于在 Python 中类型可以用 OOP 定制，在极少数“代码必须知道具体类型”的情况下，一般推荐 isinstance 的手法。更多关于类型定制的内容在第 32 章；第 19 章有一个 isinstance 适用且值得使用的示例。图 9-3：Python 的主要内置对象类型，按类别组织

深度理解

·核心概念：类型本身也是对象——type(X) 告诉你 X 是什么；type 是所有类的类（元类 metaclass 的入口）。“类的类还是类”是理解动态语言反射能力的关键一环。

·底层实现：CPython 中每个对象头都存一个指向其 PyTypeObject 的指针；type(X) 就是取出这个指针。isinstance(x, int) 还会向上回溯 mro（方法解析顺序）考虑继承。

·设计原因：list、str 这些名字是类，所以 list([1,2]) 不是“转换”而是“构造新对象”——第 9.21 节说“内置函数制造

副本"的深层原因就是：它们是构造器，总会新建对象。

·实际问题：手动 `type(x) == str` 是生硬的法维（且不认 `str` 的子类）；建议优先用 `isinstance()`；只有极少数场景（如需要精确对象类别）才用 `type` 严格比对。

·初学者误区：`type(x) == list` vs `isinstance(x, list)` 差异不了解（前者连子类都不认）；以为 `type(3)` 返回 `'int'` 这个字符串（返回的是 `<class'int'>` 类型对象）；对 OOP 还陌生时理解"类是对象"会费解——记住"类在 Python 里能当参数传"就够了。

9.30 Other Types in Python (Python 中的其它类型)

英文原文

Besides the core objects studied in this part of the book, and the program-unit objects such as functions, modules, and classes that you'll meet later, a typical Python installation has dozens of additional object types available as linked-in C extensions or imported Python classes—regular expression objects, GUI widgets, network sockets, and so on. Depending on whom you ask, the named tuple you met earlier in this chapter may fall in this category too, along with `Decimal` and `Fraction` of Chapter 5. The main difference between these extra tools and the built-in types you've seen so far is that the built-ins have language-defined syn

tax for creating their objects (e.g., 4 for an integer, [1, 2] for a list, the open function for files, and def and lambda for functions). Other tools are made available in standard-library modules that you import to use. For instance, to make a regular-expression object in pattern matching, you import re and call re.compile(). Because most objects in this noncore category are application-level tools that are beyond the scope of this language tutorial, be sure to browse Python's library reference early and often in your coding career for a comprehensive chronicle of all the supplemental tools available to Python programs.

NOTE: 那 range 呢? : Python 的文档把内置 range 函数重新归类为序列类型, 与列表和元组并列, 但这是学术上的障眼法, 本书不采纳。正如你稍后会看到的, range 返回一个可迭代 (iterable) 对象, 它按需产生结果, 而不是一个物理存储的序列。它支持一部分——但不是全部——序列操作 (例如索引可以、拼接不行), 但即便如此这也是一种实现技巧, 远不足以构成一个与真正序列同级别的"新类型"。把 range 标记为"整数的不可变序列"混淆了工具类别, 也让 Python 学习者困惑。

中文翻译

除了本书这一部分研究的核心对象, 以及你以后会遇到的函数、模块、类等程序单元对象之外, 一个典型的 Python 安装还有数十个额外的对象类型, 它们要么以链接进来的 C 扩展形式存在、要么以导入的 Python 类形式存在——正则表达式对象、GUI 控件、网络套接字等等。看你去问谁,

本章前面遇到的 `named tuple`（命名元组）也可能被归入这一类，还有第 5 章的 `Decimal` 和 `Fraction`。这些额外工具与你目前见过的内置类型之间的主要区别是：内置类型有语言定义的、创建其对象的语法（例如用 `4` 表示整数、`[1,2]` 表示列表、用 `open` 函数表示文件、用 `def` 和 `lambda` 表示函数）。而其它工具通过你要 `import` 才能使用的标准库模块提供。例如，要在模式匹配中创建一个正则表达式对象，你 `import re` 并调用 `re.compile()`。因为这一类非核心（`noncore`）对象大多是超出本语言教程范围的应用程序级工具，请务必在编程生涯早期、并且经常地翻阅 Python 的库参考手册，那里有 Python 程序可用的全部辅助工具的综合编年史。

说明：那 `range` 呢？：本节是原书 NOTE 的完整译文（见上面原文框内）。

深度理解

- 核心概念：对象类型分两大家族——核心类型（有语言专用字面量/语法）与扩展类型（`import` 后用）。判据不是“重不重要”，而是有没有语法口子：`4`、`[1,2]`、`open`、`def` 属于前者，正则对象、套接字、`namedtuple` 属于后者。
- 底层实现：内置类型的对象创建由编译器/字节码直接支持（如字面量编译成 `LOADCONST`）；扩展类型则由库代码在其工厂函数（如 `re.compile`、`socket.socket`）里构造。
- 设计原因：“语法只留给最核心的少数”，其余能力全部走 `import`——这保证了语言内核小而精（语言规范稳定），规模庞大的生态在标准库和第三方扩展里生长。

·实际问题：日常开发 90% 的需求由标准库 (re、json、socket、collections、pathlib) 满足——多翻文档、多记标准库地图，是效率最高的学习投资。

·初学者误区：对 range 的身份困惑是经典教材问题——本书与官方文档口径不同（官方叫"序列"，本书说"可迭代对象"）；理解 range 是"惰性生成器式对象"比贴什么标签重要得多。不区分"核心语法"与"库类型"，导致在自定义类型上寻找字面量语法而不可得。

9.31 Built-in Type Gotchas (内置类型陷阱)

英文原文

That's the end of our look at core data types. We'll wrap up this part of the book with a discussion of common problems that seem to trap new users (and the occasional expert), along with their solutions. Some of this is a review of ideas we've already covered, but these issues are important enough to warrant callouts again here.

中文翻译

这就是我们对核心数据类型的考察的结束。我们将通过讨论一些似乎总爱绊倒新用户（偶尔也绊倒专家）的常见问题及其解决方案，来为本部分收尾。这部分内容有些是对我们已覆盖概念的复习，但这些问题的分量足以在此再次点名强调。

深度理解

- 核心概念：陷阱（gotchas）区总结的是对象模型的"反直觉边界"——共享引用、重复一层深、循环结构、不可变约束。它们不是语法错，而是"语义出乎意料"的坑。
- 底层实现：所有本节陷阱都源于同一个根：赋值存引用而非拷贝；"只拷贝一层"是切片/拼接/重复的统一行为。
- 设计原因：默认零拷贝让性能好，代价是需要理解引用的传播；这些陷阱就是这份"默认"的说明书。
- 实际问题：面试最爱考、调试最常撞——背熟这四个坑，等于给自己装上免疫疫苗。
- 初学者误区：把每个陷阱当孤例记诵，而不理解"都因引用/浅拷贝而起"这根主线。

9.32 Assignment Creates References, Not Copies（赋值制造引用，而非拷贝）

英文原文

Yes, this is redundant, but it's such a common pitfall that it's worth underscoring one more time: shared references to mutable objects can matter. In the following, for instance, the list object assigned to the name `L` is referenced both from `L` and from inside the list assigned to the name `M`. Changing `L` in place changes what `M` references, too:

```
>>> L = [1, 2, 3] >>> M = ['X', L, 'Y'] # 嵌入一个对 L
的引用>>> M ['X', [1, 2, 3], 'Y']
```

```
>>> L[1] = 0 # 也改变了 M (M 所引用的对象) >>> M
['X', [1, 0, 3], 'Y']
```

This effect usually becomes important only in larger programs, and shared references are often exactly what you want. If objects change out from under you in ways unexpected and unwanted, though, you can avoid sharing objects easily by copying them explicitly. For lists, you can always make a top-level copy by using an empty-limits slice, among other techniques described earlier in this chapter:

```
>>> L = [1, 2, 3] >>> M = ['X', L[:], 'Y'] # 嵌入 L 的副本
>>> L[1] = 0 # 只改变 L, 不影响 M >>> L [1, 0, 3]
>>> M ['X', [1, 2, 3], 'Y']
```

Remember, slice limits default to 0 and the length of the sequence being sliced; if both are omitted, the slice extracts every item in the sequence and so makes a top-level copy (a new, unshared object).

中文翻译

是的，这有点重复，但它是如此常见的陷阱，值得再一次强调：对可变对象的共享引用确实会带来麻烦。例如在下面这段代码中，赋值给名字 L 的列表对象既被 L 引用，又被赋值给名字 M 的列表内部引用。就地修改 L 也会改变 M 所引用的东西：python L = [1, 2, 3] M = ['X', L, 'Y'] # 把对 L 的引用嵌入 M # M -> ['X', [1, 2, 3], 'Y'] L[1] = 0 # 连

M 里看到的内容也变成了 [1, 0, 3] 这个效应通常只在更大程序里才显得重要，而且共享引用往往正是你想要的。但如果对象以你未曾预料、也不想要的方式在背后被改掉了，你可以通过显式拷贝来轻松避免共享。对列表来说，无边界切片 (empty-limits slice) 总能做出一份顶层副本——本章前面描述过的技术之一：`python L = [1, 2, 3] M = ['X', L[:], 'Y']` # 嵌入 L 的一层拷贝 `L[1] = 0` # 只改 L，M 里还是 [1, 2, 3] 记住：切片边界默认是 0 和被切序列的长度；两个都省略时，切片取出序列中的每一个条目，从而得到一份顶层拷贝（一个全新的、不共享的对象）。

代码分析

·解释器如何处理：`M = ['X', L, 'Y']` 把 L 对象的地址写进 M 的元素槽；`L[1] = 0` 通过 L 的名字寻址到同一个对象。两处“视图”其实看的是同一块内存，所以修改彼此可见。改用 `L[:]` 后，M 槽位里写的是新列表对象的地址——两个对象互不相干。

·切片为何能拷贝：`L[:]` 等价于 `L[0:3]`（省略下限取 0、省略上限取长度），结果是一个新列表，元素引用（浅）拷入——对“顶层容器独立”这个需求，它恰到好处。

·设计思想：“拷贝是显式动作”——语言不猜你要引用还是要副本，因为省一次拷贝的性能收益太普遍；把选择权交给你，正是“明确优于含蓄”反过来的体现：默认共享，需要独立时开口。

深度理解

·核心概念：可变对象的共享引用会"牵一发动全身"。这不只是列表独有——函数把参数套在引用里传进来，函数里改了，调用方也看到了。

·底层实现：引用计数模型下"复制引用"零开销，"复制对象"要分配内存 + 循环拷贝——默认共享是性能的自然选择。

·设计原因：共享引用让大数据结构在程序各处传递时零拷贝，是 Python 快的一个隐性原因。

·实际问题：协同编辑的列表、嵌套在配置树里的对象、跨函数传的大结构，都要时刻分辨"我要的是视图还是副本"

。

·初学者误区：只在 REPL 小例子里没意识到；等对象被塞进 3 层嵌套后才被"神秘修改"击中——调试这类 bug 的大部分时间都在找"谁在背后改了我的数据"。

9.33 Repetition Adds One Level Deep (重复只加深一层)

英文原文

As we've learned, repeating a sequence is like adding it to itself a number of times. However, when mutable sequences are nested, the effect might not always be what you expect. For instance, in the following example X is assigned to L repeated four times, wherea

s Y is assigned to a list containing L repeated four times:

```
>> L = [4, 5, 6] >> X = L * 4 # 就像 [4, 5, 6] + [4, 5, 6] + ... >> Y = [L] * 4 # [L] + [L] + ... = [L, L, ...] >> X [4, 5, 6, 4, 5, 6, 4, 5, 6, 4, 5, 6] >> Y [[4, 5, 6], [4, 5, 6], [4, 5, 6], [4, 5, 6]]
```

Subtly, because L was nested in the second repetition, Y winds up embedding references back to the original list assigned to L, and so is open to the same sorts of side effects noted in the preceding section:

```
>> L[1] = 0 # 影响 Y, 但不影响 X >> X [4, 5, 6, 4, 5, 6, 4, 5, 6, 4, 5, 6] >> Y [[4, 0, 6], [4, 0, 6], [4, 0, 6], [4, 0, 6]]
```

This may seem artificial and academic—until it happens unexpectedly in your code! The same solutions to this problem apply here as in the previous section, as this is really just another way to create the shared mutable object reference case—make copies when you don't want shared references:

```
>> L = [4, 5, 6] >> Y = [list(L)] * 4 # 嵌入 L 的 (共享的) 副本 >> L[1] = 0 >> Y [[4, 5, 6], [4, 5, 6], [4, 5, 6], [4, 5, 6]]
```

Even more subtly, although Y doesn't share an object with L anymore, it still embeds four references to the same copy of it. If you must avoid that sharing too, you'll want to make sure each embedded copy is uni

que:

```
>> Y[0][1] = 99 # 四个副本仍然全都一样 >> Y [[4, 99,
6], [4, 99, 6], [4, 99, 6], [4, 99, 6]] >> L = [4, 5, 6] >> Y
= [list(L) for i in range(4)] >> Y [[4, 5, 6], [4, 5, 6], [4, 5
, 6], [4, 5, 6]] >> Y[0][1] = 99 # 而现在它们不一样了!
>> Y [[4, 99, 6], [4, 5, 6], [4, 5, 6], [4, 5, 6]]
```

If you remember that repetition, concatenation, and slicing copy only the top level of their operand objects, these sorts of cases make much more sense.

中文翻译

正如我们所学，重复一个序列就像把它自身相加若干次。然而，当可变序列被嵌套时，效果不一定总是你预期的那样。例如，下面示例中 X 被赋值为 L 重复四次，而 Y 被赋值为一个包含 L 的列表重复四次：python L = [4, 5, 6] X = L * 4 # 四个 4/5/6 铺开：X -> [4,5,6,4,5,6,4,5,6,4,5,6] Y = [L] * 4 # 四个内容相同的子列表：Y -> [[4,5,6],[4,5,6],[4,5,6],[4,5,6]] 微妙之处在于：因为 L 在第二次重复中被嵌套了，Y 最终嵌回的是对赋给 L 的原始列表的引用，因此对上一节提到的同类副作用毫无防备：python L[1] = 0 # 影响 Y（四个子列表里全是 0），但不影响 X（铺开的 1 离开原对象了）# Y -> [[4, 0, 6], [4, 0, 6], [4, 0, 6], [4, 0, 6]] 这可能看起来像人为制造的学院派问题——直到它在你的代码里意外出现！解决问题的方法与上一节相同，因为这实际上只是“创建共享可变对象引用”的另一种形式——当你不想共享引用时就制造副本：python L = [4, 5, 6] Y = [list(L)] * 4 # 嵌入 L 的一份（各个槽共享的）拷贝 L[1] = 0 # L 变了，但 Y 保持 [[4,5,6],...] 更微妙的是：虽然 Y 不再与 L 共享对

象，它仍然嵌着四个对同一份拷贝的引用。如果你也必须避免这种共享，就要确保每个嵌入的副本是独有的：python

```
Y[0][1] = 99 # 四个槽指向同一副本：一改全改 -> [[4,99,6]×4] L = [4, 5, 6] Y = [list(L) for i in range(4)] # 每次迭代都新建一个独立副本
```

Y[0][1] = 99 # 现在只有第一个子列表被改如果你记住：重复、拼接和切片只拷贝它们操作数对象的顶层，这类案例就很好理解了。

代码分析

·解释器如何处理 `X = L 4` 与 `Y = [L] 4`：两者都只重复"元素引用数组"。`L 4` 复制的是整数对象的引用（整数不可变，共享无害），结果是 12 个元素的平铺列表；`[L] 4` 把同一个 `L` 对象的引用复制 4 份放进新列表，所以嵌套层共享——`L[1]=0` 通过共享对象传遍 `Y`。

·`list(L)` 的"半程修复"：`[list(L)]4` 先把 `L` 浅拷贝成一个新列表，然后 4 个槽还是引用同一个新列表——所以 `Y[0][1]=99` 依旧四格齐变。

·最终修复：`[list(L) for i in range(4)]` 每次循环执行 `list(L)` 生成新的独立列表，4 个槽指向 4 个不同对象，改一个不影响其它。

·设计思想：规则很好记——重复/拼接/切片只在最外层造新容器，内层全是引用共享；需要"深层独立"时逐层显式拷贝（推导式 + `list()` 是最简洁工具）。

深度理解

·核心概念：重复操作对"顶层容器"复制、对"元素引用"共享。这个"只深一层" (one level deep) 的语义是理解一

切拷贝陷阱的钥匙。

·底层实现：[L] 4 在 C 层就是"新建长度为 4 的列表，槽位 0-3 全部填入同一个 L 指针"——指针复制，零对象创建，所以"看似 4 份"其实 1 个。

·设计原因：逐层都深拷贝会让 [1]5 这类常用操作慢且浪费；选择"顶层新建 + 内层共享"是性能与正确性的折中，代价是需要程序员自觉。

·实际问题：初始化二维矩阵 [[0]5]3 是新手最著名的翻车点（行共享）；哈希表、去重表等"每行独立"结构要用推导式初始化。

·初学者误区：[[0]5]3 打印出来"看起来对"，改一行却连坐三行；以为有"复制元素"语义；不记得拼接/切片同样只浅拷贝。

9.34 Beware of Cyclic Data Structures (当心循环数据结构)

英文原文

We encountered this concept in a prior exercise but didn't explore it much: if a collection object contains a reference to itself, it's called a cyclic object. Python prints a [...] whenever it detects a cycle in the object back to itself, rather than getting stuck in an infinite loop (as it once did long ago, when dinosaurs roamed the planet):

```
>>> L = ['stuff'] >>> L.append(L) # 追加
```

对同一对象的引用>>> L # 造成回到同一对象的循环: [...]
['stuff', [...]] Besides understanding that the three dots in square brackets represent a cycle in the object (should they ever crop up in your outputs or job interviews!), this case is worth knowing about because it can lead to real gotchas—cyclic structures may cause code of your own to fall into unexpected loops if you don't anticipate them. For instance, some programs that walk through structured data must keep a list, dictionary, or set of already visited items, and check it when they're about to step into a cycle that could cause an unwanted loop. See the Part I exercise solutions in "Part I, Getting Started" in Appendix B for more on this problem. Also watch for general discussion of recursion in Chapter 19, as well as the reloadall.py program in Chapter 25 and the ListTree class in Chapter 31 for concrete examples of programs where cycle detection can matter. As is so often the case in programming, the solution is knowledge: don't use cyclic references unless you really need to, and make sure you anticipate them in programs that must care. There are good reasons to create cycles, but unless you have code that knows how to handle them, objects that reference themselves may be more liability than asset. They need not, however, also be a surprise.

中文翻译

我们在之前的一个练习中遇到过这个概念，但没有深究：如果一个集合对象包含对自身的引用，它被称为循环对象（cyclic object）。每当 Python 检测到对象中存在回到自身的循环时，它会打印一个 [...], 而不是陷入死循环（很久很久以前，恐龙还在地球上游荡的时代，它确实会死循环）：

```
python L = ['stuff'] L.append(L) # 把对同一个对象的引用追加进自身L # 造成回到自身的循环 -> ['stuff', [...]] ([...] 即 L 自己)
```

除了要明白方括号里的三个点代表对象中的一个循环（无论它是出现在你的输出里还是面试题里！），这桩案例值得了解是因为它可能导致真正的陷阱——如果你不预先防范，循环结构可能让你自己的代码陷入意外的循环。例如，一些遍历结构化数据的程序必须维护一张已访问（already visited）条目的列表、字典或集合，并在将要踏入可能造成不必要循环之前检查它。关于这个问题，参见附录 B 中“第 I 部分，入门”的练习解答。还要关注第 19 章对递归的一般讨论，以及第 25 章的 reloadall.py 程序和第 31 章的 ListTree 类——这些都是循环检测可能很重要的程序的具体例子。和编程中常见的情况一样，解决方案就是知识：除非你确实需要，否则不要使用循环引用；并确保在必须关心的程序中预先防范它们。创建循环有正当理由，但除非你有会处理它们的代码，否则引用自身的对象可能是负担而非资产。不过，它们也大可不必成为意外。

代码分析

·解释器如何处理循环：L.append(L) 把 L 自己的引用存进 L 自己的元素槽——对象引用图出现了环。repr 打印时进行递归描述，遇到“回到已输出对象”的引用就输出 [...] 中止，避免无限递归。CPython 的 repr 算法内部用一个“所

见即记录"的集合判断环。

·为什么旧版会死循环：早期 Python 的 repr 没有环检测，遇到自引用对象无限递归、栈溢出或卡死——如今 [...] 是防卫机制的痕迹。

·遍历含环结构：任何"往下走"的算法（例如深度遍历树）遇到环可能绕圈不止；标准对策是记录 visited：进入节点前先查是否见过，见过就跳过。

·设计思想：垃圾回收器必须能处理环（CPython 的 cycle detector 专为此存在）；而程序员遍历时，也要像 GC 一样做好"已访问"记账——认识环是编写健壮递归算法的前提。

深度理解

·核心概念：循环对象 = 自引用的结构。它合法（GC 能回收）但危险（遍历很可能死循环）；[...] 是解释器的"环警报"。

·底层实现：CPython 的 repr 用回溯栈 + 已见集合检测环；垃圾回收侧，引用计数管不了环，所以有分代式循环引用收集器（cycle collector）专门回收自引用组。

·设计原因：允许环的存在（链表、图的表示需要），靠"repr 防呆 + GC 专门收集 + 遍历需记账"三件套来兜底，而不是禁止它。

·实际问题：图/双向链表/缓存 LRU 需要环；数据 dump 成 JSON 时自引用会直接报错或栈溢出——需要自行检测或转换。

·初学者误区：看到 [...] 以为是数据损坏；忘了自己在列表里塞过自身；写遍历时没做 visited 保护，程序在奇怪的地方"挂起"。

9.35 Immutable Types Can't Be Changed in Place (不可变类型不能就地修改)

英文原文

And just once more for completeness: you cannot change an immutable object in place. Instead, you construct a new object with slicing, concatenation, and so on, and assign it back to the original reference, if needed: `T = (1, 2, 3) T[2] = 4` # 错误! `T = T[:2] + (4,)` # 可以: `(1, 2, 4)` That might seem like extra coding work (and it is), but the upside is that most of the previous gotchas in this section can't happen when you're using immutable objects like tuples and strings; because they can't be changed in place, they are not generally open to the sorts of side effects that can imperil mutable objects like lists and dictionaries.

中文翻译

为完整性再说最后一次：你不能就地改变不可变对象。相反，你要用切片、拼接等方式构造一个新对象，如果需要再把它赋回原引用：`python T = (1, 2, 3) T[2] = 4` # 错误：元组不支持条目赋值 `T = T[:2] + (4,)` # 可行：用切片 + 拼接构造新元组 `(1, 2, 4)` 那看起来像是额外的编码工作（确实

如此)，但也有好处：使用元组和字符串这类不可变对象时，本节前面的大多数陷阱都不会发生；因为它们不能就地改变，所以通常不会遭受那种可能危害列表和字典等可变对象的副作用。

代码分析

·解释器如何处理：T[2] = 4 触发不可变类型的 setitem——元组对象设置过不可变标志，C 层直接抛 TypeError。
T[:2] + (4,) 依次做切片（新元组 (1,2)）和拼接（新元组 (1,2,4)），最后 T = ... 让名字 T 重新指向新对象——原来的 (1,2,3) 仍原封不动。

·为什么"额外代码"值得：不可变对象没有"原地修改"，就失去了共享引用的副作用土壤——上一节"改一处、连坐多处"的病在字符串/元组上根本不存在。

·设计思想：两种类型的哲学对比——可变类型把修改权交给调用者（灵活高风险），不可变类型把数据锁死（安全高可信）。选择哪种，取决于"数据要流动还是要被守"。

深度理解

·核心概念：不可变 = 不能就地改，改 = 造新的再赋值。
这是"可靠共享"的代价，也是它的收益。

·底层实现：str/tuple/int/float/frozenset 的类型槽里就没有可写方法；任何"修改"操作都编译为"创建新对象"。

·设计原因：不可变对象可安全地作为字典键/集合元素、可被无数变量共享、可放心作为函数默认参数——以"改需新建"的小代价换来全局一致的安全。

·实际问题：把"不会变"的身份数据（ID、路径、状态码）设计成 str/元组，意味着别处拿到它就改不了——天然防呆。

·初学者误区：想当然地 `T[2]=4`（应化作切片+拼接）；`S = S.replace(...)` 后以为旧字符串变了（其实名字指向了新对象）；不可变类型间互相转换时忽略了"重构"成本。

9.36 Chapter Summary (本章小结)

英文原文

This chapter explored the last two major core object types—the tuple and the file. We learned that tuples support all the usual sequence operations, have just a few methods, do not allow any in-place changes because they are immutable, and are generalized by the named-tuple extension type. We also learned that files are returned by the built-in open function and provide methods for reading and writing content of both the text and binary kind. Along the way we explored how to translate Python objects to and from strings for storing in files, and we looked at pickle, json, and other modules for advanced roles (object serialization and binary data). Finally, we wrapped up by reviewing some properties common to all object types (e.g., shared references) and went through a list of common mistakes ("gotchas") in the object-type domain. In the next part of this book, we'll shift gears, turning to

the topic of statement syntax—the way we code processing steps and logic in our scripts. Along the way, this next part explores all of Python's basic procedural statements. The next chapter kicks off this topic with an introduction to Python's general syntax model, which is applicable to all statement types. Before moving on, though, take the chapter quiz, and then work through the end-of-part lab exercises to review type concepts. The next part's statements largely just create and process objects, so make sure you've mastered this domain by working through all the exercises before reading on.

中文翻译

本章探索了最后两个主要的核心对象类型——元组和文件。我们学到：元组支持所有常规的序列操作、只有少数几个方法、因不可变而不允许任何就地修改，并被命名元组扩展类型所泛化。我们还学到：文件由内置 `open` 函数返回，并提供读写文本与二进制两类内容的方法。一路走来，我们探索了如何把 Python 对象与字符串互相转换以存入文件，并考察了 `pickle`、`json` 及其它模块的高级角色（对象序列化与二进制数据）。最后，我们通过复习所有对象类型共享的一些性质（例如共享引用）收尾，并过了一遍对象类型领域的常见错误（“陷阱”）清单。在本书下一部分，我们将换挡转向语句语法（statement syntax）——在脚本中编写处理步骤与逻辑的方式。这一部分将探索 Python 的所有基本过程式语句。下一章将以导入 Python 的通用语法模型来开启这个话题——该模型适用于所有语句类型。不过在继续

之前，请先做本章测验，然后完成本部分末尾的动手练习来复习类型概念。下一部分的语句大多只是创建和处理对象，所以在往下读之前，请确保通过完成所有练习掌握好这块领域。

9.37 Test Your Knowledge: Quiz (自测：测验)

英文原文

1. How can you determine how large a tuple is? Why is this tool located where it is? 2. Write an expression that changes the first item in a tuple. (4, 5, 6) should become (1, 5, 6) in the process. 3. What is the default for the processing mode argument in a file open call? 4. What module might you use to store Python objects in a file without converting them to strings yourself? 5. How might you go about copying all parts of a nested structure at once? 6. When does Python consider an object to be true?

中文翻译

1. 你如何确定一个元组有多大？为什么这个工具被放在那个位置？2. 写出一个改变元组中第一个条目的表达式。(4, 5, 6) 在此过程中应变为 (1, 5, 6)。3. 文件 open 调用中处理模式参数的默认值是什么？4. 你可以用什么模块把 Python 对象存储到文件中，而无需自己把对象转成字符串？5. 你要如何一次性拷贝一个嵌套结构的所有部分？6. Python

在什么时候认为一个对象为真？

9.38 Test Your Knowledge: Answers (自测：答案)

英文原文

1. The built-in `len` function returns the length (number of contained items) for any container object in Python, including tuples. It is a built-in function instead of a type method because it applies to many different types of objects. In general, built-in functions and expressions may span many object types; methods are specific to a single object type, though some method names may be available on more than one type (`index`, for example, works on lists and tuples).

2. Because they are immutable, you can't really change tuples in place, but you can generate a new tuple with the desired value. Given `T = (4, 5, 6)`, you can change the first item by making a new tuple from its parts by slicing and concatenating: `T = (1,) + T[1:]`. (Recall that single-item tuples require a trailing comma.) You could also convert the tuple to a list, change it in place, and convert it back to a tuple, but this is more expensive and is rarely required in practice—simply use a list if you know that the object will require in-place changes.

3. The default for the processing mode argument in a file open call is `'r'`, for reading text input. For input t

ext files, simply pass in the external file's name or path (unless you also need to customize things like buffering policies or provide a Unicode text encoding to override your platform's default—as fleshed out in Chapter 37). 4. The pickle module can be used to store Python objects in a file without explicitly converting them to strings. json similarly converts a limited set of Python objects to and from strings per the JSON format. The struct module is related, but it assumes the data is to be in packed binary format in the file. 5. Import the copy module, and call copy.deepcopy(X) if you need to copy all parts of a nested structure X. This is also rarely needed in practice; references are usually the desired behavior, and shallow copies (e.g., aList[:], aDict.copy(), set(aSet)) usually suffice for most copies. 6. An object is considered true if it is either a nonzero number or a nonempty collection object. The built-in words True and False are essentially predefined to have the same meanings as integer 1 and 0, respectively.

中文翻译

1. 内置的 len 函数返回 Python 中任何容器对象的长度（所含条目的数量），包括元组。它是内置函数而非类型方法，是因为它适用于许多不同类型的对象。一般来说，内置函数和表达式可以横跨多种对象类型；方法则特定于单一对象类型——不过某些方法名可能出现在不止一种类型上（例如 index 就能用于列表和元组）。2. 因为元组不可变，你

不能真正就地修改它们，但可以生成一个带有期望值的新元组。给定 $T = (4, 5, 6)$ ，你可以通过切片和拼接从它的组成部分构造一个新元组来改变第一个条目： $T = (1,) + T[1:]$ 。

(回忆：单元素元组需要尾随逗号。) 你也可以把元组转换为列表、就地修改、再转回元组，但这更昂贵且实践中很少需要——如果你知道对象将需要就地修改，直接使用列表即可。3. 文件 `open` 调用中处理模式参数的默认值是 `'r'`，用于读取文本输入。对输入文本文件，只传入外部文件的名称或路径即可（除非你还需要定制缓冲策略之类、或提供 Unicode 文本编码来覆盖平台默认值——详见第 37 章）。4. `pickle` 模块可用于把 Python 对象存储到文件，而无需显式地把它们转成字符串。`json` 类似地按 JSON 格式把有限的一组 Python 对象转成/转出字符串。`struct` 模块相关，但它假定文件中的数据是打包好的二进制格式。5. 如果你需要拷贝一个嵌套结构 X 的所有部分，可以导入 `copy` 模块并调用 `copy.deepcopy(X)`。这实践中也很少需要；引用通常才是想要的行为，浅拷贝（例如 `aList[:]`、`aDict.copy()`、`set(aSet)`）通常足以满足大多数拷贝需求。6. 当对象是非零数字或非空集合对象时，它被认为是真。内置词 `True` 和 `False` 本质上被预定义为与整数 1 和 0 含义相同。

9.39 Test Your Knowledge: Part II Exercises (自测：第二部分练习)

英文原文

This session asks you to get your feet wet with coding built-in object fundamentals. As before, a few new i

ideas may pop up along the way, so be sure to flip to the answers in "Part II, Objects and Operations" in Appendix B when you're done (or even when you're not). If you have limited time, consider starting with exercises 10 and 11 (the most practical of the bunch) and then working from first to last as time allows. This is all fundamental material, though, so try to do as many of these as you can; programming is a hands-on activity, and there is no substitute for practicing what you've read to make ideas gel.

中文翻译

这一节让你动手编写内置对象基础知识的代码，先弄湿脚趾。和以前一样，过程中会冒出一些新想法，所以完成后（甚至没完成时）一定要翻到附录 B 中"第 II 部分，对象与操作"的答案。如果时间有限，建议先从练习 10 和 11（其中最具有实战性的两个）开始，然后有余力就从头往后做。不过这些都是基础材料，尽量多做；编程是动手的活动，亲自动手消化你读到的内容，没有替代品。

1. The basics (基础) : Experiment interactively with the common type operations found in the various operation tables in this part of the book. To get started, bring up the Python interactive interpreter (a REPL of your choosing), type each of the following expressions, and try to explain what's happening in each case. Note that the semicolon in some of these is being used as a statement separator, to squeeze multiple statements onto a single line: for example, `X=1;X` assigns

and then prints a variable (more on statement syntax in the next part of the book). Also remember that a comma between expressions usually builds a tuple, even if there are no enclosing parentheses: `X,Y,Z` is a three-item tuple, which Python prints back to you in parentheses.

```

2 16 2 / 5, 2 / 5.0'hack' + 'code'S = 'Python'
'grok' + S S 5 S[0], S[:0], S[1:] how = 'fun'"coding %s is
%s!" % (S, how)'coding {} is {}!".format(S, how) f'codin
g {S} is {how}!'('x,')[0] ('x','y')[1] L = [1, 2, 3] + [4, 5, 6]
L, L[:], L[:0], L[-2], L[-2:] ([1, 2, 3] + [4, 5, 6])[2:4] [L[2],
L[3]] L.reverse(); L L.sort(); L L.index(4) {'a': 1,'b': 2}['b']
D = {'x': 1,'y': 2,'z': 3} D['w'] = 0 D['x'] + D['w'] D[(1, 2,
3)] = 4 list(D.keys()), list(D.values()), (1, 2, 3) in D [()], [
"", [], (), {}, None] 1 (译文) : 交互式地试验本书这一部分
各操作表中的常见类型操作。先启动 Python 交互式解释器
(你选一个 REPL) , 输入下面每个表达式, 并试着解释每个
例子里发生了什么。注意其中一些分号被用作语句分隔符
, 把多条语句挤到一行上: 例如 X=1;X 先赋值再打印变量
(下一部分会有更多语句语法) 。还要记住: 表达式之间的
逗号通常构成元组, 即使没有括号: X,Y,Z 是三项元组, Py
thon 会加括号打印给你。
2. Indexing and slicing (索引与切片) : At the interactive prompt, define a list nam
ed L that contains four strings or numbers (e.g., L=[0,
1, 2, 3]). Then, experiment with the following bounda
ry cases. You may never see these cases in real progr
ams (especially not in the bizarre ways they may app
ear here!), but they are intended to make you think a
bout the underlying model, and some may be useful i

```

n less artificial forms—slicing out of bounds can help, for example, if a sequence is not as long as you expect: a. What happens when you try to index out of bounds (e.g., `L[4]`)? b. What about slicing out of bounds (e.g., `L[-1000:100]`)? c. Finally, how does Python handle it if you try to extract a sequence in reverse, with the lower bound greater than the higher bound (e.g., `L[3:1]`)? Hint: try assigning to this slice (`L[3:1]='?'`), and see where the value is put. Do you think this may be the same phenomenon you saw when slicing out of bounds?

2 (译文)：在交互提示符下，定义一个名为 `L`、包含四个字符串或数字的列表（例如 `L=[0, 1, 2, 3]`）。然后实验下面的边界情况。你在真实程序里可能永远不会遇到这些情况（尤其是它们在这里出现的怪异形式！），但它们的目的是让你思考底层模型，而且有些情况在没那么人工的形式下可能有用——例如，如果序列没有你预期那么长，越界切片可以帮上忙：a. 尝试越界索引（例如 `L[4]`）时会发生什么？b. 越界切片（例如 `L[-1000:100]`）又会怎样？c. 最后，如果你尝试反向提取序列、下界大于上界（例如 `L[3:1]`），Python 如何处理？提示：试着给这个切片赋值（`L[3:1]='?'`），看看值被放到哪里。你觉得这可能是你在越界切片时看到过的同一现象吗？

3. Indexing, slicing, and del: Define another list `L` with four items, and assign an empty list to one of its offsets (e.g., `L[2]=[]`). What happens? Then, assign an empty list to a slice (`L[2:3]=[]`). What happens now? Recall that slice assignment deletes the slice and inserts the new value where it used to be. The `del` statement deletes offsets, keys, att

tributes, and names. Use it on your list to delete an item (e.g., `del L[0]`). What happens if you delete an entire slice (`del L[1:]`)? What happens when you assign a non-sequence to a slice (`L[1:2]=1`)? 3 (译文) : 再定义一个含四个条目的列表 L, 把一个空列表赋给它的某个偏移量 (例如 `L[2]=[]`)。会发生什么? 然后把空列表赋给一个切片 (`L[2:3]=[]`)。现在又怎样? 回忆: 切片赋值会删除该切片, 并把新值插到它原来的位置。`del` 语句删除偏移量、键、属性和名字。在你的列表上用它删除一个条目 (例如 `del L[0]`)。如果删除整个切片 (`del L[1:]`) 会怎样? 当把一个非序列赋给切片 (`L[1:2]=1`) 又怎样? 4. Tuple assignment (元组赋值) : Type the following lines: `>>> X='code'>>> Y='hack'>>> X, Y = Y, X` What do you think is happening to X and Y when you type this sequence? 4 (译文) : 输入下面几行: `X='code'、Y='hack'、X, Y = Y, X`。当你输入这段序列时, 你认为 X 和 Y 正在发生什么? 5. Dictionary keys (字典键) : Consider the following code fragments: `>>> D = {}>>> D[1] = 'a'>>> D[2] = 'b'` You've learned that dictionaries aren't accessed by offsets, so what's going on here? Does the following shed any light on the subject? (Hint: strings, integers, and tuples share which type category?) `>>> D[(1, 2, 3)] = 'c'>>> D {1:'a', 2:'b', (1, 2, 3):'c'}` 5 (译文) : 考虑以下代码片段: `D={}, D[1]='a'、D[2]='b'`。你学过字典不是按偏移访问的, 那么这里发生了什么? 下面这段是否点明了问题? (提示: 字符串、整数和元组共享哪个类型类别? ——都是不可变的, 可作为哈希键!) 6. Dictionary indexing (字典索引) : Create a dictionary named

D with three entries, for keys 'a', 'b', and 'c'. What happens if you try to index a nonexistent key (D['d'])? What does Python do if you try to assign to a nonexistent key 'd' (e.g., D['d']='hack')? How does this compare to out-of-bounds assignments and references for lists? Does this sound like the rule for variable names? 6 (译文) : 创建一个名为 D、有三个条目 (键'a'、'b'、'c') 的字典。如果索引不存在的键 (D['d']) 会发生什么? 如果给不存在的键'd' 赋值 (如 D['d']='hack') Python 会做什么? 这与列表的越界赋值和越界引用相比如何? 这听起来像变量名的规则吗? (增键自动、取键失败——与"未定义变量"的哲学一致。) 7. Generic operations (通用操作) : Run interactive tests to answer the following questions: a. What happens when you try to use the + operator on different/mixed types (e.g., string + list, list + tuple)? b. Does + work when one of the operands is a dictionary? c. Does the append method work for both lists and strings? How about using the keys method on lists? (Hint: what does append assume about its subject object?) d. Finally, what type of object do you get back when you slice or concatenate two lists or two strings? 7 (译文) : 通过交互测试回答下面问题: a. 对不同/混合类型使用 + 运算符 (例如字符串 + 列表、列表 + 元组) 时会发生什么? b. 当其中一个操作数是字典时, + 还能用吗? c. append 方法对列表和字符串都适用吗? 对列表使用 keys 方法呢? (提示: append 对其主体对象做了何种假设? ——它假定主体是可变的序列!) d. 最后, 当你切片或拼接两个列表/两个字符串时, 得到的是什么

类型的对象？（与什么类型一致——因为切片/拼接保留原类型。） 8. String indexing（字符串索引）：Define a string `S` of four characters: `S = 'hack'`. Then type the following expression: `S[0][0][0][0][0]`. Any clue as to what's happening this time? (Hint: recall that a string is a collection of characters, but Python characters are one-character strings.) Does this indexing expression still work if you apply it to a list such as `['h','a','c','k']`? Why? 8（译文）：定义一个四字符的字符串 `S`：`S = 'hack'`。然后输入表达式 `S[0][0][0][0][0]`。这次到底发生了什么，你有线索吗？（提示：字符串是字符的集合，但 Python 的字符就是单字符字符串——所以每次索引都返回单字符字符串，还能再索引。） 9. Immutable types（不可变类型）：Define a string `S` of four characters again: `S = 'hack'`. Write an assignment that changes the string to `'heck'`, using only slicing and concatenation. Could you perform the same operation using just indexing and concatenation? How about index assignment? 9（译文）：再定义一个四字符的字符串 `S`：`S = 'hack'`。只使用切片和拼接，写出一个把该字符串改为 `'heck'` 的赋值。只用索引和拼接能做到同样的操作吗？索引赋值呢（不行——字符串不可变）？ 10. Nesting（嵌套）：Write a data structure that represents your personal information: name (first, middle, last), age, job, address, email address, and phone number. You may build the data structure with any combination of built-in object types you like (lists, tuples, dictionaries, strings, numbers). Then, access the individual components of your data struc

tures by indexing. Do some structures make more sense than others for this object? 10 (译文) : 写一个表示你个人信息的数据结构: 姓名(名、中间名、姓)、年龄、职业、地址、电子邮件地址和电话号码。你可以用任意组合的内置对象类型(列表、元组、字典、字符串、数字)来构建它。然后用索引访问数据结构的各个组成部分。对这个对象而言, 某些结构会不会比其它结构更合理? 11. Files (文件) : Write a script that creates a new output file called myfile.txt and writes the string 'Hello file world!' into it. Then write another script that opens myfile.txt and reads and prints its contents. Run your two scripts from the system command line (or other script-launcher tool available to you). Does the new file show up in the directory where you ran your scripts? What if you add a different directory path to the filename passed to open? Note: file write methods do not add newline characters to your strings; add an explicit \n at the end of the string if you want to fully terminate the line in the file. 11 (译文) : 写一个创建名为 myfile.txt 的新输出文件、并把字符串 'Hello file world!' 写进去的脚本。再写另一个打开 myfile.txt、读取并打印其内容的脚本。在系统命令行(或你手头可用的其它脚本启动工具)里运行你的两个脚本。新文件会出现在你运行脚本的目录里吗? 如果传给 open 的文件名加上一个不同的目录路径呢? 注意: 文件 write 方法不会给你的字符串添加换行符; 如果你想让行在文件中完整终止, 就在字符串末尾加上显式的 \n。

深度理解

· 练习题的设计意图：这 11 道题把本章与第二部分的全部要点转成“手能摸到的操作”——从字面量与操作（1）、切片边界（2-3）、隐式元组与交换（4）、哈希键与自动插入（5-6）、类别操作差异（7）、字符即单字符字符串（8）、不可变重构（9）、嵌套建模（10）到文件落地（11）。

· 答题密钥：遇到“为什么报错”先回到三条模型——引用共享、不可变即新建、类别决定操作。例如 ('x')[0] 取的是单字符字符串'x'；[L[2], L[3]] 是新列表；D[(1,2,3)]=4 说明元组是可哈希键。

· 进阶建议：练习 10 的“个人信息结构”值得把 dict + tuple + list 三个方案各写一遍对比可读性；练习 11 完成后用 with open 重写一遍，体会资源管理。做完再去对照附录 B 答案。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

· 元组 (tuple)：记录行、函数多返回值、字典键与集合元素、% 格式化替换值、不可变配置（如 COLORS = ('red','green')）。namedtuple 用于“轻量数据类”字段语义

- 化（如账单明细、API 响应字段），它不可变、可哈希、解包方便，是 dataclass 出现前的首选。
- 文件（file）：日志落盘、配置读写、数据导入导出、大文件逐行流式处理（for line in f）、临时文件、协议数据读写（struct）与跨语言交换（JSON/CSV）。
 - 序列化三件套的选择：数据仅在本进程用 → pickle（快、全类型）；数据要出 Python 世界 → json（文本、跨语言）；自定义紧凑二进制协议 → struct；表格数据 → CSV。
 - with open 是现代文件代码的默认风格——它把"关文件"这个易错动作从程序员肩上卸下，配合 encoding= 显式声明编码，几乎可以消灭一整族文件类 bug。

与其他语言（Java/C++）的区别

主题	Python	Java / C++
元组存在性	tuple 是内建核心类型	Java 无（须用 Pair 类 /record 17+）；C++ 用 std::tuple
字符串可变性	str 不可变，bytes 也有立法	C++ std::string 可变；Java String 不可变
文件打开	内置 open()，文本/二进制用模式区分	各自 I/O 库（Files.newBufferedReader、std::ifstream）；无自动编码默认约定
资源释放	with 上下文管理器（确定性），另有 CPython 引用计数兜底	Java try-with-resources；C++ RAII（析构即释放，最接近）
跨类型比较	非数值混合大小比较直接抛 TypeError	C++ 隐式数值提升宽泛得多；Java 基本类型可比较、对象用 Comparable

对象拷贝	= 是引用, 拷贝需显式 ([:].copy()/deepcopy)	Java 引用赋值; clone () 浅拷贝; C++ = 在值语义下是拷贝 (易意外深复制!)
空值占位	单例 None, is 判空	Java/C++ 的 null/NULL (C++ 中 nullptr 无类型)
"一等对象"	类、函数、类型都是对象	C++ 函数指针有但难用; Java 方法引用是 8+ 的补充

Python 发展历史背景

- tuple 一词借自数学/关系数据库"行"; 任意位置解包 (PEP 448, Python 3.5) 让元组字面量的解包能力焕然一新, 本书正文的解包示例正是它的演示。
- 文件对象早年在 Python 2 是单一 file 类型, Python 3 细化出 io 层 (TextIOWrapper/BufferedReader...), 并把 bytes 与 str 严格分离——这是"文本/二进制模式"清晰化的根源。
- with 语句出自 PEP 343 (Python 2.5), 来自 C# 的 using 语意与 Java try-with-resources 的同类思想; 它是 Python 从"靠实现细节 (引用计数)"走向"规范性保证"的关键一步。
- 字典比较禁 >/< (Python 3 移除 cross-type ordering) 源于 Python 2 时代"按类型名排序"的混乱, 属于刻意修复的语义缺陷。

高级开发者需要掌握的相关知识

- 字节视图: bytes/bytearray/memoryview 三件套与 mmap、os.read、struct、array 的组合, 是写高性能二进

制工具（协议解析、编解码器）的基础。

·编码全景：encoding/errors/newline 参数的完整语义、BOM、codecs 模块、pathlib 的 Path.open——第 3 7 章的主题，值得提前预习。

·对象协议：bool/len（真值）、eq/lt（比较）、enter/exit（with）、getitem/iter（序列）——理解了这些，才能写出“像内置类型”的自定义类。

·序列化边界：pickle 的协议版本与安全风险、JSON 的 default= 扩展与 round-trip 陷阱、yaml/shelve/数据库的取舍——真实系统的大半崩溃都发生在序列化这一层。

·内存语义：引用计数 vs 垃圾回收、弱引用、sys.getrefcount、循环引用的 GC——它们是“共享引用/拷贝”陷阱的深层答案。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5)

·元组与文件是最后两块核心类型拼图；“对象通用性质”（比较/相等/拷贝/真值）是本书第二部分的题眼，也是第三部分语句学习的前提——不理解“对象”，就写不好任何语句。

·应该掌握到什么程度：

1. 脱口而出：(x,) vs (x)；== vs is；真值测试（空即假、0 即假、None 恒假）；write 不加换行、readline 以“报 EOF；列表/元组/字典/集合的类别与可变性归属。

2. 动手可写：用 `with open` 读写文本与二进制、`dump/load pickle` 与 `JSON`、用 `[:].copy()/deepcopy` 控制拷贝深度、用 `sorted(..., key=...)` 优雅解决混合类型排序。

3. 知其所以然：解释为什么 `[]4` 是坑、为什么 `'text' is 'text'` 可能为 `True`、为什么 `D1 < D2` 报错、为什么 `with` 比 "自动关闭" 更可靠。

·后续应该学习哪些相关内容：

·第 10 章起进入语句与语法 (`if/for/while` 正是操作本章对象的"动词")，第 11 章深入赋值与序列赋值，第 12 章布尔逻辑，第 14 章迭代器与推导式的正式定义。

·第 34 章 `with/try` 完整机制，第 37 章 `Unicode` 与文件完整扩展（第 9.12 节埋的线到这里收束）。

·第 27-32 章关于类的部分会再次引用第 9.19 节的"运算符重载"与第 9.22 节的"类可自定义比较方式"。建议做完全部 11 道 Part II 练习、对照附录 B 答案后再进入第三部分。